键落星辰

ACM 算法竞赛模板

定稿时间: 2026年9月

队员: 许嘉昊

郭金赓

曾逸轩

Contents

1. 动态规划

1.1 四边形不等式与树形背包优化

```
// a ≤ b ≤ c ≤ d : w(b,c) ≤ w(a,d),
  ↪ w(a,c) + w(b,d) ≤ w(a,d) + w(b,c)
for (int len = 2; len <= n; ++len) {
  for (int l = 1, r = len; r <= n; ++l, ++r) {
    f[l][r] = INF;
    for (int k = m[l][r - 1]; k <= m[l + 1][r]; ++k) {
      if (f[l][r] > f[l][k] + f[k + 1][r] + w(l, r)) {
        f[l][r] = f[l][k] + f[k + 1][r] + w(l, r);
        m[l][r] = k;
} } }
```

1.1.1 树形背包优化

限制: 必须取与根节点相连的一个连通块.
转化: 一个点的子树对应于DFS序中的一个区间. 则每个点的决策为, 取该点, 或者舍弃该点对应的区间. 从后往前dp, 设 $f(i,v)$ 表示从后往前考虑到i号点, 总体积为V的最优价值, 设i号点对应的区间为 $[i, i+\text{size}_i-1]$, 转移为 $f(i,v)=\max\{f(i+1,V-v_i)+w_i, f(i+\text{size}_i, v)\}$.
如果要求任意连通块, 则点分治后转为指定根的连通块问题即可.

1.2 $O(n \cdot \max a_i)$ Subset Sum

```
int SubsetSum(vector <int> &a, int t) {
  int B = *max_element(a.begin(), a.end());
  int n = (int) a.size(), s = 0, i = 0;
  while (i < n && s + a[i] <= t) s += a[i ++];
  if (i == n) return s;
  vector <int> f(2 * B + 1, -1), pre (B + 1, -1);
  f[s - (t - B)] = i;
  for ( ; i < n; i++) {
  s += a[i];
  for (int d = 0; d <= B; d++) pre[d] = max(0, f[d + B]);
  for (int d = B; d >= 0; d--)
    f[d + a[i]] = max(f[d + a[i]], f[d]);
  for (int d = 2 * B; d > B; --d)
    for (int j = pre[d - B]; j < f[d]; j++)
    f[d - a[j]] = max(f[d - a[j]], j); }
  for (i = 0; i <= B; i++) if (f[B - i] >= 0) return t - i;}
```

2. 字符串

2.1 最小表示法

```
int minimal_string(){// 下标从0开始
  int k = 0, i = 0, j = 1;
  while(k < n && i < n && j < n){
    if(a[(i+k)%n] == a[(j+k)%n]) {k++; continue;}
    if(a[(i+k)%n] > a[(j+k)%n]) i = i+k+1;
    else j = j+k+1;
    i += (i == j); k = 0; }
  return min(i, j); }
```

2.2 Manacher

```
void read(){ // 两倍空间
  s[0] = '~', s[++n] = '|';
  char c = getchar();
  while(c < 'a' || c > 'z') c = getchar();
  while('a' <= c && c <= 'z') s[++n] = c, s[++n] = '|', c =
    ↪ getchar();
}
void manacher(){
  int mx = 0, mid = -1, ans = 0;
  for(int i=1;i<=n;i++){
    if(i <= mx) h[i] = min(h[(mid<<1)-i], mx-i+1);
    else h[i] = 1;
    while(s[i-h[i]] == s[i+h[i]]) ++h[i];
    if(h[i] + i - 1 > mx) mx = h[i] + i - 1, mid = i; } }
```

2.3 KMP, exKMP

```
void kmp(const char *s, int n) { // 1-based
  fail[0] = fail[1] = 0;
  for (int i = 1; i < n; i++) { int j = fail[i];
    while (j && s[i + 1] != s[j + 1]) j = fail[j];
    if (s[i + 1] == s[j + 1]) fail[i + 1] = j + 1;
    else fail[i + 1] = 0; } }
void exkmp(const char *s, int *a, int n) { // 1-based
  int l = 0, r = 0; a[1] = n;
  for (int i = 2; i <= n; i++) {
    a[i] = i > r ? 0 : min(r - i + 1, a[i - l + 1]);
    while (i+a[i] <= n && s[1+a[i]] == s[i+a[i]]) a[i]++;
    if (i + a[i] - 1 > r) {l = i; r = i + a[i] - 1;}}}
```

```
void init(){
  int l = 0, r = 0;
  z[1] = 0;
  for(int i=2;i<=n;i++){
    if(i <= r && z[i-l+1] < r-i+1) z[i] = z[i-l+1];
    else {
      z[i] = max(0, r-i+1);
      while(i+z[i]<=n && b[i+z[i]]==b[z[i]+1]) z[i]++;
    }
    if(i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
  }
}
```

2.4 AC 自动机

注意代码是以 0 为根的，如果要 1-base 的话要改一下没有儿子时的逻辑。

```
struct AC{
int ch[N][26], fail[N], tag[N], tot;
void ins(string s, int _idx){
  int x = 0;
  for(auto c : s){
    int v = c - 'a';
    if(!ch[x][v]) ch[x][v] = ++tot;
    x = ch[x][v]; } tag[x] = _idx; }
void build(){
  queue<int> q;
  for(int i=0;i<26;i++) if(ch[0][i]) q.push(ch[0][i]);
  while(!q.empty()){
    int x = q.front(); q.pop();
    for(int i=0;i<26;i++) {
      if(ch[x][i]) fail[ch[x][i]] = ch[fail[x]][i],
        ↪ q.push(ch[x][i]);
      else ch[x][i] = ch[fail[x]][i]; } } }
} ac;
```

2.5 Lydon Word Decomposition

```
//满足s的最小后缀等于s本身的串s称为Lyndon串.
//等价于: s是它自己的所有循环移位中唯一最小的一个.
//任意字符串s可以分解为 s = s1s2sk，其中 si 是Lyndon串，
  ↪ si ≥ si+1. 且这种分解方法是唯一的.
//后缀排序后，排名的所有前缀最小值构成了 Ly 分解的左端点。
void mnsuf(char *s, int *mn, int n){ // 每个前缀的最小后缀
  //1 - base , 求 Lyndon 分解去掉 mn 即可
  for(int i = 1; i <= n;)
    int j = i + 1, k = i; mn[i] = i;
    for(; j <= n && s[k] <= s[j]; j++){
      if(s[k] < s[j]) k = mn[j] = i;
        else mn[j] = mn[k] + j - k, k++;
    for(; i <= k; i += j - k) {} } } //lyn+=s[i..i+k-j-1]
void mxsuf(char *s, int *mx, int n){ // 每个前缀的最大后缀
  fill(mx + 1, mx + n + 1, 0); // 1 - base
  for(int i = 1; i <= n;){
    int j = i + 1, k = i; !mx[i] ? mx[i] = i : 0;
    for(; j <= n && s[k] >= s[j]; j++){
      !mx[j] ? mx[j] = i : 0;
      s[k] > s[j] ? k = i : k++; }
    for(; i <= k; i += j - k) {} } }
```

2.6 后缀自动机

```
struct SAM{ // 别忘记空间是两倍 改字符集记得全改了
  int last = 1, tot = 1;
  int ch[MAXN<<1][26], len[MAXN<<1], f[MAXN<<1];
  void ins(char c){
    c -= 'a';
    int p = last, cur = last = ++tot;
    len[cur] = len[p] + 1;
    for(;p && !ch[p][c];p=f[p]) ch[p][c] = cur;
    if(!p) {f[cur] = 1; return;}
    int q = ch[p][c];
    if(len[q] == len[p] + 1) {f[cur] = q; return;}
    int clone = ++tot;
    for(int i=0;i<26;i++) ch[clone][i] = ch[q][i];
    f[clone] = f[q], len[clone] = len[p] + 1;
    f[q] = f[cur] = clone;
    for(;p && ch[p][c] == q;p=f[p]) ch[p][c] = clone;
  }
} sam;
```

2.7 广义后缀自动机

```
struct SAM{
  int ch[MAXN<<1][26], f[MAXN<<1], len[MAXN<<1], tot = 1;
```

```
 | int ins(int c, int last){
 | | c -= 'a';
 | | int cur, p;
 | | if(ch[last][c]){
 | | | p = last, cur = ch[p][c];
 | | | if(len[p] + 1 == len[cur]) return cur;
 | | | int q = cur, clone = ++tot;
 | | | for(int i=0;i<26;i++) ch[clone][i] = ch[q][i];
 | | | len[clone] = len[p] + 1;
 | | | f[clone] = f[q], f[q] = clone;
 | | | for(;p&&ch[p][c]==q;p=f[p]) ch[p][c] = clone;
 | | | return clone;
 | | }
 | | p = last, cur = ++tot;
 | | len[cur] = len[p] + 1;
 | | for(;p&&!ch[p][c];p=f[p]) ch[p][c] = cur;
 | | if(!p) {f[cur] = 1; return cur;}
 | | int q = ch[p][c];
 | | if(len[p] + 1 == len[q]) {f[cur] = q; return cur;}
 | | int clone = ++tot;
 | | for(int i=0;i<26;i++) ch[clone][i] = ch[q][i];
 | | len[clone] = len[p] + 1;
 | | f[clone] = f[q], f[q] = f[cur] = clone;
 | | for(;p&&ch[p][c]==q;p=f[p]) ch[p][c] = clone;
 | | return cur;
 | }
 | void ins(char* s){
 | | int last = 1;
 | | for(int i=0;s[i];i++) last = ins(s[i], last);
 | }
} sam;
```

2.8 SAMSA & 后缀树

```
bool vis[MAXN * 2]; char s[MAXN];
int id[MAXN * 2], ch[MAXN * 2][26], height[MAXN], stamp = 0;
void dfs(int x) {
 | if (id[x]) { height[stamp++] = val[last];
 | | sa[stamp] = id[x]; last = x; }
 | for (int c = 0; c < 26; c++)
 | | if (ch[x][c]) dfs(ch[x][c]);
 | last = par[x]; }
int main() { last = ++cnt; scanf("%s", s + 1);
 | int n = strlen(s + 1); for (int i = n; i; i--) {
 | | expand(s[i] - 'a'); id[last] = i; }
 | vis[1] = true; for (int i = 1; i <= cnt; i++) if (id[i])
 | | | for (int x = i,pos = n; x && !vis[x]; x = par[x]){
 | | | | vis[x] = true; pos -= val[x] - val[par[x]];
 | | | | ch[par[x]][s[pos + 1] - 'a'] = x; }
 | dfs(1); for (int i = 1; i <= n; i++)
 | | printf("%d%c", sa[i], i < n ? ' ' : '\n');
 | for (int i = 1; i < n; i++) printf("%d%c", height[i],
 | | i < n ? ' ' : '\n'); return 0; }
```

2.9 后缀数组

```
// height[i] = lcp(sa[i], sa[i - 1])
// 如果有多组数据，全部都清空到 max(n, m)
constexpr int MAXN = 1000005;
void get_sa(char *s, int n, int *sa,
 | | int *rnk, int *height) { // 1-based
 | static int buc[MAXN], id[MAXN], p[MAXN], t[MAXN];
 | int m = 300;
 | for (int i = 1; i <= n; i++) buc[rnk[i] = s[i]]++;
 | for (int i = 1; i <= m; i++) buc[i] += buc[i - 1];
 | for (int i = n; i; i--) sa[buc[rnk[i]]--] = i;
 | memset(buc, 0, sizeof(int) * (m + 1));
 | for (int k = 1, cnt = 0; cnt != n; k *= 2, m = cnt) {
 | | cnt = 0;
 | | for (int i = n; i > n - k; i--) id[++cnt] = i;
 | | for (int i = 1; i <= n; i++)
 | | | if (sa[i] > k) id[++cnt] = sa[i] - k;
 | | for (int i = 1; i <= n; i++) buc[p[i]=rnk[id[i]]]++;
 | | for (int i = 1; i <= m; i++) buc[i] += buc[i - 1];
 | | for (int i = n; i; i--) sa[buc[p[i]]--] = id[i];
 | | memset(buc, 0, sizeof(int) * (m + 1));
 | | memcpy(t, rnk, sizeof(int) * (n + 1));
 | | t[n + 1] = 0; // 记得清空 n + 1
 | | cnt = 0; for (int i = 1; i <= n; i++) {
 | | | if (t[sa[i]] != t[sa[i - 1]] ||
 | | | | t[sa[i] + k] != t[sa[i - 1] + k]) cnt++;
 | | | rnk[sa[i]] = cnt; } }
 | for (int i = 1; i <= n; i++) sa[rnk[i]] = i;
 | for (int i = 1, k = 0; i <= n; i++) { if (k) k--;
 | | if (rnk[i] > 1) while (sa[rnk[i] - 1] + k <= n &&
 | | | | | s[i + k] == s[sa[rnk[i] - 1] + k]) k++;
 | | height[rnk[i]] = k; } } // 两个都要判，否则会左/右越界
char s[MAXN]; int sa[MAXN], rnk[MAXN], height[MAXN];
int main() { scanf("%s", s + 1); int n = strlen(s + 1);
 | get_sa(s, n, sa, rnk, height); }
```

2.10 回文树

0 的子树是长为偶数的串, 1 的子树是长为奇数的. 0 代表空串, 1 没有意义.

```
struct PAM{ // sum[i]: 以i结尾的回文串的个数
 | int ch[N][26], len[N], fail[N], tot = 1, now = 1, sum[N];
 | void build(){
 | | fail[0] = 1, len[1] = -1;
 | | s[0] = '#';
 | | for(int i=1;s[i];i++){
 | | | int j, v = (s[i]-'a');
 | | | while(s[i-len[now]-1] != s[i]) now = fail[now];
 | | | if(!ch[now][v]){
 | | | | len[++tot] = len[now] + 2;
 | | | | j = fail[now];
 | | | | while(s[i-len[j]-1] != s[i]) j = fail[j];
 | | | | fail[tot] = ch[j][v];
 | | | | ch[now][v] = tot;
 | | | | sum[tot] = sum[fail[tot]] + 1;
 | | | }
 | | | now = ch[now][v];
 | | }
 | }
} pam; // 定理: 长度为n的字符串S最多有n个本质不同回文串
```

2.11 字符串 Hash 类

Random primes generated at Fri Aug 28 20:59:51 2026
1e12 992345235311 1023452349389 1034563455703 1045674566711
1e13 9956785678283 10123412340527 10345634568167 10456745671051
2e13 19123412346833 20123412342739 20345634564811 20345634566399
1e15 1012341234122513 1012341234125971 1023452345237339
1e17 99456745674569233 99567856785674471 105678567856781981
1e18 993456345634564303 1012341234123415363 1023452345234524169

```
static constexpr u128 inv = []() {
 | u128 ret = P;
 | for (int i = 0; i < 6; i++) ret *= 2 - ret * P;
 | return ret; }();
constexpr u128 chk = u128(-1) / P;
bool check(i128 a, i128 b) { // 注意: i128
 | if (a < b) swap(a, b);
 | return (a - b) * inv <= chk; }
```

2.12 String Conclusions

双回文串

如果 $s = x_1x_2 = y_1y_2 = z_1z_2, |x_1| < |y_1| < |z_1|, x_2, y_1, y_2, z_1$ 是回文串, 则 x_1 和 z_2 也是回文串.

Border 和周期

如果 r 是 S 的一个border, 则 $|S| - r$ 是 S 的一个周期.
如果 p 和 q 都是 S 的周期, 且满足 $p + q \leq |S| + gcd(p, q)$, 则 $gcd(p, q)$ 也是一个周期.

字符串匹配与Border

若字符串 S, T 满足 $2|S| \geq |T|$, 则 S 在 T 中所有匹配位置成等差数列.
若 S 的匹配次数大于2, 则等差数列的周期恰好等于 S 的最小周期.

Border 的结构

字符串 S 的所有不小于 $|S|/2$ 的border长度组成一个等差数列.
字符串 S 的所有 border 按长度排序后可分成 $O(\log |S|)$ 段, 每段是一个等差数列.

回文串Border

回文串长度为 t 的后缀是一个回文后缀, 等价于 t 是该串的border. 因此回文后缀的长度也可以划分成 $O(\log |S|)$ 段.

子串最小后缀

设 $s[p..n]$ 是 $s[i..n], (l \leq i \leq r)$ 中最小者, 则minsuf(l, r) 等于 $s[p..r]$ 的最短非空 border. minsuf(l, r) = $\min\{s[p..r], \text{minsuf}(r - 2^k + 1, r)\}$, $(2^k < rl + 1 \leq 2^{k+1})$.

子串最大后缀

从左往右扫, 用set维护后缀的字典序递减的单调队列, 并在对应时刻添加"小于事件" 点以便在之后修改队列; 查询直接在set里lower_bound.

3. 数论（不含博弈论）

3.1 Long Long $O(1)$ 乘, Barrett

```cpp
LL modmul(LL a, LL b, LL M) { // skip2004, M < 63bit
  LL ret = a * b - M * LL(1.L * a / M * b + 0.5);
  return ret < 0 ? ret + M : ret; }
ULL modmul(ULL a, ULL b, LL M) { // orz@CF, M in 63 bit
  ULL c = (long double)a * b / M;
  LL ret = LL(a * b - c * M) % LL(M); // must be signed
  return ret < 0 ? ret + M : ret; }
// use int128 instead if M > 63 bit
struct DIV {
  ULL p, ip;
  void init (ULL _p) { p = _p; ip = -1llu / p; }
  int mod (ULL x) { // x < 2 ^ 64
    ULL q = ULL(((u128)ip * x) >> 64);
    ULL r = x - q * p;
    return int(r >= p ? r - p : r);
} }; // speedup only when mod is not const
```

3.2 exgcd, 逆元

```cpp
LL exgcd(LL a, LL b, LL &x, LL &y) {
  if (b == 0) return x = 1, y = 0, a;
  LL t = exgcd(b, a % b, y, x);
  y -= a / b * x; return t;}
LL inv(LL x, LL m) {
  LL a, b; exgcd(x, m, a, b); return (a % m + m) % m; }
```

递推逆元: $\text{inv}(i) \equiv (P - P/i) \cdot \text{inv}(P \bmod i)$

3.3 CRT 中国剩余定理

```cpp
bool crt_merge(LL a1, LL m1, LL a2, LL m2, LL &A, LL &M) {
LL c = a2 - a1, d = __gcd(m1, m2); //合并两个模方程
if(c % d) return 0; // gcd(m1, m2) | (a2 - a1) 时才有解
c = (c % m2 + m2) % m2; c /= d; m1 /= d; m2 /= d;
c = c * inv(m1 % m2, m2) % m2; //0逆元可任意值
M = m1*m2*d; A = (c *m1 %M *d %M +a1) % M; return 1;}//有解
```

3.4 Miller Rabin, Pollard Rho

```cpp
mt19937_64 rng(123);
#define rand() (LL)(rng() & LLONG_MAX)
const LL BASE[] = {2, 7, 61};//int(7,3e9)
//{2,325,9375,28178,450775,9780504,1795265022}LL(37)
struct miller_rabin {
bool check (LL M, LL base) {
  LL a = M - 1;
  while (~a & 1) a >>= 1;
  LL w = qpow (base, a, M); // qpow should use mul
  for (; a != M - 1 && w != 1 && w != M - 1; a <<= 1)
    w = mul (w, w, M);
  return w == M - 1 || (a & 1) == 1; }
bool solve (LL a) {//O((3 or 7) · log n · mul)
  if (a < 4) return a > 1;
  if (~a & 1) return false;
  for (int i = 0; i < sizeof(BASE)/sizeof(BASE[0]); ++i) {
      LL chk = BASE[i] % a;
      if (chk == 0) return true;
    if (!check (a, chk)) return false;
  }
  return true; } };
miller_rabin is_prime;
LL get_factor (LL a, LL seed) {//O(n^{1/4} · log n · mul)
  LL x = rand() % (a - 1) + 1, y = x;
  for (int head = 1, tail = 2; ; ) {
    x = mul (x, x, a); x = (x + seed) % a;
    if (x == y) return a;
    LL ans = gcd (abs (x - y), a);
    if (ans > 1 && ans < a) return ans;
    if (++head == tail) { y = x; tail <<= 1; } } }
void factor (LL a, vector<LL> &d) {
  if (a <= 1) return;
  if (is_prime.solve (a)) d.push_back (a);
  else {
    LL f = a;
    for (; f >= a; f = get_factor (a, rand() % (a - 1) + 1));
    factor (a / f, d);
    factor (f, d); } }
```

3.5 扩展卢卡斯

```cpp
int l,a[33],p[33],P[33];
U fac(int k,LL n){// 求 n! mod pk^tk, 返回值 U{ 不包含 pk 的值 ,pk 出现的次数 }
  if (!n)return U{1,0};LL x=n/p[k],y=n/P[k],ans=1;int i;
  if(y){// 求出循环节的答案
    for(i=2;i<P[k];i++)if(i%p[k])ans=ans*i%P[k];
    ans=Pw(ans,y,P[k]);
  }for(i=y*P[k];i<=n;i++) if(i%p[k])ans=ans*i%M;// 求零散部分
  U z=fac(k,x);return U{ans*z.x%M,x+z.z};
}LL get(int k,LL n,LL m){// 求 C(n,m) mod pk^tk
  U a=fac(k,n),b=fac(k,m),c=fac(k,n-m);// 分三部分求解
  return Pw(p[k],a.z-b.z-c.z,P[k])*a.x%P[k]*inv(b.x,P[k])%P[k]*inv(c.x,P[k])%P[k];
}LL CRT(){// CRT 合并答案
  LL d,w,y,x,ans=0;
  fr(i,1,l)w=M/P[i],exgcd(w,P[i],x,y),ans=(ans+w*x%M*a[i])%M;
  return (ans+M)%M;
}LL C(LL n,LL m){// 求 C(n,m)
  fr(i,1,l)a[i]=get(i,n,m);
  return CRT();
}LL exLucas(LL n,LL m,int M){
  int jj=M,i //求 C(n,m)mod M,M=prod(pi^ki), O(pi^kilg^2n)
  for(i=2;i*i<=jj;i++)if(jj%i==0)
    for(p[++l]=i,P[l]=1;jj%i==0;P[l]*=p[l])jj/=i;
  if(jj>1)l++,p[l]=P[l]=jj;
  return C(n,m);}
```

3.6 阶乘取模

```cpp
// n! mod p^q Time : O(pq^2 log^2 n / log p)
// Output : {a, b} means a*p^b
using Val=unsigned long long; //Val 需要 mod p^q 意义下 + *
typedef vector<Val> poly;
poly polymul(const poly &a,const poly &b){
  int n = (int) a.size(); poly c (n, Val(0));
  for (int i = 0; i < n; ++ i) {
    for (int j = 0; i + j < n; ++ j) {
      c[i + j] = c[i + j] + a[i] * b[j]; } }
  return c; } Val choo[70][70];
poly polyshift(const poly &a, Val delta) {
  int n = (int) a.size(); poly res (n, Val(0));
  for (int i = 0; i < n; ++ i) { Val d = 1;
    for (int j = 0; j <= i; ++ j) {
      res[i - j] = res[i - j]+a[i]*choo[i][j]*d;
      d = d * delta; } } return res; }
void prepare(int q) {
  for (int i = 0; i < q; ++ i) { choo[i][0] = Val(1);
    for (int j = 1; j <= i; ++ j)
      choo[i][j]=choo[i-1][j-1]+choo[i-1][j]; } }
pair<Val, LL> fact(LL n, LL p, LL q) { Val ans = 1;
  for (int r = 1; r < p; ++ r) {
    poly x (q, Val(0)), res (q, Val(0));
    res[0] = 1; LL _res = 0; x[0] = r; LL _x = 0;
    if (q > 1) x[1] = p, _x = 1; LL m = (n - r + p) / p;
    while (m) { if (m & 1) {
        res=polymul(res,polyshift(x,_res)); _res+=_x; }
      m >>= 1; x = polymul(x, polyshift(x, _x)); _x+=_x; }
    ans = ans * res[0]; }
  LL cnt = n / p; if (n >= p) { auto tmp=fact(n / p, p, q);
    ans = ans * tmp.first; cnt += tmp.second; }
  return {ans, cnt}; }
```

3.7 类欧几里得直线下格点统计

```cpp
// sum_{i=0}^{n-1} floor((a+bi)/m), n, m, a, b > 0
LL solve(LL n, LL a, LL b, LL m){
  if (b == 0) return n * (a / m);
  if (a >= m) return n * (a / m) + solve(n, a % m, b, m);
  if (b >= m) return (n-1)*n/2*(b/m) + solve(n,a,b%m,m);
  return solve((a + b * n) / m, (a + b * n) % m, m, b); }
```

3.8 万能欧几里德

```cpp
Val work(LL P, LL R, LL Q, LL n, Val VU, Val VR) {
//(Px+R)/Q, 1<=x<=i, 经过整点先U再R
  if(!(((i128)n * P + R) / Q)) return ksm(VR, n);
  if(P>=Q) return work(P%Q,R,Q,n, VU, ksm(VU, P/Q) * VR);
  Val res; swap(VU,VR);
  res = ksm(VU, (Q-R-1)/P)*VR;
  LL m = ((i128)n * P + R) / Q;
  res = res * work(Q, (Q-R-1)%P, P, m-1, VU, VR);
  return res * ksm(VU, n - ((i128)m*Q - R - 1) / P); }
```

3.9 平方剩余

```cpp
// x^2=a (mod p),0 <=a<p, 返回 true or false 代表是否存在解
// p必须是质数, 若是多个单次质数的乘积, 可以分别求解再用CRT合并
// 复杂度为 O(log n)
void multiply(ll &c, ll &d, ll a, ll b, ll w) {
| int cc = (a * c + b * d % MOD * w) % MOD;
| int dd = (a * d + b * c) % MOD; c = cc, d = dd; }
bool solve(int n, int &x) {
| if (n==0) return x=0,true; if (MOD==2) return x=1,true;
| if (power(n, MOD / 2, MOD) == MOD - 1) return false;
| ll c = 1, d = 0, b = 1, a, w;
| // finding a such that a^2 - n is not a square
| do { a = rand() % MOD; w = (a * a - n + MOD) % MOD;
| | if (w == 0) return x = a, true;
| } while (power(w, MOD / 2, MOD) != MOD - 1);
| for (int times = (MOD + 1) / 2; times; times >>= 1) {
| | if (times & 1) multiply(c, d, a, b, w);
| | multiply(a, b, a, b, w); }
| // x = (a + sqrt(w)) ^ ((p + 1) / 2)
| return x = c, true; }
```

3.10 原根

定义 使得 $a^x \bmod m = 1$ 的最小的x, 记作 $\delta_m(a)$. 若 $a \equiv g^s \mod m$, 其中 g 为 m 的一个原根. 则虽然 s 随 g 的不同取值有所不同, 但是必然满足 $\delta_m(a) = \gcd(s, \varphi(m))$.

性质 $\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a),k)}$

k 次剩余 给定方程 $x^k \equiv a \mod m$, 求所有解. 若 k 与 $\varphi(m)$ 互质, 则可以直接求出 k 对 $\varphi(m)$ 的逆元. 否则, 将 k 拆成两部分, $k = uv$, 其中 $u \perp \varphi(m)$, $v | \varphi(m)$, 先求 $x^v \equiv a \mod m$, 则 $ans = x^{u^{-1}}$. 下面讨论 $k | \varphi(m)$ 的问题. 任取一原根 g, 对两侧取离散对数, 设 $x = g^s, a = g^t$, 其中 t 可以用BSGS求出, 则问题转化为求出所有的 s 满足 $ks \equiv t \mod \varphi(m)$, exgcd 即可求解, 显然有解的条件是 $k | \delta_m(a)$.

3.11 min25筛

```cpp
typedef long long ll;

ll n;
ll pri[N], sta[M], g[M]; int tot, mod;
int getid(ll x) {
| if(x <= sqrt(n)) return x;
| return tot + 1 - (n / x);
}

int solve(ll n, int id) {
| if(n < pri[id]) return 0;
| if(n / pri[id] < pri[id])
| | return (g[getid(n)] - (id > 1 ? g[pri[id - 1]] : 0) + mod) % mod;

| ll e = n / pri[id]; int d = pri[id], Sum = 0;
| while(e) {
| | Sum = (Sum + (ll)d * (d - 1 + mod) % mod * (solve(e, id + 1) + 1)) % mod;
| | d = (ll)d * pri[id] % mod; e /= pri[id];
| }

| return (Sum + solve(n, id + 1)) % mod;
}

void init(){
| for(int i=1; i<=pri[0]; i++)
| | for(int j=tot; j>=1; j--){
| | | if(sta[j] / pri[i] < pri[i]) break;
| | | int num = (g[getid(sta[j] / pri[i])] - g[pri[i] - 1] + mod) % mod;
| | | g[j] = (g[j] - (ll)pri[i] * pri[i] % mod * num % mod + mod) % mod;
| | }
}
```

3.12 Pell 方程

```cpp
// x^2 - n * y^2 = 1 最小正整数根, n 为完全平方数时无解
// x_{k+1} = x_0 x_k + n y_0 y_k
// y_{k+1} = x_0 y_k + y_0 x_k
pair<LL, LL> peLL(LL n) {
| static LL p[N], q[N], g[N], h[N], a[N];
| p[1] = q[0] = h[1] = 1; p[0] = q[1] = g[1] = 0;
| a[2] = (LL)(floor(sqrtl(n) + 1e-7L));
| for(int i = 2; ; i ++) {
| | g[i] = -g[i - 1] + a[i] * h[i - 1];
| | h[i] = (n - g[i] * g[i]) / h[i - 1];
| | a[i + 1] = (g[i] + a[2]) / h[i];
| | p[i] = a[i] * p[i - 1] + p[i - 2];
| | q[i] = a[i] * q[i - 1] + q[i - 2];
| | if(p[i] * p[i] - n * q[i] * q[i] == 1)
| | | return {p[i], q[i]}; }}
```

3.13 解一元三次方程

```cpp
double a(p[3]), b(p[2]), c(p[1]), d(p[0]);
double k(b / a), m(c / a), n(d / a);
double p(-k * k / 3. + m);
double q(2. * k * k * k / 27 - k * m / 3. + n);
Complex omega[3] = {Complex(1, 0), Complex(-0.5, 0.5 * sqrt(3)), Complex(-0.5, -0.5 * sqrt(3))};
Complex r1, r2; double delta(q * q / 4 + p * p * p / 27);
if (delta > 0) {
| r1 = cubrt(-q / 2. + sqrt(delta));
| r2 = cubrt(-q / 2. - sqrt(delta));
} else {
| r1 = pow(-q / 2. + pow(Complex(delta), 0.5), 1. / 3);
| r2 = pow(-q / 2. - pow(Complex(delta), 0.5), 1. / 3); }
for(int _(0); _ < 3; _++) {
| Complex x = -k/3. + r1*omega[_] + r2*omega[_* 2 % 3]; }
```

3.14 Formulas 公式表

3.14.1 Mobius Inversion

$$F(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right)$$

$$F(n) = \sum_{n|d} f(d) \Rightarrow f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) F(d)$$

$$[x = 1] = \sum_{d|x} \mu(d), \quad x = \sum_{d|x} \mu(d)$$

3.14.2 杜教筛

$$S_\varphi(n) = \frac{n(n+1)}{2} - \sum_{d=2}^{n} S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$S_\mu(n) = 1 - \sum_{d=2}^{n} S_\mu\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

3.14.3 降幂公式

$$a^k \equiv a^{k \bmod \varphi(p) + \varphi(p)}, \ k \geq \varphi(p)$$

3.14.4 其他常用公式

$$\sum_{i=1}^{n} [(i,n) = 1]\, i = n \frac{\varphi(n) + e(n)}{2}$$

$$\sum_{i=1}^{n} \sum_{j=1}^{i} [(i,j) = d] = S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$\sum_{i=1}^{n} \sum_{j=1}^{m} [(i,j) = d] = \sum_{d|k} \mu\left(\frac{k}{d}\right) \left\lfloor \frac{n}{k} \right\rfloor \left\lfloor \frac{m}{k} \right\rfloor$$

$$\sum_{i=1}^{n} f(i) \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} g(j) = \sum_{i=1}^{n} g(i) \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} f(j)$$

3.14.5 单位根反演

$$\sum_{i=0}^{\lfloor \frac{n}{k} \rfloor} C_n^{ik}$$

引理

$$\frac{1}{k} \sum_{i=0}^{k-1} \omega_k^{in} = [k \mid n]$$

反演

$$Ans = \sum_{i=0}^{n} C_n^i [k \mid i]$$

$$= \sum_{i=0}^{n} C_n^i \left(\frac{1}{k} \sum_{j=0}^{k-1} \omega_k^{ij}\right)$$

$$= \frac{1}{k} \sum_{i=0}^{n} C_n^i \sum_{j=0}^{k-1} \omega_k^{ij}$$

$$= \frac{1}{k} \sum_{j=0}^{k-1} \left(\sum_{i=0}^{n} C_n^i (\omega_k^j)^i\right)$$

$$= \frac{1}{k} \sum_{j=0}^{k-1} (1 + \omega_k^j)^n$$

另, 如果要求的是 $[n\%k = t]$, 其实就是 $[k \mid (n-t)]$. 同理推式子即可.

3.14.6 Arithmetic Function

$$(p-1)! \equiv -1 (mod\ p)$$

$$a > 1, m, n > 0, \text{then} \gcd(a^m - 1, a^n - 1) = a^{\gcd(n,m)} - 1$$

$$\mu^2(n) = \sum_{d^2|n} \mu(d)$$

$$a > b, \gcd(a,b) = 1, \text{then} \gcd(a^m - b^m, a^n - b^n) = a^{\gcd(m,n)} - b^{\gcd(m,n)}$$

$$\prod_{k=1, gcd(k,m)=1}^{m} k \equiv \begin{cases} -1 & \mod m, m = 4, p^q, 2p^q \\ 1 & \mod m, \text{otherwise} \end{cases}$$

$$\sigma_k(n) = \sum_{d|n} d^k = \prod_{i=1}^{\omega(n)} \frac{p_i^{(a_i+1)k} - 1}{p_i^k - 1}$$

$$J_k(n) = n^k \prod_{p|n} \left(1 - \frac{1}{p^k}\right)$$

$J_k(n)$ is the number of k-tuples of positive integers all less than or equal to n that form a coprime $(k+1)$-tuple together with n.

$$\sum_{\delta|n} J_k(\delta) = n^k$$

$$\sum_{i=1}^{n} \sum_{j=1}^{n} [gcd(i,j) = 1] ij = \sum_{i=1}^{n} i^2 \varphi(i)$$

$$\sum_{\delta|n} \delta^s J_r(\delta) J_s\left(\frac{n}{\delta}\right) = J_{r+s}(n)$$

$$\sum_{\delta|n}\varphi(\delta)d\left(\frac{n}{\delta}\right)=\sigma(n),\ \sum_{\delta|n}|\mu(\delta)|=2^{\omega(n)}$$

$$\sum_{\delta|n}2^{\omega(\delta)}=d(n^2),\ \sum_{\delta|n}d(\delta^2)=d^2(n)$$

$$\sum_{\delta|n}d\left(\frac{n}{\delta}\right)2^{\omega(\delta)}=d^2(n),\ \sum_{\delta|n}\frac{\mu(\delta)}{\delta}=\frac{\varphi(n)}{n}$$

$$\sum_{\delta|n}\frac{\mu(\delta)}{\varphi(\delta)}=d(n),\ \sum_{\delta|n}\frac{\mu^2(\delta)}{\varphi(\delta)}=\frac{n}{\varphi(n)}$$

$$n|\varphi(a^n-1)$$

$$\sum_{\substack{1\le k\le n\\ \gcd(k,n)=1}}f(\gcd(k-1,n))=\varphi(n)\sum_{d|n}\frac{(\mu*f)(d)}{\varphi(d)}$$

$$\varphi(\mathrm{lcm}(m,n))\varphi(\gcd(m,n))=\varphi(m)\varphi(n)$$

$$\sum_{\delta|n}d^3(\delta)=(\sum_{\delta|n}d(\delta))^2$$

$$d(uv)=\sum_{\delta|\gcd(u,v)}\mu(\delta)d(\frac{u}{\delta})d(\frac{v}{\delta})$$

$$\sigma_k(u)\sigma_k(v)=\sum_{\delta|\gcd(u,v)}\delta^k\sigma_k(\frac{uv}{\delta^2})$$

$$\mu(n)=\sum_{k=1}^{n}[\gcd(k,n)=1]\cos 2\pi\frac{k}{n}$$

$$\varphi(n)=\sum_{k=1}^{n}[\gcd(k,n)=1]=\sum_{k=1}^{n}\gcd(k,n)\cos 2\pi\frac{k}{n}$$

$$\begin{cases} S(n)=\sum_{k=1}^{n}(f*g)(k)\\ \sum_{k=1}^{n}S(\lfloor\frac{n}{k}\rfloor)=\sum_{i=1}^{n}f(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}(g*1)(j)\end{cases}$$

$$\begin{cases} S(n)=\sum_{k=1}^{n}(f\cdot g)(k),g \text{ completely multiplicative}\\ \sum_{k=1}^{n}S\left(\left\lfloor\frac{n}{k}\right\rfloor\right)g(k)=\sum_{k=1}^{n}(f*1)(k)g(k)\end{cases}$$

3.14.7 Binomial Coefficients

C	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

$$\binom{n}{k}\equiv[n\&k=k]\pmod 2$$

$$\sum_{k=0}^{n}\binom{k}{m}=\binom{n+1}{m+1}$$

$$\sqrt{1+z}=1+\sum_{k=1}^{\infty}\frac{(-1)^{k-1}}{k\times 2^{2k-1}}\binom{2k-2}{k-1}z^k$$

$$\sum_{k=0}^{r}\binom{r-k}{m}\binom{s+k}{n}=\binom{r+s+1}{m+n+1}$$

$$C_{n,m}=\binom{n+m}{m}-\binom{n+m}{m-1},n\ge m$$

$$\binom{n}{k}=(-1)^k\binom{k-n-1}{k},\ \sum_{k\le n}\binom{r+k}{k}=\binom{r+n+1}{n}$$

$$\binom{n_1+\cdots+n_p}{m}=\sum_{k_1+\cdots+k_p=m}\binom{n_1}{k_1}\cdots\binom{n_p}{k_p}$$

3.14.8 Fibonacci Numbers, Lucas Numbers

$$F(z)=\frac{z}{1-z-z^2}$$

$$\hat{\phi}=\frac{1-\sqrt{5}}{2}$$

$$\sum_{k=1}^{n}f_k=f_{n+2}-1,\ \sum_{k=1}^{n}f_k^2=f_nf_{n+1}$$

$$\sum_{k=0}^{n}f_kf_{n-k}=\frac{1}{5}(n-1)f_n+\frac{2}{5}nf_{n-1}$$

$$\frac{f_{2n}}{f_n}=f_{n-1}+f_{n+1}$$

$$f_1+2f_2+3f_3+\cdots+nf_n=nf_{n+2}-f_{n+3}+2$$

$$\gcd(f_m,f_n)=f_{\gcd(m,n)}$$

$$f_n^2+(-1)^n=f_{n+1}f_{n-1}$$

$$f_{n+k}=f_nf_{k+1}+f_{n-1}f_k$$

$$f_{2n+1}=f_n^2+f_{n+1}^2$$

$$(-1)^kf_{n-k}=f_nf_{k-1}-f_{n-1}f_k$$

```python
def fib(n): # F(n), F(n + 1)
        if not n: return (0, 1)
        a, b = fib(n >> 1)
        c = a * (2 * b - a)
        d = a * a + b * b
        if n & 1:
                return (d, c + d)
        else:
                return (c, d)
```

$$\text{Modulo } f_n,f_{mn+r}\equiv\begin{cases} f_r, & m \bmod 4=0;\\ (-1)^{r+1}f_{n-r}, & m \bmod 4=1;\\ (-1)^nf_r, & m \bmod 4=2;\\ (-1)^{r+1+n}f_{n-r}, & m \bmod 4=3.\end{cases}$$

$$L_0=2,L_1=1,L_n=L_{n-1}+L_{n-2}=(\frac{1+\sqrt{5}}{2})^n+(\frac{1-\sqrt{5}}{2})^n$$

$$L(x)=\frac{2-x}{1-x-x^2}$$

除了 $n=0,4,8,16$, L_n 是素数, 则 n 是素数.

$$\phi=\frac{1+\sqrt{5}}{2},\ \hat{\phi}=\frac{1-\sqrt{5}}{2}$$

$$F_n=\frac{\phi^n-\hat{\phi}^n}{\sqrt{5}},\ L_n=\phi^n+\hat{\phi}^n$$

$$\frac{L_n+F_n\sqrt{5}}{2}=\left(\frac{1+\sqrt{5}}{2}\right)^n$$

3.14.9 Sum of Powers

$$\sum_{i=1}^{n}i^2=\frac{n(n+1)(2n+1)}{6},\ \sum_{i=1}^{n}i^3=\left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{i=1}^{n}i^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{i=1}^{n}i^5=\frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

3.14.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430...

$$c_0=1,c_n=\sum_{i=0}^{n-1}c_ic_{n-1-i}=c_{n-1}\frac{4n-2}{n+1}=\frac{\binom{2n}{n}}{n+1}=\binom{2n}{n}-\binom{2n}{n-1}$$

$$c(x)=\frac{1-\sqrt{1-4x}}{2x}$$

Usage: n 对括号序列; n 个点满二叉树; $n\times n$ 的方格左下到右上不过对角线方案数; 凸 $n+2$ 边形三角形分割数; n 个数的出栈方案数; $2n$ 个顶点连接, 线段两两不交的方案数.

类卡特兰数 从 $(1,1)$ 出发走到 (n,m), 只能向右或者向上走, 不能越过 $y=x$ 这条线 (即保证 $x\ge y$), 合法方案数是 $C_{n+m-2}^{n}-C_{n+m-2}^{n-1}$.

3.14.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835...

圆上 n 点间画不相交弦的方案数. 选 n 个数 $k_1,k_2,...,k_n\in\{-1,0,1\}$, 保证 $\sum_i^a k_i(1\le a\le n)$ 非负且所有数总和为 0 的方案数.

$$M_{n+1}=M_n+\sum_{i}^{n-1}M_iM_{n-1-i}=\frac{(2n+3)M_n+3nM_{n-1}}{n+3}$$

$$M_n=\sum_{i=0}^{\lfloor\frac{n}{2}\rfloor}\binom{n}{2k}\mathrm{Catlan}(k)$$

$$M(X)=\frac{1-x-\sqrt{1-2x-3x^2}}{2x^2}$$

3.14.12 Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833...

$$D_1=0,D_2=1,D_n=n!(\frac{1}{0!}-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}+...+\frac{(-1)^n}{n!})$$

$$D_n=(n-1)(D_{n-1}+D_{n-2})$$

3.14.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ...

n 个元素集合划分的方案数.

$$B_n=\sum_{k=1}^{n}\left\{ {n \atop k} \right\},\ B_{n+1}=\sum_{k=0}^{n}\binom{n}{k}B_k$$

$$B_{p^m+n}\equiv mB_n+B_{n+1}\pmod p$$

$$B(x)=\sum_{n=0}^{\infty}\frac{B_n}{n!}x^n=e^{e^x-1}$$

3.14.14 Stirling Numbers

第一类 n 个元素集合分作 k 个非空轮换方案数.

$$\left[{n+1 \atop k}\right]=n\left[{n \atop k}\right]+\left[{n \atop k-1}\right]$$

$$s(n,k)=(-1)^{n-k}\left[{n \atop k}\right]$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	2	3	1			
4	0	6	11	6	1		
5	0	24	50	35	10	1	
6	0	120	274	225	85	15	1
7	0	720	1764	1624	735	175	21

$$\left[{n+1 \atop 2}\right]=n!H_n\ \text{(see 3.14.16)}$$

$$x^{\underline{n}}=\sum_k\left[{n \atop k}\right](-1)^{n-k}x^k$$

$$x^{\overline{n}}=\sum_k\left[{n \atop k}\right]x^k$$

For fixed k, EGF:

$$\sum_{n=0}^{\infty}\left[{n \atop k}\right]\frac{x^n}{n!}=\frac{x^k}{k!}\left(\frac{\ln(1-x)}{x}\right)^k$$

第二类 把 n 个元素集合分作 k 个非空子集方案数.

$$\left\{{n+1 \atop k}\right\}=k\left\{{n \atop k}\right\}+\left\{{n \atop k-1}\right\}$$

$$m!\left\{{n \atop m}\right\}=\sum_k\binom{m}{k}k^n(-1)^{m-k}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	1	3	1			
4	0	1	7	6	1		
5	0	1	15	25	10	1	
6	0	1	31	90	65	15	1
7	0	1	63	301	350	140	21

$$x^n=\sum_k\left\{{n \atop k}\right\}x^{\underline{k}}$$

$$=\sum_k\left\{{n \atop k}\right\}(-1)^{n-k}x^{\overline{k}}$$

For fixed k, EGF and OGF:

$$\sum_{n=0}^{\infty}\left\{{n \atop k}\right\}\frac{x^n}{n!}=\frac{x^k}{k!}\left(\frac{e^x-1}{x}\right)^k$$

$$\sum_{n=0}^{\infty}\left\{{n \atop k}\right\}x^n=x^k\prod_{i=1}^{k}(1-ix)^{-1}$$

3.14.15 Eulerian Numbers

n\k	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

$$\left\langle {n \atop k} \right\rangle = (k+1)\left\langle {n-1 \atop k} \right\rangle + (n-k)\left\langle {n-1 \atop k-1} \right\rangle$$

$$x^n = \sum_k \left\langle {n \atop k} \right\rangle \binom{x+k}{n}$$

$$\left\langle {n \atop m} \right\rangle = \sum_{k=0}^{m} \binom{n+1}{k}(m+1-k)^n(-1)^k$$

3.14.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60...

$$H_n = \sum_{k=1}^{n}\frac{1}{k}, \sum_{k=1}^{n} H_k = (n+1)H_n - n$$

$$\sum_{k=1}^{n} kH_k = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$$

$$\sum_{k=1}^{n}\binom{k}{m}H_k = \binom{n+1}{m+1}(H_{n+1} - \frac{1}{m+1})$$

3.14.17 卡迈克尔函数

卡迈克尔函数表示模 m 剩余系下最大的阶, 即 $\lambda(m) = \max_{a\perp m}\delta_m(a)$.

容易看出, 若 $\lambda(m) = \varphi(m)$, 则 m 存在原根.

该函数可由下述方法计算: 分解质因数 $m = p_1^{\alpha_1}p_2^{\alpha_2}...p_t^{\alpha_t}$. 则 $\lambda(m) = \mathrm{lcm}(\lambda(p_1^{\alpha_1}), p_2^{\alpha_2}, ..., p_t^{\alpha_t})$. 其中对奇质数 p, $\lambda(p^\alpha) = (p-1)p^{\alpha-1}$. 对2的幂, $\lambda(2^k) = 2^{k-2}, s.t. k \ge 3$. $\lambda(4) = \lambda(2) = 2$.

3.14.18 求拆分数

```
def penta(k):
  return k*(3*k-1)//2
def compute_partition(goal):
  p = [1]
  for n in range(1,goal+1):
        p.append(0)
        for k in range(1,n+1):
          c = (-1)**(k+1)
          for t in [penta(k), penta(-k)]:
                if (n-t) >= 0:
                  p[n] = p[n] + c*p[n-t]
  return p
```

$\Phi(x) = \prod_{n=1}^{\infty}(1-x^n)$
$= \sum_{n=-\infty}^{\infty}(-1)^k x^{k(3k-1)/2}$

记 $p(n)$ 表示 n 的拆分数,$f(n,k)$ 表示将 n 拆分且每种数字使用次数必须小于等于 k 的拆分数. 则

$$P(x)\Phi(x) = 1, F(x^k)\Phi(x) = 1$$

暴力拆开卷积, 可以得到将1,-1,2,-2... 带入五边形数 $(-1)^k x^{k(3k-1)/2}$ 中, 由于小于n的五边形数只有 $\sqrt{n}$ 个, 可以 $O(n\sqrt{n})$ 计算答案:

$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + \cdots$,
$f(n,k) = p(n) - p(n-k) - p(n-2k) + p(n-5k) + p(n-7k) - \cdots$

3.14.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ...

$$B(x) = \sum_{i\ge 0}\frac{B_i x^i}{i!} = \frac{x}{e^x - 1}$$

$$B_n = [n=0] - \sum_{i=0}^{n-1}\binom{n}{i}\frac{B_i}{n-k+1}, \sum_{i=0}^{n}\binom{n+1}{i}B_i = 0$$

$$S_n(m) = \sum_{i=0}^{m-1} i^n = \sum_{i=0}^{n}\binom{n}{i}B_{n-i}\frac{m^{i+1}}{i+1}$$

$B_0 = 1$, $B_1 = -\frac{1}{2}$, $B_4 = -\frac{1}{30}$, $B_6 = \frac{1}{42}$, $B_8 = -\frac{1}{30}$, ….

(除了 $B_1 = -\frac{1}{2}$ 以外, 伯努利数的奇数项都是 0.)

自然数幂次和关于次数的EGF:

$$F(x) = \sum_{k=0}^{\infty}\frac{\sum_{i=0}^{n} i^k}{k!}x^k$$

$$= \sum_{i=0}^{n} e^{ix} = \frac{e^{(n+1)x-1}}{e^x - 1}$$

3.14.20 kMAX-MIN反演

$$k\mathrm{MAX}(S) = \sum_{T\subset S, T\neq\emptyset}(-1)^{|T|-k}C_{|T|-1}^{k-1}\mathrm{MIN}(T)$$

代入 $k = 1$ 即为MAX-MIN反演:

$$\mathrm{MAX}(S) = \sum_{T\subset S, T\neq\emptyset}(-1)^{|T|-1}\mathrm{MIN}(T)$$

3.14.21 伍德伯里矩阵不等式

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}$$

该等式可以动态维护矩阵的逆, 令 $C = [1]$, U, V 分别为 $1\times n$ 和 $n\times 1$ 的向量, 这样可以构造出 UCV 为只有某行或者某列不为0的矩阵, 一次修改复杂度为 $O(n^2)$.

3.14.22 Sum of Squares

$r_k(n)$ 表示用 k 个平方数组成 n 的方案数. 假设:

$$n = 2^{a_0}p_1^{2a_1}\cdots p_r^{2a_r}q_1^{b_1}\cdots q_s^{b_s}$$

其中 $p_i \equiv 3 \mod 4$, $q_i \equiv 1 \mod 4$, 那么

$$r_2(n) = \begin{cases} 0 & \text{if any } a_i \text{ is a half-integer} \\ 4\prod_1^r(b_i+1) & \text{if all } a_i \text{ are integers} \end{cases}$$

$r_3(n) > 0$ 当且仅当 n 不满足 $4^a(8b+7)$ 的形式 (a, b 为整数).

3.14.23 枚举勾股数 Pythagorean Triple

枚举 $x^2 + y^2 = z^2$ 的三元组: 可令 $x = m^2 - n^2$, $y = 2mn$, $z = m^2 + n^2$, 枚举 m 和 n 即可 $O(n)$ 枚举勾股数. 判断素勾股数方法: m, n 至少一个为偶数并且 m, n 互质, 那么 x, y, z 就是素勾股数.

3.14.24 四面体体积 Tetrahedron Volume

If U, V, W, u, v, w are lengths of edges of the tetrahedron (first three form a triangle; u opposite to U and so on)

$$V = \frac{\sqrt{4u^2v^2w^2 - \sum_{cyc}u^2(v^2+w^2-U^2)^2 + \prod_{cyc}(v^2+w^2-U^2)}}{12}$$

3.14.25 杨氏矩阵与钩子公式

满足: 格子 (i, j) 没有元素, 则它右边和上边相邻格子也没有元素; 格子 (i, j) 有元素 $a[i][j]$, 则它右边和上边相邻格子要么没有元素, 要么有元素且比 $a[i][j]$ 大.

计数: $F_1 = 1, F_2 = 2, F_n = F_{n-1} + (n-1)F_{n-2}, F(x) = e^{x+\frac{x^2}{2}}$

钩子公式: 对于给定形状 λ, 不同杨氏矩阵的个数为:

$$d_\lambda = \frac{n!}{\prod h_\lambda(i,j)}$$

$h_\lambda(i,j)$ 表示该格子右边和上边的格子数量加1.

3.14.26 常见博弈游戏

Nim-K游戏 n 堆石子轮流拿, 每次最多可以拿k堆石子, 谁走最后一步输。结论: 把每一堆石子的sg值 (即石子数量) 二进制分解, 先手必败当且仅当每一位二进制位上1的个数是 (k+1) 的倍数.

Anti-Nim游戏 n 堆石子轮流拿, 谁走最后一步输。结论: 先手胜当且仅当1. 所有堆石子数都为1且游戏的SG值为0 (即有偶数个孤单堆-每堆只有1个石子数) 2. 存在某堆石子数大于1且游戏的SG值不为0.

斐波那契博弈 有一堆物品, 两人轮流取物品, 先手最少取一个, 至多无上限, 但不能把物品取完, 之后每次取的物品数不能超过上一次取的物品数的二倍且至少为一件, 取走最后一件物品的人获胜. 结论: 先手胜当且仅当物品数 n 不是斐波那契数.

威佐夫博弈 有两堆石子, 博弈双方每次可以取一堆石子中的任意个, 不能不取, 或者取两堆石子中的相同个. 先取完者赢. 结论: 求出两堆石子 A 和 B 的差值 C, 如果 $\left\lfloor C * \frac{\sqrt{5}+1}{2} \right\rfloor = min(A, B)$ 那么后手赢, 否则先手赢.

约瑟夫环 n 个人 $0, \ldots, n-1$, 令 $f_{i,m}$ 为 i 个人报 m 的胜利者, 则 $f_{1,m} = 0, f_{i,m} = (f_{i-1,m} + m) \bmod i$.

阶梯Nim 在一个阶梯上, 每次选一个台阶上任意个式子移到下一个台阶上, 不可移动者输. 结论: SG值等于奇数层台阶上石子数的异或和. 对于树形结构也适用, 奇数层节点上所有石子数异或起来即可.

图上博弈 给定无向图, 先手从某点开始走, 只能走相邻且未走过的点, 无法移动者输. 对该图求最大匹配, 若某个点不一定在最大匹配中则先手必败, 否则先手必胜.

最大最小定理求纳什均衡点 在二人零和博弈中, 可以用以下方式求出一个纳什均衡点: 在博弈双方中任选一方, 求混合策略 $\mathbf{p}$ 使得对方选择任意一个纯策略时, 己方的最小收益最大 (等价于对方的最大收益最小). 据此可以求出双方在此局面下的最优期望得分, 分别等于己方最大的最小收益和对方最小的最大收益. 一般而言, 可以得到形如

$$\max_{\mathbf{p}}\min_i \sum_{p_j\in\mathbf{p}} p_j w_{i,j}, \text{s.t. } p_j \ge 0, \sum p_j = 1$$

的形式. 当 $\sum p_j w_{i,j}$ 可以表示成只与 i 有关的函数 $f(i)$ 时, 可以令初始时 $p_i = 0$, 不断调整 $\sum p_j w_{i,j}$ 最小的那个i的概率 p_i, 直至无法调整或者 $\sum p_j = 1$ 为止.

3.14.27 概率相关

$D(X) = E(X-E(X))^2 = E(X^2) - (E(X))^2, D(X+Y) = D(X) + D(Y) D(aX) = a^2 D(X)$

$$E[x] = \sum_{i=1}^{\infty} P(X \ge i)$$

m 个数的方差:

$$s^2 = \frac{\sum_{i=1}^m x_i^2}{m} - \overline{x}^2$$

3.14.28 邻接矩阵行列式的意义

在无向图中取若干个环, 一种取法权值就是边权的乘积, 对行列式的贡献是 $(-1)^{\text{even}}$, 其中 even 是偶环的个数.

3.14.29 Others (某些近似数值公式在这里)

$$S_j = \sum_{k=1}^{n} x_k^j$$

$$h_m = \sum_{1\le j_1 < \cdots < j_m \le n} x_{j_1}\cdots x_{j_m}, \quad H_m = \sum_{1\le j_1\le\cdots\le j_m\le n} x_{j_1}\cdots x_{j_m}$$

$$h_n = \frac{1}{n}\sum_{k=1}^{n}(-1)^{k+1}S_k h_{n-k}$$

$$H_n = \frac{1}{n}\sum_{k=1}^{n} S_k H_{n-k}$$

$$\sum_{k=0}^{n} kc^k = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}$$

$$\sum_{i=1}^{n}\frac{1}{n} \approx \ln\left(n + \frac{1}{2}\right) + \frac{1}{24(n+0.5)^2} + \Gamma, (\Gamma \approx 0.5772156649015328606065)$$

$$n! = \sqrt{2\pi n}(\frac{n}{e})^n(1 + \frac{1}{12n} + \frac{1}{288n^2} + O(\frac{1}{n^3}))$$

$$\max\{x_a - x_b, y_a - y_b, z_a - z_b\} - \min\{x_a - x_b, y_a - y_b, z_a - z_b\}$$
$$= \frac{1}{2}\sum_{cyc}|(x_a - y_a) - (x_b - y_b)|$$

$$(a+b)(b+c)(c+a) = \frac{(a+b+c)^3 - a^3 - b^3 - c^3}{3}$$

$$a^3 + b^3 = (a+b)(a^2 - ab + b^2), a^3 - b^3 = (a-b)(a^2 + ab + b^2)$$

$n \bmod 2 = 1$:

$$a^n + b^n = (a+b)(a^{n-1} - a^{n-2}b + a^{n-3}b^2 - \ldots - ab^{n-2} + b^{n-1})$$

划分问题: n 个 $k-1$ 维向量最多把 k 维空间分为 $\sum_{i=0}^{k} C_n^i$ 份.

3.15 Calculus, Integration Table 导数积分表

$(\frac{u}{v})' = \frac{u'v-uv'}{v^2}$
$(a^x)' = (\ln a)a^x$
$(\tan x)' = \sec^2 x$
$(\cot x)' = \csc^2 x$
$(\sec x)' = \tan x \sec x$
$(\csc x)' = -\cot x \csc x$
$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$
$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$
$(\arctan x)' = \frac{1}{1+x^2}$
$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$
$(\operatorname{arccsc} x)' = -\frac{1}{x\sqrt{1-x^2}}$
$(\operatorname{arcsec} x)' = \frac{1}{x\sqrt{1-x^2}}$
$(\tanh x)' = \operatorname{sech}^2 x$
$(\coth x)' = -\operatorname{csch}^2 x$
$(\operatorname{sech} x)' = -\operatorname{sech} x \tanh x$
$(\operatorname{csch} x)' = -\operatorname{csch} x \coth x$
$(\operatorname{arcsinh} x)' = \frac{1}{\sqrt{1+x^2}}$
$(\operatorname{arccosh} x)' = \frac{1}{\sqrt{x^2-1}}$
$(\operatorname{arctanh} x)' = \frac{1}{1-x^2}$
$(\operatorname{arccoth} x)' = \frac{1}{x^2-1}$
$(\operatorname{arccsch} x)' = -\frac{1}{|x|\sqrt{1+x^2}}$
$(\operatorname{arcsech} x)' = -\frac{1}{x\sqrt{1-x^2}}$

· $ax^2 + bx + c(a > 0)$

$$\int \frac{dx}{ax^2+bx+c} = \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan \frac{2ax+b}{\sqrt{4ac-b^2}} + C & (b^2 < 4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln \left| \frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right| + C & (b^2 > 4ac) \end{cases}$$

$$\int \frac{x}{ax^2+bx+c} dx = \frac{1}{2a} \ln |ax^2+bx+c| - \frac{b}{2a} \int \frac{dx}{ax^2+bx+c}$$

· $\sqrt{\pm ax^2 + bx + c}(a > 0)$

$$\int \frac{dx}{\sqrt{ax^2+bx+c}} = \frac{1}{\sqrt{a}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2+bx+c}| + C$$

$$\int \sqrt{ax^2+bx+c}dx = \frac{2ax+b}{4a}\sqrt{ax^2+bx+c} + \frac{4ac-b^2}{8\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2+bx+c}| + C$$

$$\int \frac{x}{\sqrt{ax^2+bx+c}} dx = \frac{1}{a}\sqrt{ax^2+bx+c} - \frac{b}{2\sqrt{a^3}} \ln |2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}| + C$$

$$\int \frac{dx}{\sqrt{c+bx-ax^2}} = -\frac{1}{\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \sqrt{c+bx-ax^2}dx = \frac{2ax-b}{4a}\sqrt{c+bx-ax^2} + \frac{b^2+4ac}{8\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \frac{x}{\sqrt{c+bx-ax^2}} dx = -\frac{1}{a}\sqrt{c+bx-ax^2} + \frac{b}{2\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

· $\sqrt{\pm\frac{x-a}{x-b}}$ 或 $\sqrt{(x-a)(x-b)}, (a < b)$

$$\int \frac{dx}{\sqrt{(x-a)(b-x)}} = 2 \arcsin \sqrt{\frac{x-a}{b-x}} + C$$

$$\int \sqrt{(x-a)(b-x)}dx = \frac{2x-a-b}{4}\sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4} \arcsin \sqrt{\frac{x-a}{b-x}} + C$$

三角函数的积分 (省略 $+C$)

$\int \tan xdx = -\ln|\cos x|$
$\int \cot xdx = \ln|\sin x|$
$\int \sec xdx = \ln\left|\tan\left(\frac{\pi}{4}+\frac{x}{2}\right)\right| = \ln|\sec x + \tan x|$
$\int \csc xdx = \ln\left|\tan\frac{x}{2}\right| = \ln|\csc x - \cot x|$
$\int \sec^2 xdx = \tan x$
$\int \csc^2 xdx = -\cot x$
$\int \sec x \tan xdx = \sec x$
$\int \csc x \cot xdx = -\csc x$
$\int \sin^2 xdx = \frac{x}{2} - \frac{1}{4}\sin 2x$
$\int \cos^2 xdx = \frac{x}{2} + \frac{1}{4}\sin 2x$

$\int \sin^n xdx = -\frac{1}{n}\sin^{n-1} x \cos x + \frac{n-1}{n}\int \sin^{n-2} xdx$
$\int \cos^n xdx = \frac{1}{n}\cos^{n-1} x \sin x + \frac{n-1}{n}\int \cos^{n-2} xdx$
$\int \frac{dx}{\sin^n x} = -\frac{1}{n-1}\frac{\cos x}{\sin^{n-1} x} + \frac{n-2}{n-1}\int \frac{dx}{\sin^{n-2} x}$
$\int \frac{dx}{\cos^n x} = \frac{1}{n-1}\frac{\sin x}{\cos^{n-1} x} + \frac{n-2}{n-1}\int \frac{dx}{\cos^{n-2} x}$

$$\int \cos^m x \sin^n xdx$$
$$= \frac{1}{m+n}\cos^{m-1} x \sin^{n+1} x + \frac{m-1}{m+n}\int \cos^{m-2} x \sin^n xdx$$
$$= -\frac{1}{m+n}\cos^{m+1} x \sin^{n-1} x + \frac{n-1}{m+1}\int \cos^m x \sin^{n-2} xdx$$

$$\int \frac{dx}{a+b\sin x} = \begin{cases} \frac{2}{\sqrt{a^2-b^2}} \arctan \frac{a\tan\frac{x}{2}+b}{\sqrt{a^2-b^2}} & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2-a^2}} \ln \left| \frac{a\tan\frac{x}{2}+b-\sqrt{b^2-a^2}}{a\tan\frac{x}{2}+b+\sqrt{b^2-a^2}} \right| & (a^2 < b^2) \end{cases}$$

$$\int \frac{dx}{a+b\cos x} = \begin{cases} \frac{2}{a+b}\sqrt{\frac{a+b}{a-b}} \arctan\left(\sqrt{\frac{a-b}{a+b}}\tan\frac{x}{2}\right) & (a^2 > b^2) \\ \frac{1}{a+b}\sqrt{\frac{a+b}{a-b}} \ln \left| \frac{\tan\frac{x}{2}+\sqrt{\frac{a+b}{b-a}}}{\tan\frac{x}{2}-\sqrt{\frac{a+b}{b-a}}} \right| & (a^2 < b^2) \end{cases}$$

$\int \frac{dx}{a^2\cos^2 x + b^2\sin^2 x} = \frac{1}{ab}\arctan\left(\frac{b}{a}\tan x\right)$
$\int \frac{dx}{a^2\cos^2 x - b^2\sin^2 x} = \frac{1}{2ab}\ln\left|\frac{b\tan x + a}{b\tan x - a}\right|$
$\int x\sin axdx = \frac{1}{a^2}\sin ax - \frac{1}{a}x\cos ax$
$\int x^2\sin axdx = -\frac{1}{a}x^2\cos ax + \frac{2}{a^2}x\sin ax + \frac{2}{a^3}\cos ax$
$\int x\cos axdx = \frac{1}{a^2}\cos ax + \frac{1}{a}x\sin ax$
$\int x^2\cos axdx = \frac{1}{a}x^2\sin ax + \frac{2}{a^2}x\cos ax - \frac{2}{a^3}\sin ax$

反三角函数的积分 (其中 $a > 0$)

$\int \arcsin\frac{x}{a}dx = x\arcsin\frac{x}{a} + \sqrt{a^2-x^2} + C$
$\int x\arcsin\frac{x}{a}dx = \left(\frac{x^2}{2}-\frac{a^2}{4}\right)\arcsin\frac{x}{a} + \frac{x}{4}\sqrt{a^2-x^2} + C$
$\int x^2\arcsin\frac{x}{a}dx = \frac{x^3}{3}\arcsin\frac{x}{a} + \frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2} + C$
$\int \arccos\frac{x}{a}dx = x\arccos\frac{x}{a} - \sqrt{a^2-x^2} + C$
$\int x\arccos\frac{x}{a}dx = \left(\frac{x^2}{2}-\frac{a^2}{4}\right)\arccos\frac{x}{a} - \frac{x}{4}\sqrt{a^2-x^2} + C$
$\int x^2\arccos\frac{x}{a}dx = \frac{x^3}{3}\arccos\frac{x}{a} - \frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2} + C$
$\int \arctan\frac{x}{a}dx = x\arctan\frac{x}{a} - \frac{a}{2}\ln(a^2+x^2) + C$
$\int x\arctan\frac{x}{a}dx = \frac{1}{2}(a^2+x^2)\arctan\frac{x}{a} - \frac{a}{2}x + C$
$\int x^2\arctan\frac{x}{a}dx = \frac{x^3}{3}\arctan\frac{x}{a} - \frac{a}{6}x^2 + \frac{a^3}{6}\ln(a^2+x^2) + C$

指数函数的积分

$\int a^x dx = \frac{1}{\ln a}a^x + C$
$\int e^{ax}dx = \frac{1}{a}a^{ax} + C$
$\int xe^{ax}dx = \frac{1}{a^2}(ax-1)a^{ax} + C$
$\int x^n e^{ax}dx = \frac{1}{a}x^n e^{ax} - \frac{n}{a}\int x^{n-1}e^{ax}dx$
$\int xa^x dx = \frac{x}{\ln a}a^x - \frac{1}{(\ln a)^2}a^x + C$
$\int x^n a^x dx = \frac{1}{\ln a}x^n a^x - \frac{n}{\ln a}\int x^{n-1}a^x dx$
$\int e^{ax}\sin bxdx = \frac{1}{a^2+b^2}e^{ax}(a\sin bx - b\cos bx) + C$
$\int e^{ax}\cos bxdx = \frac{1}{a^2+b^2}e^{ax}(b\sin bx + a\cos bx) + C$
$\int e^{ax}\sin^n bxdx = \frac{1}{a^2+b^2n^2}e^{ax}\sin^{n-1} bx(a\sin bx - nb\cos bx) + \frac{n(n-1)b^2}{a^2+b^2n^2}\int e^{ax}\sin^{n-2} bxdx$
$\int e^{ax}\cos^n bxdx = \frac{1}{a^2+b^2n^2}e^{ax}\cos^{n-1} bx(a\cos bx + nb\sin bx) + \frac{n(n-1)b^2}{a^2+b^2n^2}\int e^{ax}\cos^{n-2} bxdx$

对数函数的积分

$\int \ln xdx = x\ln x - x + C$
$\int \frac{dx}{x\ln x} = \ln|\ln x| + C$
$\int x^n\ln xdx = \frac{1}{n+1}x^{n+1}(\ln x - \frac{1}{n+1}) + C$
$\int (\ln x)^n dx = x(\ln x)^n - n\int(\ln x)^{n-1}dx$
$\int x^m(\ln x)^n dx = \frac{1}{m+1}x^{m+1}(\ln x)^n - \frac{n}{m+1}\int x^m(\ln x)^{n-1}dx$

3.16 Zeller 日期公式

```
// weekday=(id+1)%7;{Sun=0,Mon=1,...}   getId(1, 1, 1) = 0
int getId(int y, int m, int d) {
  if (m < 3) { y --; m += 12; }
  return 365 * y + y / 4 - y / 100 + y / 400 + (153 * (m -
  ↪ 3) + 2) / 5 + d - 307; }
//  y<0: 统一加400的倍数年
auto date(int id) {
  int x=id+1789995, n, i, j, y, m, d;
  n = 4 * x / 146097; x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001; x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447; d = x - 2447 * j / 80; x = j / 11;
  m = j + 2 - 12 * x; y = 100 * (n - 49) + i + x;
  return make_tuple(y, m, d); }
```

3.17 有理数二分: Stern-Brocot 树, Farey 序列

```
def build(a, b, c, d, level = 1):
  x = a + c; y = b + d
  build(a, b, x, y, level + 1)
  build(x, y, c, d, level + 1)
build(0, 1, 1, 0) # 最简分数, Stern-Brocot
build(0, 1, 1, 1) # 最简真分数, Farey
```

3.18 Constant Table 常数表

Random primes generated at Fri Aug 28 20:59:54 2026
5e2 379 397 409 419 433 463 487 499 503 523 577 599 607 613
1e3 853 887 919 929 953 983 997 1019 1061 1091 1093 1117
3e4 27917 28181 28387 30347 31231 31333 31547 31799 32027
1e5 93169 93979 95083 100379 102811 104773 106213 106993
1e6 941573 954307 979093 1000457 1004657 1011067 1047559
5e6 4687967 4886069 4893179 4907753 4924643 5248259 5339303
1e7 9526453 9837739 10120111 10227403 10317157 10458559 10554091
2e7 18836131 19614997 20071483 20908339 20956321 21166193
1e9 936392777 963636257 990722687 993633803 1004788619 1033459429
2e9 1923812333 1939885391 1986645613 1986687649 2011488607
NTT 976224257 r=3 (20) 985661441 r=3 (22) 998244353 r=3 (23)
1004535809 r=3 (21) 1007681537 r=3 (20) 1012924417 r=5 (21)
1045430273 r=3 (20) 1051721729 r=6 (20) 1053818881 r=7 (20)

4. 多项式与生成函数

4.1 FFT

```
using cp = complex<double>; const double PI = acos(-1.0);
vector<cp> omega[25]; // 单位根
// n 是 DFT 的最大长度，例如如果最多有两个长为 m 的多项式相乘，
// 或者求逆的长度为 m，那么 n 需要 >= 2m
void fft_init(int n) { // n = 2^k
  for (int k = 2, d = 0; k <= n; k *= 2, d++) {
    omega[d].resize(k + 1);
    for (int i = 0; i <= k; i++) // polar 是用模和辐角求复
    ↪ 数
      omega[d][i] = polar(1.0, 2 * PI * i / k); } }
void fft(cp* a, int n, int t) {
  for (int i = 1, j = 0; i < n - 1; i++) {
    int k = n; do j ^= (k >>= 1); while (j < k);
    if (i < j) swap(a[i], a[j]); }
  for (int k = 1, d = 0; k < n; k *= 2, d++)
    for (int i = 0; i < n; i += k * 2)
      for (int j = 0; j < k; j++) {
        cp w = omega[d][t > 0 ? j : k * 2 - j];
```

```cpp
      cp u = a[i + j], v = w * a[i + j + k];
      a[i + j] = u + v; a[i + j + k] = u - v; }
  if (t < 0) for (int i = 0; i < n; i++) a[i] /= n; }
```

4.2 NTT

```cpp
vector<int> omega[25]; // 单位根
// n 是 DFT 的最大长度，例如如果最多有两个长为 m 的多项式相乘，
// 或者求逆的长度为 m，那么 n 需要 >= 2m
void ntt_init(int n) { // n = 2^k
  for (int k = 2, d = 0; k <= n; k *= 2, d++) {
    omega[d].resize(k + 1);
    int wn = qpow(3, (p - 1) / k), tmp = 1;
    for (int i = 0; i <= k; i++) { omega[d][i] = tmp;
      tmp = (LL)tmp * wn % p; } } }
// 传入的数必须是 [0, p) 范围内，不能有负的
// 否则把 d == 16 改成 d % 8 == 0 之类，多取几次模
void ntt(int *c, int n, int tp) {
  static ULL a[N];
  for (int i = 0; i < n; i++) a[i] = c[i];
  for (int i = 1, j = 0; i < n - 1; i++) {
    int k = n; do j ^= (k >>= 1); while (j < k);
    if (i < j) swap(a[i], a[j]); }
  for (int k = 1, d = 0; k < n; k *= 2, d++) {
    if (d == 16) for (int i = 0; i < n; i++) a[i] %= p;
    for (int i = 0; i < n; i += k * 2)
      for (int j = 0; j < k; j++) {
        int w = omega[d][tp > 0 ? j : k * 2 - j];
        ULL u = a[i + j], v = w * a[i + j + k] % p;
        a[i + j] = u + v;
        a[i + j + k] = u - v + p; } }
  if (tp>0) {for (int i = 0; i < n; i++) c[i] = a[i] % p;}
  else { int inv = qpow(n, p - 2);
    for (int i = 0; i < n; i++) c[i] = a[i] * inv % p;}}
```

4.3 MTT 任意模数卷积

```cpp
void dft(cp* a, cp* b, int n) { static cp c[MAXN];
  for (int i = 0; i < n; i++)
    c[i] = cp(a[i].real(), b[i].real());
  fft(c, n, 1);
  for (int i = 0; i < n; i++) { int j = (n - i) & (n - 1);
    a[i] = (c[i] + conj(c[j])) * 0.5;
    b[i] = (c[i] - conj(c[j])) * -0.5i; } }
void idft(cp* a, cp* b, int n) { static cp c[MAXN];
  for (int i = 0; i < n; i++) c[i] = a[i] + 1i * b[i];
  fft(c, n, -1);
  for (int i = 0; i < n; i++) {
    a[i] = c[i].real(); b[i] = c[i].imag(); } }
vector<int> multiply(const vector<int>& u,
    const vector<int>& v, int mod) { // 任意模数卷积
  static cp a[2][MAXN], b[2][MAXN], c[3][MAXN];
  int base = ceil(sqrt(mod));
  int n = (int)u.size(), m = (int)v.size();
  int fft_n = 1; while (fft_n < n + m - 1) fft_n *= 2;
  for (int i = 0; i < 2; i++) {
    fill(a[i], a[i] + fft_n, 0);
    fill(b[i], b[i] + fft_n, 0); }
  for (int i = 0; i < 3; i++)
    fill(c[i], c[i] + fft_n, 0);
  for (int i = 0; i < n; i++) { // 一定要取模！
    a[0][i] = (u[i] % mod) % base;
    a[1][i] = (u[i] % mod) / base; }
  for (int i = 0; i < m; i++) { // 一定要取模！
    b[0][i] = (v[i] % mod) % base;
    b[1][i] = (v[i] % mod) / base; }
  dft(a[0], a[1], fft_n); dft(b[0], b[1], fft_n);
  for (int i = 0; i < fft_n; i++) {
    c[0][i] = a[0][i] * b[0][i];
    c[1][i] = a[0][i] * b[1][i] + a[1][i] * b[0][i];
    c[2][i] = a[1][i] * b[1][i]; }
  fft(c[1], fft_n, -1); idft(c[0], c[2], fft_n);
  int base2 = base * base % mod;
  vector<int> ans(n + m - 1);
  for (int i = 0; i < n + m - 1; i++)
    ans[i] = ((LL)(c[0][i].real() + 0.5) +
      (LL)(c[1][i].real() + 0.5) % mod * base +
      (LL)(c[2][i].real() + 0.5) % mod * base2) % mod;
  return ans; }
```

4.4 多项式运算

4.4.1 多项式求逆 开根 ln exp

```cpp
using poly = vector<int>;
poly poly_calc(const poly& u, const poly& v, // 长度要相同
  function<int(int, int)> op) { // 返回长度是两倍
  static int a[MAXN], b[MAXN], c[MAXN];
  int n = (int)u.size();
  memcpy(a, u.data(), sizeof(int) * n);
  fill(a + n, a + n * 2, 0);
  memcpy(b, v.data(), sizeof(int) * n);
  fill(b + n, b + n * 2, 0);
  ntt(a, n * 2, 1); ntt(b, n * 2, 1);
  for (int i = 0; i < n * 2; i++) c[i] = op(a[i], b[i]);
  ntt(c, n * 2, -1); return poly(c, c + n * 2); }
poly poly_mul(const poly& u, const poly& v) { // 乘法
  return poly_calc(u, v, [](int a, int b)
    { return (LL)a * b % p; }); } // 返回长度是两倍
poly poly_inv(const poly& a) { // 求逆，返回长度不变
  poly c{qpow(a[0], p - 2)}; // 常数项一般都是 1
  for (int k = 2; k <= (int)a.size(); k *= 2) {
    c.resize(k); poly b(a.begin(), a.begin() + k);
    c = poly_calc(b, c, [](int bi, int ci) {
      return ((2 - (LL)bi * ci) % p + p) * ci % p; });
    memset(c.data() + k, 0, sizeof(int) * k); }
  c.resize(a.size()); return c; }
poly poly_sqrt(const poly& a) { // 开根，返回长度不变
  poly c{1}; // 常数项不是 1 的话要写二次剩余
  for (int k = 2; k <= (int)a.size(); k *= 2) {
    c.resize(k); poly b(a.begin(), a.begin() + k);
    b = poly_mul(b, poly_inv(c));
    for (int i = 0; i < k; i++) // inv_2 是 2 的逆元
      c[i] = (LL)(c[i] + b[i]) * inv_2 % p; }
  c.resize(a.size()); return c; }
poly poly_derivative(const poly& a) { poly c(a.size());
  for (int i = 1; i < (int)a.size(); i++) // 求导
    c[i - 1] = (LL)a[i] * i % p; return c; }
poly poly_integrate(const poly& a) { poly c(a.size());
  for (int i = 1; i < (int)a.size(); i++) // 不定积分
    c[i] = (LL)a[i - 1] * inv[i] % p; return c; }
poly poly_ln(const poly& a) { // ln，常数项非0，返回长度不变
  auto c = poly_mul(poly_derivative(a), poly_inv(a));
  c.resize(a.size()); return poly_integrate(c); }
// exp，常数项必须是 0，返回长度不变
// 常数很大并且总代码很长，一般可以改用分治 FFT
// 依据：设 G(x) = exp F(x)，则 g_i = 1/i \sum_{k=1}^{i-1} g_{i-k} k f_k
poly poly_exp(const poly& a) { poly c{1};
  for (int k = 2; k <= (int)a.size(); k *= 2) {
    c.resize(k); auto b = poly_ln(c);
    for (int i = 0; i < k; i++)
      b[i] = (a[i] - b[i] + p) % p;
    (++b[0]) %= p; c = poly_mul(b, c);
    memset(c.data() + k, 0, sizeof(int) * k); }
  c.resize(a.size()); return c; }
```

4.4.2 多项式除法 取模

需要抄求逆。

```cpp
poly poly_auto_mul(poly a, poly b) { // 自动判断长度的乘法
  int res_len = (int)a.size() + (int)b.size() - 1;
  int ntt_n = 1; while (ntt_n < res_len) ntt_n *= 2;
  a.resize(ntt_n); b.resize(ntt_n);
  ntt(a.data(), ntt_n, 1); ntt(b.data(), ntt_n, 1);
  for (int i = 0; i < ntt_n; i++)
    a[i] = (LL)a[i] * b[i] % p;
  ntt(a.data(), ntt_n, -1); a.resize(res_len); return a; }
// 多项式除法，a 和 b 长度可以任意
// 商的长度是 n - m + 1，余数的长度是 m - 1
poly poly_div(const poly& a, const poly& b) {
  int n = (int)a.size(), m = (int)b.size();
  if (n < m) return {};
  int ntt_n = 1; while (ntt_n < n - m + 1) ntt_n *= 2;
  poly f(ntt_n), g(ntt_n);
  for (int i = 0; i < n - m + 1; i++) f[i] = a[n - i - 1];
  for (int i = 0; i < m && i < n - m + 1; i++)
    g[i] = b[m - i - 1];
  auto g_inv = poly_inv(g);
  fill(g_inv.begin() + n - m + 1, g_inv.end(), 0);
  auto c = poly_mul(f, g_inv); c.resize(n - m + 1);
  reverse(c.begin(), c.end()); return c; }
// 多项式取模，a 和 b 长度可以任意，返回 (余数, 商)
pair<poly, poly> poly_mod(const poly& a, const poly& b) {
  int n = (int)a.size(), m = (int)b.size();
  if (n < m) return {a, {}};
  auto d = poly_div(a, b); auto c = poly_auto_mul(b, d);
  poly r(m - 1);
  for (int i = 0; i < m - 1; i++)
    r[i] = (a[i] - c[i] + p) % p;
  return {r, d}; }
```

4.4.3 多点求值

需要抄取模。

```
struct poly_eval { poly f; vector<int> x; // 函数和询问点
  | vector<poly> gs; vector<int> ans; // gs 是预处理数组
  | poly_eval(poly f, vector<int> x) : f(f), x(x) {}
  | void pretreat(int l, int r, int o) { poly& g = gs[o];
  |   | if (l == r) { g = poly{p - x[l], 1}; return; }
  |   | int mid = (l + r) / 2; pretreat(l, mid, o * 2);
  |   | pretreat(mid + 1, r, o * 2 + 1);
  |   | if (o > 1)
  |   |   | g = poly_auto_mul(gs[o * 2], gs[o * 2 + 1]); }
  | void solve(int l, int r, int o, const poly& f) {
  |   | if (l == r) { ans[l] = f[0]; return; }
  |   | int mid = (l + r) / 2;
  |   | solve(l, mid, o * 2, poly_mod(f, gs[o * 2]).first);
  |   | solve(mid + 1, r, o * 2 + 1,
  |   |   | poly_mod(f, gs[o * 2 + 1]).first); }
  | vector<int> operator() () { // 包装好的接口
  |   | int n = (int)f.size(), m = (int)x.size();
  |   | if (m <= n) x.resize(m = n + 1);
  |   | else if (n < m - 1) f.resize(n = m - 1);
  |   | int bit_ceil = 1; while (bit_ceil < m) bit_ceil *= 2;
  |   | ntt_init(bit_ceil * 2); // 注意这里 ntt_init 过了
  |   | gs.resize(2 * bit_ceil + 1); pretreat(0, m - 1, 1);
  |   | ans.resize(m); solve(0, m - 1, 1, f); return ans;} };
```

4.4.4 插值

牛顿插值　实现时可以用 k 次差分替代右边的式子，也可以卷积。

$$f(n)=\sum_{i=0}^{k}\binom{n}{i}r_i \iff r_i=\sum_{j=0}^{i}(-1)^{i-j}\binom{i}{j}f(j)$$

拉格朗日插值

$$f(x)=\sum_i f(x_i)\prod_{j\neq i}\frac{x-x_j}{x_i-x_j}$$

4.5 线性递推

$O(k^2\log n)$

```
// Complexity: init O(n^2log) query O(n^2logk)
// Requirement: const LOG const MOD
// Example: In: {1, 3} {2, 1} an = 2an-1 + an-2
// |  |    Out: calc(3) = 7
typedef vector<int> poly;
struct LinearRec {
  | int n; poly first, trans; vector<poly> bin;
poly add(poly &a, poly &b) {
  | poly res(n * 2 + 1, 0);
  | // 不要每次新开 vector，可以使用矩阵乘法优化
  | for (int i = 0; i <= n; ++i) {
  |   | for (int j = 0; j <= n; ++j) {
  |   |   | (res[i+j]+=(LL)a[i] * b[j] % MOD) %= MOD;
  | for (int i = 2 * n; i > n; --i) {
  |   | for (int j = 0; j < n; ++j) {
  |   |   | (res[i-1-j]+=(LL)res[i]*trans[j]%MOD) %=MOD;}
  |   | res[i] = 0; }
  | res.erase(res.begin() + n + 1, res.end());
  | return res; }
LinearRec(poly &first, poly &trans): first(first), trans(trans) {
  | n = first.size(); poly a(n + 1, 0); a[1] = 1;
  | bin.push_back(a); for (int i = 1; i < LOG; ++i)
  |   | bin.push_back(add(bin[i - 1], bin[i - 1])); }
int calc(int k) { poly a(n + 1, 0); a[0] = 1;
  | for (int i = 0; i < LOG; ++i)
  |   | if (k >> i & 1) a = add(a, bin[i]);
  | int ret = 0; for (int i = 0; i < n; ++i)
  |   | if ((ret += (LL)a[i + 1] * first[i] % MOD) >= MOD)
  |   |   | ret -= MOD;
  | return ret; }};
```

$O(k\log k\log n)$ - Bostan-Mori

```
int bostan_mori(int k, poly a, poly b) {
  | int n = (int)a.size(); while (k) { poly c = b;
  |   | for (int i = 1; i < n; i += 2) c[i] = (p - c[i]) % p;
  |   | a = poly_mul(a, c); b = poly_mul(b, c);
  |   | for (int i = 0; i < n; i++) {
  |   |   | a[i] = a[i * 2 + k % 2]; b[i] = b[i * 2]; }
  |   | a.resize(n); b.resize(n); k /= 2; }
  | return (LL)a[0] * qpow(b[0], p - 2) % p; }
// a_n = \sum_{i=1}^m f_i a_{n-i}   (f_0 = 0), f.size() = a.size() + 1
int linear_recurrance(int n, poly f, poly a) {
  | int m = (int)a.size(), ntt_n = 1;
  | while (ntt_n <= m) ntt_n *= 2; ntt_init(ntt_n * 2);
  | f.resize(ntt_n); a.resize(ntt_n); f[0] = 1;
  | for (int i = 1; i <= m; i++) f[i] = (p - f[i]) % p;
  | a = poly_mul(a, f); a.resize(ntt_n);
  | fill(a.data() + m, a.data() + ntt_n, 0);
  | return bostan_mori(n, a, f); }
```

$O(k\log k\log n)$ - 多项式取模

需要抄前面的多项式取模。预处理 $O(k\log k\log n)$，固定 n 和系数只改变初始值的话，询问一次 $O(k)$。注意只询问一次的话不如 Bostan-Mori 快。

```
poly poly_power_mod(LL k, const poly& m) { // x^k mod m
  | poly ans{1}, a{0, 1}; while (k) { if (k & 1)
  |   |   | ans = poly_mod(poly_auto_mul(ans, a), m).first;
  |   | a = poly_mod(poly_auto_mul(a, a), m).first; k /= 2; }
  | return ans; }
// a_n = \sum_{i=1}^m c_i a_{n-i}   (c_0 = 0)
struct linear_recurrence { poly f; // f是预处理结果
  | linear_recurrence(const poly& c, LL n) {
  |   | assert(c[0] == 0); // c[0] 是没有用的
  |   | int m = (int)c.size() - 1;
  |   | int ntt_n = 1; while (ntt_n < m * 2) ntt_n *= 2;
  |   | ntt_init(ntt_n); // 图省事就直接 ntt_init(1 << 18)
  |   | poly t(m + 1); t[m] = 1;
  |   | for (int i = 0; i < m; i++)t[i] = (p - c[m - i]) % p;
  |   | f = poly_power_mod(n, t); }
  | int operator()(const vector<int>& a) { // 0~m-1项初始值
  |   | assert(a.size() == f.size()); int ans = 0;
  |   | for (int i = 0; i < (int)a.size(); i++)
  |   |   | ans = (ans + (LL)f[i] * a[i]) % p;
  |   | return ans; } };
```

4.6 Berlekamp-Massey 最小多项式

如果要求出一个次数为 k 的递推式，则输入的数列需要至少有 $2k$ 项。

返回的内容满足 $\sum_{j=0}^{m-1}a_{i-j}c_j=0$，并且 $c_0=1$。

如果不加最后的处理的话，代码返回的结果会变成 $a_i=\sum_{j=0}^{m-1}c_{j-1}a_{i-j}$，有时候这样会方便接着跑递推，需要的话就删掉最后的处理。

```
vector<int> berlekamp_massey(const vector<int> &a) {
  | vector<int> v, last; // v is the answer, 0-based
  | int k = -1, delta = 0;
  | for (int i = 0; i < (int)a.size(); i++) { int tmp = 0;
  |   | for (int j = 0; j < (int)v.size(); j++)
  |   |   | tmp = (tmp + (LL)a[i - j - 1] * v[j]) % p;
  |   | if (a[i] == tmp) continue;
  |   | if (k < 0) { k = i; delta = (a[i] - tmp + p) % p;
  |   |   | v = vector<int>(i + 1); continue; }
  |   | vector<int> u = v;
  |   | int val = (LL)(a[i] - tmp + p) *
  |   |   | qpow(delta, p - 2) % p;
  |   | if (v.size() < last.size() + i - k)
  |   |   | v.resize(last.size() + i - k);
  |   | (v[i - k - 1] += val) %= p;
  |   | for (int j = 0; j < (int)last.size(); j++) {
  |   |   | v[i - k + j] = (v[i - k + j] -
  |   |   |   | (LL)val * last[j]) % p;
  |   |   | if (v[i - k + j] < 0) v[i - k + j] += p; }
  |   | if ((int)u.size() - i < (int)last.size() - k) {
  |   |   | last = u; k = i; delta = a[i] - tmp;
  |   |   | if (delta < 0) delta += p; } }
  | for (auto &x : v) x = (p - x) % p;
  | v.insert(v.begin(), 1); //一般是需要最小递推式的，处理一下
  | return v; } // \forall i, \sum_{j=0}^m a_{i-j} v_j = 0
```

如果要求向量序列的递推式，就把每位乘一个随机权值 (或者说是乘一个随机行向量 v^T) 变成求数列递推式即可。如果是矩阵序列的话就随机一个行向量 u^T 和列向量 v，然后把矩阵变成 u^TAv 的数列。

优化矩阵快速幂DP　假设 f_i 有 n 维，先暴力求出 $f_{0\sim 2n-1}$，然后跑 Berlekamp-Massey，最后调用快速线性递推即可。

求矩阵最小多项式　矩阵 A 的最小多项式是次数最小的并且 $f(A)=0$ 的多项式 f。实际上最小多项式就是 $\{A^i\}$ 的最小递推式，所以直接调用 Berlekamp-Massey 就好了，显然它的次数不超过 n。

瓶颈在于求出 A^i，实际上我们只要处理 A^iv 就行了，每次对向量做递推。

求稀疏矩阵的行列式　如果能求出特征多项式，则常数项乘上 $(-1)^n$ 就是行列式，但是最小多项式不一定就是特征多项式。

把 A 乘上一个随机对角阵 B，则 AB 的最小多项式有很大概率就是特征多项式，最后再除掉 $\det B$ 就行了。

求稀疏矩阵的秩 设 A 是一个 $n\times m$ 的矩阵, 首先随机一个 $n\times n$ 的对角阵 P 和一个 $m\times m$ 的对角阵 Q, 然后计算 $QAPA^TQ$ 的最小多项式即可。实际上不用计算这个矩阵，因为求最小多项式时要用它乘一个向量，我们依次把这几个矩阵乘到向量里就行了。答案就是最小多项式除掉所有 x 因子后剩下的次数。

解稀疏方程组 $Ax=b$，其中 A 是一个 $n\times n$ 的满秩稀疏矩阵，b 和 x 是 $1\times n$ 的列向量，A,b 已知，需要解出 x。

做法：显然 $x=A^{-1}b$. 如果我们能求出 $\{A^ib\}(i\ge 0)$ 的最小递推式 $\{r_{0\ldots m-1}\}(m\le n)$，那么就有结论

$$A^{-1}b=-\frac{1}{r_{m-1}}\sum_{i=0}^{m-2}A^ibr_{m-2-i}$$

因为 A 是稀疏矩阵，直接按定义递推出 $b\ldots A^{2n-1}b$ 即可。

```cpp
vector<int> solve_sparse_equations(const vector<tuple<int, int, int> > &A, const vector<int> &b) {
  int n = (int)b.size(); // 0-based
  vector<vector<int> > f({b});
  for (int i = 1; i < 2 * n; i++) {
    vector<int> v(n); auto &u = f.back();
    for (auto [x, y, z] : A) // [x, y, value]
      v[x] = (v[x] + (long long)u[y] * z) % p;
    f.push_back(v); }
  vector<int> w(n); mt19937 gen;
  for (auto &x : w)
    x = uniform_int_distribution<int>(1, p - 1)(gen);
  vector<int> a(2 * n);
  for (int i = 0; i < 2 * n; i++)
    for (int j = 0; j < n; j++)
      a[i] = (a[i] + (long long)f[i][j] * w[j]) % p;
  auto c = berlekamp_massey(a); int m = (int)c.size();
  vector<int> ans(n);
  for (int i = 0; i < m - 1; i++)
    for (int j = 0; j < n; j++)
      ans[j] = (ans[j] +
        (long long)c[m - 2 - i] * f[i][j]) % p;
  int inv = qpow(p - c[m - 1], p - 2);
  for (int i = 0; i < n; i++)
    ans[i] = (long long)ans[i] * inv % p;
  return ans; }
```

4.7 FWT

```cpp
/* |
And: (1,1; 0,1) (1,−1; 0,1) Or: (1,0; 1,1) (1,0; −1,1) Xor: (1,1; 1,−1) (0.5,0.5; 0.5,−0.5)
IFWT的矩阵时FWT的逆，对于任意运算 ⊕，满足FWT的矩阵需要：
C[i][j] × C[i][k] = C[i][j ⊕ k]
对于不存在FWT矩阵的运算：通过映射01变成另外一个可行的运算。*/
const LL XOR[2][2] = {{1, 1}, {1, M-1}};
const LL i2 = (M+1)/2, iXOR[2][2] = {{i2, i2}, {i2, M-i2}};
void FWT(LL f[], const LL C[2][2], int n) {
  for (int t = 1; t < n; t <<= 1) {
  for (int l = 0; l < n; l += t + t) {
    for (int i = 0; i < t; i++) {
    LL x = f[l + i], y = f[l + t + i];
    f[l + i]     = (C[0][0] * x + C[0][1] * y) % M;
    f[l + t + i] = (C[1][0] * x + C[1][1] * y) % M;
    } } } }
```

4.8 K 进制 FWT

```cpp
// n : power of k, omega[i] : (primitive kth root) ^ i
void fwt(int* a, int k, int type) {
  static int tmp[K];
  for (int i = 1; i < n; i *= k)
  for (int j = 0, len = i * k; j < n; j += len)
    for (int low = 0; low < i; low++) {
    for (int t = 0; t < k; t++)
      tmp[t] = a[j + t * i + low];
    for (int t = 0; t < k; t++){
      int x = j + t * i + low;
      a[x] = 0;
      for (int y = 0; y < k; y++)
      a[x] = int(a[x] + 1ll * tmp[y] * omega[(k + type) * t * y % k] % MOD);
    }
    }
  if (type == -1)
  for (int i = 0, invn = inv(n); i < n; i++)
    a[i] = int(1ll * a[i] * invn % MOD); }
```

5. 线性代数

5.1 Simplex 单纯形

```cpp
const LD eps = 1e-9, INF = 1e9; const int N = 105;
namespace Simplex {
int n, m, id[N], tp[N]; LD a[N][N];
void pivot(int r, int c) {
  swap(id[r + n], id[c]);
  LD t = -a[r][c]; a[r][c] = -1;
  for (int i = 0; i <= n; i++) a[r][i] /= t;
  for (int i = 0; i <= m; i++) if (a[i][c] && r != i) {
    t = a[i][c]; a[i][c] = 0;
    for (int j = 0; j <= n; j++) a[i][j] += t*a[r][j];}}
bool solve() {
  for (int i = 1; i <= n; i++) id[i] = i;
  for ( ; ; ) {
    int i = 0, j = 0; LD w = -eps;
    for (int k = 1; k <= m; k++)
      if (a[k][0] < w || (a[k][0] < -eps && rand() & 1))
        w = a[i = k][0];
    if (!i) break;
    for (int k = 1; k <= n; k++)
      if (a[i][k] > eps) {j = k; break;}
    if (!j) { printf("Infeasible"); return 0;}
    pivot(i, j);}
  for ( ; ; ) {
    int i = 0, j = 0; LD w = eps, t;
    for (int k = 1; k <= n; k++)
      if (a[0][k] > w) w = a[0][j = k];
    if (!j) break;
    w = INF;
    for (int k = 1; k <= m; k++)
      if (a[k][j] < -eps && (t = -a[k][0]/a[k][j]) < w)
        w = t, i = k;
    if (!i) { printf("Unbounded"); return 0;}
    pivot(i, j);}
  return 1;}
LD ans() {return a[0][0];}
void output() {
  for (int i = n + 1; i <= n + m; i++) tp[id[i]] = i - n;
  for (int i = 1; i <=n; i++) printf("%.9lf ", tp[i] ? a[tp[i]][0] : 0);}
}using namespace Simplex;
int main() { int K; cin >> n >> m >> K;
for (int i = 1; i <= n; i++) {LD x; cin >> x; a[0][i] = x;}
for (int i = 1; i <= m; i++) {LD x;
  for (int j = 1; j <= n; j++) cin >> x, a[i][j] = -x;
  cin >> x; a[i][0] = x;}
if (solve()) { printf("%.9lf\n", (LD)ans()); if (K) output();}}
// 标准型: maximize c^T x, subject to Ax ≤ b and x ≥ 0
// 对偶型: minimize b^T y, subject to A^T x ≥ c and y ≥ 0
```

5.2 高斯消元最小范数解

```cpp
typedef vector <LD> vec; /* sum a[i][0..d] = 0 */
pair<vec,vector<vec>> gauss(vector<vec> &a, int n, int d) {
  vector <int> pivot(d, -1);
  for (int i = 0, o = 0; i < d; i++) {
    int j = o; while (j < n && abs(a[j][i]) < eps) j++;
    if (j == n) continue;
    swap(a[j], a[o]); LD w = a[o][i];
    for (int k = 0; k <= d; k++) a[o][k] /= w;
    for (int x = 0; x < n; x++)
      if (x != o && abs(a[x][i]) > eps) {
        w = a[x][i];
        for (int k = 0; k <= d; k++)
          a[x][k] -= a[o][k] * w;    }
    pivot[i] = o++;
  } vec x0(d); vector <vec> t; int free = 0;
  for (int i = 0; i < d; i++)
    if (pivot[i] != -1) x0[i] = -a[pivot[i]][d];
    else free ++;
  for (int i = 0; i < d; i++) if (pivot[i] == -1) {
    vec x(d); x[i] = -1;
    for (int j = 0; j < d; j++)
      if (pivot[j] != -1) x[j] = a[pivot[j]][i];
    t.push_back(x);
  } if (t.size()) {
    vector <vec> f;
    for (int u = 0; u < free; u++) {
      vec x(free + 1);
      for (int i = 0; i < free; i++)
        for (int j = 0; j < d; j++)
```

```
          x[i] += t[u][j] * t[i][j];
      for (int j = 0; j < d; j++)
        x[free] += t[u][j] * x0[j];
      f.push_back(x);
    }
    auto [k, tt] = gauss(f, free, free);
    assert (tt.size() == 0);
    for (int x = 0; x < free; x++)
      for (int i = 0; i < d; i++)
        x0[i] += k[x] * t[x][i];
  } return {x0, t}; }
```

6. 数据结构

6.1 bitset

```
template<int len = 1> // 动态size的bitset
void solve(int x) {
  if(siz[x] > len) return (void) solve<min(len*2, MAXN)>(x);
  bitset<len> dp; }
```

6.2 FHQ treap

```
struct FHQ{
int val[N], dat[N], siz[N], ls[N], rs[N], tot = 0, rt;
void push_up(int x) {siz[x] = siz[ls[x]] + siz[rs[x]] + 1;}
int merge(int x, int y){
  if(!x || !y) return x+y;
  if(dat[x] < dat[y]) {push_down(x); rs[x] = merge(rs[x], y); push_up(x); return x;}
  else {push_down(y); ls[y] = merge(x, ls[y]); push_up(y); return y;} }
void splitVal(int p, int k, int& x, int& y){
  if(!p) {x = y = 0; return;}
  push_down(p);
  if(val[p] <= k) {x = p; splitVal(rs[x], k, rs[x], y);}
  else {y = p; splitVal(ls[y], k, x, ls[y]);}
  push_up(p);
}
void splitSiz(int p, int k, int& x, int& y){
  if(!p) {x = y = 0; return;}
  push_down(p);
  if(siz[ls[p]] < k) {x = p; splitSiz(rs[p], k-siz[ls[p]]-1, rs[x], y);}
  else {y = p; splitSiz(ls[p], k, x, ls[y]);}
  push_up(p);
}
int new_node(int v){
  siz[++tot] = 1; val[tot] = v;
  dat[tot] = rd(); return tot;}
void ins(int v){
  int x, y; splitVal(rt, v, x, y);
  rt = merge(merge(x, new_node(v)), y); }
void del(int v){
  int x, y, z;
  splitVal(rt, v, x, z), splitVal(x, v-1, x, y);
  y = merge(ls[y], rs[y]); rt = merge(merge(x, y), z); }
int rk(int v){
  int x, y; splitVal(rt, v-1, x, y);
  int t = siz[x] + 1; rt = merge(x, y); return t; }
int kth(int p, int k){
  if(!p) return 0;
  while(true){
    if(siz[ls[p]] >= k) p = ls[p];
    else if(siz[ls[p]] + 1 == k) return val[p];
    else k -= siz[ls[p]] + 1, p = rs[p]; } }
int pre(int v){
  int x, y; splitVal(rt, v-1, x, y);
  int t = kth(x, siz[x]); rt = merge(x, y); return t; }
int nxt(int v){
  int x, y; splitVal(rt, v, x, y);
  int t = kth(y, 1); rt = merge(x, y); return t; }
int s[MAXN]; // 线性建立FHQ
int build(int n){
  int top = 0;
  for(int i=1;i<=n;i++){
    int tmp = new_node(a[i]), last = 0;
    while(top && dat[s[top]] > dat[tmp])
      push_up(s[top]), last = s[top--];
    if(top) rs[s[top]] = tmp; ls[tmp] = last, s[++top] = tmp;
  }
  while(top) push_up(s[top--]); return s[1]; }
} tr;
```

6.3 RangeBIT

```
struct BIT{
  #define lowbit(x) (x & (-x))
  ll d[MAXN], d1[MAXN];
  void upd(int x, ll k) {
    for (int pos = x; x <= n; x += lowbit(x))
      d[x] += k, d1[x] += (pos - 1) * k;
  }
  void upd(int l, int r, ll k) {upd(l, k), upd(r+1, -k);}
  ll qry(int x) {
    ll ret = 0, pos = x;
    for (int pos = x; x; x -= lowbit(x))
      ret += d[x] * pos - d1[x];
    return ret;
  }
  ll qry(int l, int r) {return qry(r) - qry(l-1);}
};
```

6.4 李超线段树

```
namespace Li{ // 默认取max 可持久化李超记得从x转移而非p
  int tag[MAXN];
  double k[MAXN], b[MAXN];
  const double EPS = 1e-10;
  double calc(int i, double x) {return x * k[i] + b[i];}
  void update(int nl, int nr, int l, int r, int p, int x){
    if(l == r) {if(calc(tag[p], l) < calc(x, l)) tag[p] = x; return;}
    int mid = (l+r)>>1;
    if(nl <= l && r <= nr){
      if(!tag[p]) {tag[p] = x; return;}
      double l1 = calc(tag[p], l), l2 = calc(x, l), r1 = calc(tag[p], r), r2 = calc(x, r);
      if(abs(l1 - l2) < EPS && abs(r1 - r2) < EPS) return;
      if(l1 > l2 && r1 > r2) return;
      if(l2 > l1 && r2 > r1) tag[p] = x;
      if(calc(tag[p], mid) < calc(x, mid)) swap(x, tag[p]);
      if(k[x] > k[tag[p]]) update(nl, nr, mid+1, r, rs, x);
      if(k[x] < k[tag[p]]) update(nl, nr, l, mid, ls, x);
      return;
    }
    if(nl <= mid) update(nl, nr, l, mid, ls, x);
    if(nr > mid) update(nl, nr, mid+1, r, rs, x);
  }
  double query(int pos, int l, int r, int p){
    if(l == r) return calc(tag[p], pos);
    int mid = (l+r)>>1;
    if(pos <= mid) return max(calc(tag[p], pos), query(pos, l, mid, ls));
    else return max(calc(tag[p], pos), query(pos, mid+1, r, rs));
  }
}
```

6.5 线段树分裂

```
void split(int& x, int& y, int nl, int nr, int l, int r){
  if(nl <= l && r <= nr) {y = x, x = 0; return;}
  y = new_node();
  int mid = (l+r)>>1;
  if(nl <= mid) split(ls[x], ls[y], nl, nr, l, mid);
  if(nr > mid) split(rs[x], rs[y], nl, nr, mid+1, r);
  push_up(x), push_up(y);
}
```

6.6 左偏树

```
namespace Leftist{
  int a[MAXN], ls[MAXN], rs[MAXN], dis[MAXN], fa[MAXN];
  bool vis[MAXN]; // dis[0] = -1;
  void init() {for(int i=1;i<=n;i++) fa[i] = i;}
  int find(int x) {return fa[x] == x ? x : fa[x] = find(fa[x]);}
  int _merge(int x, int y){
    if(!x || !y) return x + y;
    if(a[y] < a[x]) swap(x, y);
    rs[x] = _merge(rs[x], y);
```

```cpp
    if(dis[ls[x]] < dis[rs[x]]) swap(ls[x], rs[x]);
    dis[x] = dis[rs[x]] + 1;
    return x;
  }
  void merge(int x, int y){
    if(vis[x] ||vis[y]) return;
    x = find(x), y = find(y);
    if(x != y) fa[x] = fa[y] = _merge(x, y);
  }
  int del(int x){
    if(vis[x]) return -1; // deleted flag
    x = find(x); vis[x] = true;
    int ret = a[x].val;
    fa[ls[x]] = fa[rs[x]] = fa[x] = _merge(ls[x], rs[x]);
    ls[x] = rs[x] = dis[x] = 0;
    return ret;
  }
}
```

6.7 非递归线段树

6.7.1 区间加，区间求最大值

```cpp
void update(int l, int r, int d) {
  for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
    if (l < M) {
      t[l] = max(t[l*2], t[l*2+1]) + mark[l];
      t[r] = max(t[r*2], t[r*2+1]) + mark[r]; }
    if (~l & 1) { t[l ^ 1] += d; mark[l ^ 1] += d; }
    if (r & 1) { t[r ^ 1] += d; mark[r ^ 1] += d; } }
  for (; l; l >>= 1, r >>= 1)
    if (l < M) t[l] = max(t[l*2], t[l*2+1]) + mark[l],
            t[r] = max(t[r*2], t[r*2+1]) + mark[r]; }
int query(int l, int r) {
  int maxl = -INF, maxr = -INF;
  for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
    maxl += mark[l]; maxr += mark[r];
    if (~l & 1) maxl = max(maxl, t[l ^ 1]);
    if ( r & 1) maxr = max(maxr, t[r ^ 1]); }
  while (l) { maxl += mark[l]; maxr += mark[r];
    l >>= 1; r >>= 1; }
  return max(maxl, maxr); }
```

6.8 LCT 动态树

```cpp
namespace LCT{
#define ls(x) c[x][0]
#define rs(x) c[x][1]
int val[N], ans[N], c[N][2], f[N], rev[N];
void push_up(int x){
  ans[x] = max({ans[ls(x)], ans[rs(x)], val[x]}); }
void opt(int x) {swap(ls(x), rs(x)), rev[x] ^= 1;}
void push_down(int x) {
  if(rev[x]) opt(ls(x)), opt(rs(x)), rev[x] = 0;}
bool nrt(int x) {return ls(f[x]) == x || rs(f[x]) == x;}
bool get(int x) {return x == rs(f[x]);}
void rot(int x){
  int y = f[x], z = f[y], t = get(x), w = c[x][t^1];
  if(nrt(y)) c[z][get(y)] = x;
  c[x][t^1] = y, c[y][t] = w;
  if(w) {f[w] = y;} f[y] = x, f[x] = z; push_up(y); }
void push_all(int x) {if(nrt(x)) push_all(f[x]); push_down(x);}
void splay(int x){
  push_all(x);
  while(nrt(x)){
    int y = f[x];
    if(nrt(y)) rot(get(x) == get(y) ? y : x);
    rot(x);
  } push_up(x); }
void access(int x) {for(int y=0;x;y=x, x=f[x]) splay(x), rs(x) = y, push_up(x);}
void mkrt(int x) {access(x); splay(x); opt(x);}
int findrt(int x){
  access(x), splay(x);
  while(ls(x)) push_down(x), x = ls(x);
  splay(x); return x; }
void split(int x, int y) {mkrt(x), access(y), splay(y);}
bool reach(int x, int y) {mkrt(x); return findrt(y) == x;}
void link(int x, int y) {if(!reach(x, y)) f[x] = y;}
void cut(int x, int y) {if(reach(x, y) && f[y] == x && !ls(y)) f[y] = rs(x) = 0;}
// 维护子树信息树的直径，F[x]:splay子树深度最小的点往下最大深度
void push_up(int x){
  siz[x] = siz[ls(x)] + siz[rs(x)] + 1;
  F[x] = max(F[ls(x)], F[rs(x)] + siz[ls(x)] + 1);
  G[x] = max(G[rs(x)], G[ls(x)] + siz[rs(x)] + 1);
  if(s[x].size())
    F[x] = max(F[x], *(--s[x].end()) + 1 + siz[ls(x)]),
    G[x] = max(G[x], *(--s[x].end()) + 1 + siz[rs(x)]); }
void opt(int x) {swap(ls(x), rs(x)), swap(F[x], G[x]), rev[x] ^= 1;}
void access(int x) {
  for(int y=0;x;y=x, x=f[x]) {
    splay(x);
    if(rs(x)) s[x].insert(F[rs(x)]);
    if(rs(x) = y) s[x].erase(s[x].find(F[y]));
    push_up(x); } }
void link(int x, int y) {mkrt(x);if(findrt(y) != x) mkrt(y), f[x] = y, s[y].insert(F[x]);}
#undef ls
#undef rs
}
```

6.9 Hash Table

```cpp
template <class T,int P = 314159/*,451411,1141109,2119969*/>
struct hashmap {
ULL id[P]; T val[P];
int R[P]; // del: few clears
hashmap() {memset(id, -1, sizeof id);}
T get(const ULL &x) const {
  for (int i = int(x % P), j = 1; ~id[i]; i = (i + j) % P, j = (j + 2) % P /*unroll if needed*/) {
    if (id[i] == x) return val[i]; }
  return 0; }
T& operator [] (const ULL &x) {
  for (int i = int(x % P), j = 1;     ; i = (i + j) % P, j = (j + 2) % P) {
    if (id[i] == x) return val[i];
    else if (id[i] == -1llu) {
      id[i] = x;
      R[++R[0]] = i;  // del: few clears
      return val[i]; } } }
void clear() { // del: few clears
for (int &x = R[0]; x; id[R[x]] = -1, val[R[x]] = 0, --x);
void fullclear() {memset(id, -1, sizeof id); R[0] = 0; } };
```

6.10 基数排序

```cpp
const int SZ = 1 << 8; // almost always fit in L1 cache
void SORT(int a[], int c[], int n, int w) {
  for(int i=0; i<SZ; i++) b[i] = 0;
  for(int i=1; i<=n; i++) b[(a[i]>>w) & (SZ-1)]++;
  for(int i=1; i<SZ; i++) b[i] += b[i - 1];
  for(int i=n; i; i--) c[b[(a[i]>>w) & (SZ-1)]--] = a[i];}
void Sort(int *a, int n){
  SORT(a, c, n, 0); SORT(c, a, n, 8);
  SORT(a, c, n, 16); SORT(c, a, n, 24); }
```

7. 树上问题

7.1 虚树

```cpp
bool cmp(int x, int y) {return id[x] < id[y];}
void build(vector<int> v){
  sort(v.begin(), v.end(), cmp);
  int top = 0;
  s[top=1] = 1; vt::e[1].clear();
  for(auto x : v){
    if(x == 1) continue;
    int l = lca(x, s[top]);
    if(l != s[top]){
      while(id[l] < id[s[top-1]]) vt::push(s[top-1], s[top]), top--;
      if(l != s[top-1]) vt::e[l].clear(), vt::push(l, s[top]), s[top] = l;
      else vt::push(l, s[top--]);
    }
    s[++top] = x; vt::e[x].clear();
  }
  for(int i=1;i<top;i++) vt::push(s[i], s[i+1]);
}
```

7.2 树哈希

```cpp
ULL get(vector <ULL> ha) {
  sort(ha.begin(), ha.end());
  ULL ret = 0xdeadbeef;
```

```
| for (auto i : ha) {
| | ret = ret * P + i, ret ^= ret << 17;
| } return ret * 997; }
```

8. 计算几何

8.1 二维几何基础操作

```
#define cp const point &
int turn (cp a, cp b, cp c) { return sgn(det(b-a, c-a)); }
vector <point> convex_hull (vector <point> a) {
| sort (a.begin(), a.end()); // 小于号 (y, x) 字典序
| a.erase(unique(a.begin(), a.end()), a.end());//必要时去重
| int n = (int) a.size (), cnt = 0;
| vector <point> ret;
| for (int i = 0; i < n; ++i) {
| | while (cnt > 1
| | && turn (ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
| | | --cnt, ret.pop_back (); // 保留凸包边界上的点: < 0
| | ++cnt, ret.push_back (a[i]); }
| for (int i = n - 2, fixed = cnt; i >= 0; --i) {
| | while (cnt > fixed
| | && turn (ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
| | | --cnt, ret.pop_back (); // 保留凸包边界上的点: < 0
| | ++cnt, ret.push_back (a[i]); }
| if (n > 1) ret.pop_back (); // n <= 2 是凸包吗?
| return ret; } // 小于号为 (y, x) 时边 [0, 2pi) 逆时针
```

```
vector <point> add (vector <point> a, vector <point> b) {
// size > 0, rotate(begin, min, end), 无重, 小于号 (y, x)
| if (a.size() == 1 || b.size() == 1) {
| | vector <point> ret;
| | for (auto i : a) for (auto j : b) ret.push_back(i+j);
| | return ret; }
| vector <point> x, y;
| for (int i = 0; i < a.size(); i++)
| | x.push_back(a[(i + 1) % a.size()] - a[i]);
| for (int i = 0; i < b.size(); i++)
| | y.push_back(b[(i + 1) % b.size()] - b[i]);
| vector <point> ret (x.size() + y.size());
| merge(x.begin(), x.end(), y.begin(), y.end(),
| |   ret.begin(), [](cp u, cp v) {
| | return half(u)-half(v) ? half(u) : det(u, v) > 0;});
| point cur = a[0] + b[0];
| for (auto &i : ret) swap(i, cur), cur = cur + i;
| return ret; } // ret 可能共线, 但没有重点
```

```
struct point {
| point rot(LD t) const { // 逆时针
| | return {x*cos(t) - y*sin(t), x*sin(t) + y*cos(t)};}
| point rot90() const { return {-y, x}; }};
bool two_side(cp a, cp b, cl c) {
| return turn(c.s, c.t, a) * turn(c.s, c.t, b) < 0; }
point line_inter(cl a, cl b) { // 直线交点
| LD s1 = det(a.t - a.s, b.s - a.s);
| LD s2 = det(a.t - a.s, b.t - a.s);
| return (b.s * s2 - b.t * s1) / (s2 - s1); }
vector <point> cut (const vector<point> &c, line l) {
| vector <point> ret; // 线切凸包
| int n = (int) c.size(); if (!n) return ret;
| for (int i = 0; i < n; ++i) {
| | int j = (i + 1) % n;
| | if (turn (l.s, l.t, c[i]) >= 0) ret.push_back(c[i]);
| | if (two_side (c[i], c[j], l))
| | | ret.push_back(line_inter(l, {c[i], c[j]})); }
| return ret; }
bool pos (cp a,cl b){ // point_on_segment 点在线段上
| return turn(b.s, b.t, a) == 0 // 在直线上
| && sgn(dot(b.s - a, b.t - a)) <= 0;}
bool inter_judge(cl a,cl b) { // 线段判非严格交
| if (pos (b.s, a) || pos (b.t, a)) return true;
| if (pos (a.s, b) || pos (a.t, b)) return true;
| return two_side (a.s, a.t, b)
| | && two_side (b.s, b.t, a); }
point proj_to_line (cp a, cl b) { // 点在直线投影
| point st = b.t - b.s;
| return b.s + st * (dot(a - b.s, st) / dot(st, st));}
LD p2l (cp a, cl b) { // point_to_line
| return abs(det(b.t-b.s, a-b.s)) / dis(b.s, b.t); }
LD p2s (cp a, cl b) { // point_to_segment 注意退化
| if ( sgn(dot(b.s - a, b.t - b.s))
|     * sgn(dot(b.t - a, b.t - b.s)) >= 0)
| | return min (dis (a, b.s), dis (a, b.t));
| return p2l (a, b); }
bool point_on_ray (cp a, cl b) { // 点在射线上
| return turn (b.s, b.t, a) == 0
| && sgn(dot(a - b.s, b.t - b.s)) >= 0; }
bool ray_inter_judge(line a, line b) { // 射线判交
| LD s1, s2; // can be LL
| s1 = det(a.t - a.s, b.s - a.s);
| s2 = det(a.t - a.s, b.t - a.s);
| if (sgn(s1) == 0 && sgn(s2) == 0) {
| | return sgn(dot(a.t - a.s, b.s - a.s)) >= 0
| |  | || sgn(dot(b.t - b.s, a.s - b.s)) >= 0; }
| if (!sgn(s1 - s2) || sgn(s1) == sgn(s2 - s1)) return 0;
| swap(a, b);
| s1 = det(a.t - a.s, b.s - a.s);
| s2 = det(a.t - a.s, b.t - a.s);
| return sgn(s1) != sgn(s2 - s1); }
```

```
int half(cp a){return a.y > 0||(a.y == 0 && a.x > 0)?1:0;}
bool turn_left(cl a, cl b, cl c) {
  return turn(a.s, a.t, line_inter(b, c)) > 0; }
bool is_para(cl a, cl b){return!sgn(det(a.t-a.s,b.t-b.s));}
bool cmp(cl a, cl b) {
  int sign = half(a.t - a.s) - half(b.t - b.s);
  int dir = sgn(det(a.t - a.s, b.t - b.s));
  if (!dir && !sign) return turn(a.s, a.t, b.t) < 0;
  else return sign ? sign > 0 : dir > 0; }
vector <point> hpi(vector <line> h) { // 半平面交
  sort(h.begin(), h.end(), cmp);
  vector <line> q(h.size()); int l = 0, r = -1;
  for(auto &i : h) {
   while (l < r && !turn_left(i, q[r - 1], q[r])) --r;
   while (l < r && !turn_left(i, q[l], q[l + 1])) ++l;
   if (l <= r && is_para(i, q[r])) continue;
   q[++r] = i; }
  while (r - l > 1 && !turn_left(q[l], q[r - 1], q[r])) --r;
  while (r - l > 1 && !turn_left(q[r], q[l], q[l + 1])) ++l;
  if(r - l < 2) return {};
  vector <point> ret(r - l + 1);
  for(int i = l; i <= r; i++)
 | ret[i - l] = line_inter(q[i], q[i == r ? l : i + 1]);
  return ret; }
// 空集会在队列里留下一个开区域; 开区域会被判定为空集。
// 为了保证正确性, 一定要加足够大的框, 尽可能避免零面积区域。
// 实在需要零面积区域边缘, 需要仔细考虑 turn_left 的实现。
```

8.2 整数半平面交

```
struct line : point {
| LD z; // ax + by + c >= 0
| line () {}
| line (LD a, LD b, LD c): point(a, b), z(c) {}
| line (cp a, cp b): point((b-a).rot90(), z(det(a, b)){}
| LD operator () (cp a) const{return dot(a, *this) + z;}};
point line_inter (cl u, cl v) {
return point(det({u.z, u.y}, {v.z, v.y}),
| |  | det({u.x, u.z}, {v.x, v.z}) ) / -det(u, v); }
LD dis (cl l, cp x = {0, 0}) { return l(x) / l.len(); }
bool is_para(cl x, cl y) { return !sgn(det(x, y)); }
LD det(cl a, cl b, cl c) {
| return det(a,b)*c.z + det(b,c)*a.z + det(c,a)*b.z;}
int check(cl a, cl b, cl c) { // sgn left(a, inter(b, c))
| return sgn(det(b, c, a)) * sgn(det(b, c)); }
bool turn_left(cl a, cl b, cl c){return check(a, b, c) >0;}
bool cmp (cl a, cl b) {
if (is_para(a, b) && dot(a, b) > 0) return dis(a) < dis(b);
return half(a) == half(b) ? sgn(det(a,b))>0 : half(b)>0;}
// 用以上函数替换 HPI 函数, 需要 half(point)
line perp(cl l) { return {l.y, -l.x, 0}; } // 垂直
line para(cl l, cp o) { // 过一点平行
| return {l.x, l.y, l.z - l(o)}; }
point proj(cp x, cl l) {return x - l * (l(x)/l.len2());}
point refl(cp x, cl l) {return x - l * (l(x)/l.len2())*2;}
bool is_perp(cl x, cl y) { return !sgn(dot(x, y)); }
LD area(cl a, cl b, cl c) { // 0 when error
| LD d = det(a, b, c);
| return d * d / (det(a, b) * det(b, c) * det(c, a)); }
vector <line> cut (const vector <line>& o, line l){
| vector <line> ret; int n = (int) o.size();
| for (int i = 0; i < n; i++) {
| | cl u = o[i], v = o[(i+1) % n], w = o[(i + 2) % n];
| | int va = check(l, u, v), vb = check(l, v, w);
| | if (va > 0 || vb > 0 || (va == 0 && vb == 0))
| | | ret.push_back(v);
| | if (va >= 0 && vb < 0) ret.push_back(l);
```

```
| } return ret; }
```

8.3 凸包询问：凸包内、切点、交点、最近点

```
int n; vector <point> a; // 可以封装成一个 struct
bool inside (cp u) { // 点在凸包内
 | int l = 1, r = n - 2;
 | while (l < r) {
 | | int mid = (l + r + 1) / 2;
 | | if (turn(a[0], a[mid], u) >= 0) l = mid;
 | | else r = mid - 1; }
 | return turn (a[0], a[l], u) >= 0
 | | && turn (a[l], a[l + 1], u) >= 0
 | | && turn (a[l + 1], a[0], u) >= 0; }
int search (auto f) { // 凸包求极值，需要 C++17
 | int l = 0, r = n - 1, d = 1;
 | if (f(a[r], a[l])) swap(l, r), d = -1;
 | while (d * (r - l) > 1) {
 | | int mid = (l + r) / 2;
 | | if (f(a[mid], a[l]) && f(a[mid], a[mid - d])) l = mid;
 | | else r = mid; } return l; }
pair<int, int> get_tan(cp u) { // 求切线
 return // 严格在凸包外；需要边界上时，特判 a[n-1] -> a[0]
 {search([&](cp x, cp y){return turn(u, y, x) > 0;}),
  search([&](cp x, cp y){return turn(u, x, y) > 0;})};}
point at (int i) { return a[i % n]; }
int inter (cp u, cp v, int l, int r) {
 | int sl = turn(u, v, at(l));
 | while (l + 1 < r) {
 | | int m = (l + r) / 2;
 | | if (sl == turn(u, v, at(m))) l = m;
 | | else r = m; | } return l % n; }
bool get_inter(cp u, cp v, int &i, int &j) { // 求直线交点
 | int p0 = search([&](cp x, cp y){
 |  return det(v - u, x - u) < det(v - u, y - u);}),
 | p1 = search([&](cp x, cp y) {
 |  return det(v - u, x - u) > det(v - u, y - u);});
 | if (turn(u, v, a[p0]) * turn(u, v, a[p1]) < 0) {
 | | if (p0 > p1) swap(p0, p1);
 | | i = inter (u, v, p0, p1);
 | | j = inter (u, v, p1, p0 + n);
 | | return true; } else return false; }
LD near (cp u, int l, int r) {
 | if (l > r) r += n;
 | int sl = sgn (dot(u - at(l), at(l + 1) - at(l)));
 | LD ret = p2s (u, {at(l), at(l + 1)});
 | while (l + 1 < r) {
 | | int m = (l + r) / 2;
 | | if (sl == sgn (dot(u - at(m), at(m + 1) - at(m))))
 | |  | l = m; else r = m; }
   return min(ret, p2s(u, {at(l), at(l + 1)})); }
LD get_near (cp u) { // 求凸包外点到凸包最近点
 | if (inside(u)) return 0;
 | auto [x, y] = get_tan(u);
 | return min(near(u, x, y), near(u, y, x)); }
```

8.4 点、线段在简单多边形内

```
bool point_in_polygon (cp u, const vector <point> & p) {
 | int n = (int) p.size (), cnt = 0;
 | for (int i = 0; i < n; ++i) {
 | | point a = p[i], b = p[(i + 1) % n];
 | | if (pos (u, {a, b})) return true;
 | | int x = turn (a, u, b);
 | | int y = sgn (a.y - u.y);
 | | int z = sgn (b.y - u.y);
 | | if (x > 0 && y <= 0 && z > 0) ++cnt;
 | | if (x < 0 && z <= 0 && y > 0) --cnt; }
 | return cnt != 0; } // < 0 在逆时针多边形内; > 0 顺时针
bool in_polygon (cp u, cp v) {
 | // u, v in polygon; p contain those u, v on border
 | for (int i = 0; i < n; i++) {
 | | int j = (i + 1) % n, k = (i + 2) % n;
 | | cp ii = p[i], jj = p[j], kk = p[k];
 | | if (inter_judge_strict({u, v}, {ii, jj})) return 0;
 | | if (point_on_segment (jj, {u, v})) {
 | | | bool good = true, left = turn (ii, jj, kk) >= 0;
 | | | for (auto x : {u, v})
 | | | | if (left)
 | | | | | good &= | turn(ii, jj, x) >= 0
 | | | | | | | && turn(jj, kk, x) >= 0;
 | | | | else
 | | | | | good &= !(   turn(jj, x, kk) > 0
 | | | | | | |    && turn(jj, ii, x) > 0);
 | | | if (!good) return 0;
 | | } } return 1; }
LD get_far (int uid, int vid) {
 | // u -> v in polygon, check the ray u -> polygon
 | cp u = p[uid], v = p[vid];
 | LD far = 1e9;
 | for (int i = 0; i < n; i++) {
 | | int j = (i + 1) % n, k = (i + 2) % n;
 | | cp ii = p[i], jj = p[j], kk = p[k];
 | | if (two_side (ii, jj, {u, v})) {
 | | | LD s1 = det(jj - ii, u - ii);
 | | | LD s2 = det(jj - ii, v - ii);
 | | | if (sgn(s1 - s2) && sgn(s1) != sgn(s2 - s1))
 | | | | far = min(far,
 | | | | | |    dis(u, line_inter({ii,jj},{u,v})));}
 | | if (j != uid && point_on_ray (jj, {u, v})) {
 | | | bool good = turn(ii, jj, kk) <= 0;
 | | | for (auto x : {u - (jj - u), jj + (jj - u)})
 | | | | good &= !(   turn(ii, jj, x) > 0
 | | | | | |    && turn(jj, kk, x) > 0);
 | | | if (!good) far = min(far, dis(u, jj));
 | | } } return far; }
void work() {
 | for (int i = 0; i < n; i++)
 | | for (int j = 0; j < n; j++) if (i != j) {
 | | | if (!in_polygon(p[i], p[j])) continue;
 | | | LD ret = get_far(i,j)+get_far(j,i)-dis(p[i],p[j]);
 | | | ans = max(ans, ret); } }
```

8.5 $O(n^2)$ Ear Clipping 三角剖分

```
vector <array <int, 3>> tri;
void solve () {
 | list <int> l;
 | for (int k = 0; k < n; k++) l.push_back(k);
 | auto check = [&](auto u, auto v, auto w) {
 | | if (turn(u, v, w) <= 0) return false;
 | | for (auto i : l)
 | | | if (turn(u, v, p[i]) == 1 &&
 | | | | turn(v, w, p[i]) == 1 &&
 | | | | turn(w, u, p[i]) == 1) return false;
 | | return true; };
 | for (auto it = l.begin(); l.size() > 3; ) {
 | | auto u = (it == l.begin() ? prev(l.end()) : prev(it));
 | | auto v = (next(it) == l.end() ? l.begin() : next(it));
 | | if (!check(p[*u], p[*it], p[*v])) it = v;
 | | else {
 | | | tri.push_back({*u, *it, *v});
 | | | l.erase(it);
 | | | it = u; } }
 | tri.push_back({l.front(), *next(l.begin()), l.back()}); }
```

8.6 单插入动态凸包

```
struct hull { // upper hull, left to right
set <point> a; LL tot;
hull () {tot = 0;}
LL calc(auto it) {
 | auto u = it == a.begin() ? a.end() : prev(it);
 | auto v = next(it);
 | LL ret = 0;
 | if (u != a.end()) ret += det(*u, *it);
 | if (v != a.end()) ret += det(*it, *v);
 | if (u != a.end() && v != a.end()) ret -= det(*u, *v);
 | return ret; }
void insert (point p) {
 | if (!a.size()) { a.insert (p); return; }
 | auto it = a.lower_bound (p);
 | bool out;
 | if (it == a.begin()) out = (p < *it); // special case
 | else if (it == a.end()) out = true;
 | else out = turn(*prev(it), *it, p) > 0;
 | if (!out) return;
 | while (it != a.begin()) {
 | | auto o = prev(it);
 | | if (o == a.begin() || turn(*prev(o), *o, p) < 0) break;
 | | else erase(o); }
 | while (it != a.end()) {
 | | auto o = next(it);
 | | if (o == a.end() || turn(p, *it, *o) < 0) break;
 | | else erase(it), it = o; }
 | tot += calc(a.insert(p).first); }
void erase(auto it) { tot -= calc(it); a.erase(it); } };
```

8.7 圆

```cpp
struct circle { point c; LD r;};
bool in_circle(cp a, const circle &b) {
  return sgn(b.r - dis(b.c, a)) >= 0; }
circle make_circle(cp u, cp v) {
  point p = (u + v) / 2;
  return {p, dis(u, p)}; }
circle make_circle(cp a, cp b, cp c) { // 三点共线 inf / nan
  point bc = c - b, ca = a - c, ab = b - a;
  point o = (b + c - bc.rot90()*dot(ca,ab)/det(ca,ab)) / 2;
  return {o, dis(o, a)}; }
circle min_circle (vector <point> p) { // 最小覆盖圆
 circle ret({0, 0}, 0);
 shuffle (p.begin (), p.end (), rng);
 for (int i = 0; i < (int) p.size (); ++i)
  if (!in_circle(p[i], ret)) {
   ret = circle (p[i], 0);
   for (int j = 0; j < i; ++j) if (!in_circle(p[j], ret)) {
  ret = make_circle (p[i], p[j]);
  for (int k = 0; k < j; ++k) if (!in_circle(p[k], ret))
    ret = make_circle (p[i], p[j], p[k]);
  } } return ret; }
pair <point, point> line_circle_inter (cl a, circle c) {
  LD d = p2l (c.c, a);
  // 需要的话返回 vector <point>
  /* if (sgn (d - R) >= 0) return {}; */
  LD x = sqrt (sqr(c.r) - sqr(d)); // sqrt(max(0., ...))
  return {
    proj_to_line (c.c, a) + (a.s - a.t).unit() * x,
    proj_to_line (c.c, a) - (a.s - a.t).unit() * x }; }
LD circle_inter_area (circle a, circle b) { // 圆面积交
  LD d = dis (a.c, b.c);
  if (sgn (d - (a.r + b.r)) >= 0) return 0;
  if (sgn (d - abs(a.r - b.r)) <= 0) {
    LD r = min (a.r, b.r);
    return r * r * PI; }
  LD x = (d * d + a.r * a.r - b.r * b.r) / (2 * d),
      t1 = acos (min (1., max (-1., x / a.r))),
      t2 = acos (min (1., max (-1., (d - x) / b.r)));
  return sqr(a.r)*t1 + sqr(b.r)*t2 - d*a.r*sin(t1);}
vector <point> circle_inter (circle a, circle b) { // 圆交点
  if (a.c == b.c || sgn (dis (a.c, b.c) - a.r - b.r) > 0
    || sgn (dis (a.c, b.c) - abs (a.r - b.r)) < 0)
    return {};
  point r = (b.c - a.c).unit();
  LD d = dis (a.c, b.c);
  LD x = ((sqr (a.r) - sqr (b.r)) / d + d) / 2;
  LD h = sqrt (sqr (a.r) - sqr (x));
  if (sgn (h) == 0) return {a.c + r * x};
  return {a.c + r * x + r.rot90 () * h,
      a.c + r * x - r.rot90 () * h}; }
// 返回按照顺时针方向
vector <point> tangent (cp a, circle b) {
  return circle_inter (make_circle (a, b.c), b); }
vector <line> extangent (circle a, circle b) {
  vector <line> ret; // 未考虑两圆内切的一条外切线
  if (sgn(dis(a.c, b.c)-abs(a.r - b.r))<=0) return ret;
  if (sgn(a.r - b.r) == 0) {
    point dir = b.c - a.c;
    dir = (dir * a.r / dir.len()).rot90();
    ret.push_back({a.c + dir, b.c + dir});
    ret.push_back({a.c - dir, b.c - dir});
  } else {
    point p = (b.c * a.r - a.c * b.r) / (a.r - b.r);
    auto u = tangent(p, a), v = tangent(p, b);
    if (u.size() == 2 && v.size() == 2) {
      if (sgn(a.r-b.r) < 0)
        swap(u[0], u[1]), swap(v[0], v[1]);
      ret.push_back({u[0], v[0]});
      ret.push_back({u[1], v[1]}); } }
  return ret; }
vector <line> intangent(circle a, circle b) {
  vector <line> ret; // 未考虑两圆外切的一条内切线
  point p = (b.c * a.r + a.c * b.r) / (a.r + b.r);
  vector u = tangent(p, a), v = tangent(p, b);
  if (u.size() == 2 && v.size() == 2) {
    ret.push_back({u[0], v[0]});
    ret.push_back({u[1], v[1]}); } return ret; }
```

8.8 圆反演，阿波罗尼茨圆

所有关于两点 A, B 满足 $PA/PB = k$ 且不等于 1 的点 P 的轨迹是一个圆.
圆幂: 半径为 R 的圆 O, 任意一点 P 到 O 的幂为 $h = OP^2 - R^2$
圆幂定理: 过 P 的直线交圆在 A 和 B 两点, 则 $PA \cdot PB = |h|$
根轴: 到两圆等幂点的轨迹是一条垂直于连心线的直线

反演: 已知一圆 C, 圆心为 O, 半径为 r, 如果 P 与 P' 在过圆心 O 的直线上, 且 $OP \cdot OP' = r^2$, 则称 P 与 P' 关于 O 互为反演. 一般 C 取单位圆.
反演的性质:
不过反演中心的直线反形是过反演中心的圆, 反之亦然.
不过反演中心的圆, 它的反形是一个不过反演中心的圆.
两条直线在交点 A 的夹角, 等于它们的反形在相应点 A' 的夹角, 但方向相反.
两个相交圆周在交点 A 的夹角等于它们的反形在相应点 A' 的夹角, 但方向相反.
直线和圆周在交点 A 的夹角等于它们的反演图形在相应点 A' 的夹角, 但方向相反.
正交圆反形也正交. 相切圆反形也相切, 当切点为反演中心时, 反形为两条平行线. 两两相切的圆 r1, r2, r3, 求与他们都相切的圆 r4. 分母取负号, 答案再取绝对值, 为外切圆半径. 分母取正号为内切圆半径. $r_4^{\pm} = \frac{r_1 r_2 r_3}{r_1 r_2 + r_1 r_3 + r_2 r_3 \pm 2\sqrt{r_1 r_2 r_3 (r_1 + r_2 + r_3)}}$

```cpp
circle inv_c2c(point O, LD R, circle A) {
  LD OA = dis(A.c, O);
  LD RB = 0.5 * R * R * (1 / (OA - A.r) - 1 / (OA + A.r));
  LD OB = OA * RB / A.r;
  point B = O + (A.c - O) * (OB / OA);
  return {B, RB};
}  // 点 O 在圆 A 外, 求圆 A 的反演圆 B, R 是反演半径
circle inv_l2c(point O, LD R, line l) {
  point P = proj_to_line(O, l);
  LD d = dis(O, P);
  LD RB = R * R / (2 * d);
  point VB = (P - O) / d * RB;
  return {O + VB, RB};
}  // 不过 O 点的直线反演为过 O 点的圆, R 是反演半径
line inv_c2l (point O, LD R, circle A) {
  LD t = R * R / (2 * A.r);
  point p = O + (A.c - O).unit() * t;
  return {p, p + (O - p).rot90()};
}  // 过 O 点的圆反演为不过 O 点的直线, R 是反演半径
```

8.9 圆并

```cpp
int C; circle c[MAXN]; LD area[MAXN];
struct event { // 如果需要边界而非面积, 那么仔细考虑事件顺序
 point p; LD ang; int delta;
 bool operator <(const event &a){return ang < a.ang;}};
void addevent(cc a, cc b, vector<event> &e, int &cnt) {
 LD d2=dis2(a.c, b.c),dw=((a.r-b.r)*(a.r+b.r)/d2+1)/2,pw=
sqrt(max(0.,-(d2-sqr(a.r-b.r))*(d2-sqr(a.r+b.r))/sqr(2*d2));
 point d = b.c - a.c, p = d.rot(PI / 2),
  q0 = a.c + d * dw + p * pw,
  q1 = a.c + d * dw - p * pw;
 LD ang0 = atan2((q0 - a.c).y, (q0 - a.c).x),
  ang1 = atan2((q1 - a.c).y, (q1 - a.c).x);
 e.push_back({q1,ang1,1}); e.push_back({q0,ang0,-1});
 cnt += ang1 > ang0; }
bool issame(cc a, cc b) {
 return sgn((a.c-b.c).dis()) == 0 && sgn(a.r-b.r) == 0; }
bool overlap(cc a, cc b) {
 return sgn(a.r - b.r -(a.c - b.c).dis()) >= 0; }
bool intersect(cc a, cc b) {
 return sgn((a.c - b.c).dis() - a.r - b.r) < 0; }
void solve() {
 fill(area, area + C + 2, 0);
 for(int i = 0; i < C; ++i) { int cnt = 1;
  vector<event> e;
  for(int j=0; j<i; ++j) if(issame(c[i],c[j])) ++cnt;
  for(int j = 0; j < C; ++j)
   if(j != i && !issame(c[i], c[j]) && overlap(c[j], c[i]))
    ++cnt;
  for(int j = 0; j < C; ++j)
   if(j != i && !overlap(c[j], c[i]) && !overlap(c[i], c[j])
    && intersect(c[i], c[j]))
  addevent(c[i], c[j], e, cnt);
  if(e.empty()) area[cnt] += PI * c[i].r * c[i].r;
  else {
   sort(e.begin(), e.end());
   e.push_back(e.front());
   for(int j = 0; j + 1 <(int)e.size(); ++j) {
  cnt += e[j].delta;
  area[cnt] += det(e[j].p,e[j + 1].p) / 2;
  LD ang = e[j + 1].ang - e[j].ang;
  if(ang < 0) ang += PI * 2;
  area[cnt] += ang * c[i].r * c[i].r / 2 - sin(ang) *
    c[i].r * c[i].r / 2; } } } }
```

8.10 多边形与圆交

```cpp
LD angle (cp u, cp v) {
  return 2 * asin(dis(u.unit(), v.unit()) / 2); }
LD area(cp s, cp t, LD r) { // 2 * area
  LD theta = angle(s, t);
```

```cpp
  | LD dis = p2s ({0, 0}, {s, t});
  | if (sgn(dis - r) >= 0) return theta * r * r;
  | auto [u, v] = line_circle_inter({s, t}, {{0, 0}, r});
  | point lo = sgn(det(s, u)) >= 0 ? u : s;
  | point hi = sgn(det(v, t)) >= 0 ? v : t;
  | return det(lo, hi) + (theta - angle(lo, hi)) * r * r; }
LD solve(vector<point> &p, cc c) {
  | LD ret = 0;
  | for (int i = 0; i < (int) p.size (); ++i) {
  |   | auto u = p[i] - c.c;
  |   | auto v = p[(i + 1) % p.size()] - c.c;
  |   | int s = sgn(det(u, v));
  |   | if |    (s > 0) ret += area (u, v, c.r);
  |   | else if (s < 0) ret -= area (v, u, c.r);
  | } return abs (ret) / 2; } //ret在p逆时针时为正
```

8.11 球面基础, 经纬度球面距离

球面距离: 连接球面两点的大圆劣弧 (所有曲线中最短)

球面角: 球面两个大圆弧所在半平面形成的二面角

球面凸多边形: 把一个球面多边形任意一边向两方无限延长成大圆, 其余边都在此大圆的同旁.

球面角盈 E: 球面凸n边形的内角和与 $(n-2)\pi$ 的差

离北极夹角 θ, 距离 h 的球冠: $S = 2\pi Rh = 2\pi R^2(1-\cos\theta)$, $V = \frac{\pi h^2}{3}(3R-h)$

球面凸n边形面积: $S = ER^2$

```cpp
// lontitude 经度范围: ±π, latitude 纬度范围: ±π/2
LD sphereDis(LD lon1, LD lat1, LD lon2, LD lat2, LD R) {
  | return R * acos(cos(lat1) * cos(lat2) * cos(lon1 - lon2)
    ↪+ sin(lat1) * sin(lat2)); }
```

8.12 圆上整点

```cpp
vector <LL> solve(LL r) {
  | vector <LL> ret; // non-negative Y pos
  | ret.push_back(0);
  | LL l = 2 * r, s = sqrt(l);
  | for (LL d=1; d<=s; d++) if (l%d==0) {
  |   | LL lim=LL(sqrt(l/(2*d)));
  |   | for (LL a = 1; a <= lim; a++) {
  |   |   | LL b = sqrt(l/d-a*a);
  |   |   | if (a*a+b*b==l/d && __gcd(a,b)==1 && a!=b)
  |   |   |   | ret.push_back(d*a*b);
  |   | } if (d*d==l) break;
  |   | lim = sqrt(d/2);
  |   | for (LL a=1; a<=lim; a++) {
  |   |   | LL b = sqrt(d - a * a);
  |   |   | if (a*a+b*b==d && __gcd(a,b)==1 && a!=b)
  |   |   |   | ret.push_back(l/d*a*b);
  | } } return ret; }
```

8.13 三角形

```cpp
point incenter (cp a, cp b, cp c) {
  | double p = dis(a, b) + dis(b, c) + dis(c, a);
  | return ( a*dis(b, c) + b*dis(c, a) + c*dis(a, b) ) / p;}
point circumcenter (cp a, cp b, cp c) {
  | point p = b - a, q = c - a, s (dot(p,p)/2, dot(q,q)/2);
  | double d = det(p, q); return a + point(det(s, {p.y,
    ↪q.y}), det({p.x, q.x}, s)) / d; }
point orthocenter (cp a, cp b, cp c) {
  | return a + b + c - circumcenter (a, b, c) * 2.0; }
point fermat_point (cp a, cp b, cp c) {
  | if (a == b) return a; if (b == c) return b;
  | if (c == a) return c;
  | double ab = dis(a, b), bc = dis(b, c), ca = dis(c, a);
  | double cosa = dot(b - a, c - a) / ab / ca;
  | double cosb = dot(a - b, c - b) / ab / bc;
  | double cosc = dot(b - c, a - c) / ca / bc;
  | double sq3 = PI / 3.0; point mid;
  | if (sgn (cosa + 0.5) < 0) mid = a;
  | else if (sgn (cosb + 0.5) < 0) mid = b;
  | else if (sgn (cosc + 0.5) < 0) mid = c;
  | else if (sgn (det(b - a, c - a)) < 0)
  |   | mid = line_inter ({a, b + (c - b).rot (sq3)}, {b, c +
    ↪(a - c).rot (sq3)});
  | else mid = line_inter ({a, c + (b - c).rot (sq3)}, {c, b
    ↪+ (a - b).rot (sq3)});
  | return mid; } // minimize(|A-x|+|B-x|+|C-x|)
```

8.14 相关公式: 三角形心, 海伦公式, 欧拉定理, 内接球, 外接圆

Heron's Formula

$p = \frac{a+b+c}{2}, a \ge b \ge c,$

$S = \sqrt{p(p-a)(p-b)(p-c)},$

$S = \frac{1}{2}\sqrt{a^2c^2 - \left(\frac{a^2+c^2-b^2}{2}\right)^2}.$

四面体内接球球心

假设 s_i 是第 i 个顶点相对面的面积, 则有

$$\begin{cases} x = \dfrac{s_1x_1 + s_2x_2 + s_3x_3 + s_4x_4}{s_1+s_2+s_3+s_4} \\ y = \dfrac{s_1y_1 + s_2y_2 + s_3y_3 + s_4y_4}{s_1+s_2+s_3+s_4} \\ z = \dfrac{s_1z_1 + s_2z_2 + s_3z_3 + s_4z_4}{s_1+s_2+s_3+s_4} \end{cases}$$

体积可以使用 1/6 混合积求, 内接球半径为

$$r = \frac{3V}{s_1+s_2+s_3+s_4}$$

三角形内心

$$\vec{I} = \frac{a\vec{A} + b\vec{B} + c\vec{C}}{a+b+c}$$

三角公式

$\sin(a) + \sin(b) = 2\sin(\frac{a+b}{2})\cos(\frac{a-b}{2})$

$\sin(a) - \sin(b) = 2\cos(\frac{a+b}{2})\sin(\frac{a-b}{2})$

$\cos(a) + \cos(b) = 2\cos(\frac{a+b}{2})\cos(\frac{a-b}{2})$

$\cos(a) - \cos(b) = -2\sin(\frac{a+b}{2})\sin(\frac{a-b}{2})$

$\sin(a \pm b) = \sin a\cos b \pm \cos a \sin b$

$\cos(a \pm b) = \cos a \cos b \mp \sin a \sin b$

$\tan(a \pm b) = \frac{\tan(a) \pm \tan(b)}{1 \mp \tan(a)\tan(b)}$

$\tan(a) \pm \tan(b) = \frac{\sin(a \pm b)}{\cos(a)\cos(b)}$

$\sin(na) = n\cos^{n-1} a \sin a - \binom{n}{3}\cos^{n-3} a \sin^3 a + \binom{n}{5}\cos^{n-5} a \sin^5 a - \dots$

$\cos(na) = \cos^n a - \binom{n}{2}\cos^{n-2} a \sin^2 a + \binom{n}{4}\cos^{n-4} a \sin^4 a - \dots$

8.14.1 超球坐标系

$$\begin{aligned} x_1 &= r\cos(\phi_1) \\ x_2 &= r\sin(\phi_1)\cos(\phi_2) \\ &\cdots \\ x_{n-1} &= r\sin(\phi_1)\cdots\sin(\phi_{n-2})\cos(\phi_{n-1}) \\ x_n &= r\sin(\phi_1)\cdots\sin(\phi_{n-2})\sin(\phi_{n-1}) \\ \phi_{n-1} &\in [0, 2\pi] \\ \forall i = 1..n-1 \phi_i &\in [0, \pi] \end{aligned}$$

8.14.2 三维旋转公式

绕着 $(0,0,0)-(ux,uy,uz)$ 旋转 θ, (ux,uy,uz) 是单位向量. $\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = R\begin{bmatrix} x \\ y \\ z \end{bmatrix}$

$$R = \begin{pmatrix} \cos\theta + u_x^2(1-\cos\theta) & u_xu_y(1-\cos\theta) - u_z\sin\theta & u_xu_z(1-\cos\theta) + u_y\sin\theta \\ u_yu_x(1-\cos\theta) + u_z\sin\theta & \cos\theta + u_y^2(1-\cos\theta) & u_yu_z(1-\cos\theta) - u_x\sin\theta \\ u_zu_x(1-\cos\theta) - u_y\sin\theta & u_zu_y(1-\cos\theta) + u_x\sin\theta & \cos\theta + u_z^2(1-\cos\theta) \end{pmatrix}.$$

8.14.3 立体角公式

$$\phi: \text{二面角}$$

$$\Omega = (\phi_{ab} + \phi_{bc} + \phi_{ac})\ \mathrm{rad} - \pi\ \mathrm{sr}$$

$$\tan\left(\frac{1}{2}\Omega/\mathrm{rad}\right) = \frac{\left|\vec{a}\,\vec{b}\,\vec{c}\right|}{abc + \left(\vec{a}\cdot\vec{b}\right)c + (\vec{a}\cdot\vec{c})\,b + \left(\vec{b}\cdot\vec{c}\right)a}$$

$$\theta_s = \frac{\theta_a + \theta_b + \theta_c}{2}$$

8.14.4 常用体积公式

- 棱锥 Pyramid $V = \frac{1}{3}Sh$.
- 球 Sphere $V = \frac{4}{3}\pi R^3$.
- 棱台 Frustum $V = \frac{1}{3}h(S_1 + \sqrt{S_1S_2} + S_2)$.
- 椭球 Ellipsoid $V = \frac{4}{3}\pi abc$.

三角形外心

$$\vec{O} = \frac{\vec{A} + \vec{B} - \frac{\overrightarrow{BC}\cdot\overrightarrow{CA}}{\overrightarrow{AB}\times\overrightarrow{BC}}\overrightarrow{AB}^T}{2}$$

三角形垂心

$$\vec{H} = 3\vec{G} - 2\vec{O}$$

三角形偏心

$$\frac{-a\vec{A} + b\vec{B} + c\vec{C}}{-a+b+c}$$

内角的平分线和对边的两个外角平分线交点, 外切圆圆心. 剩余两点的同理.

三角形内接外接圆半径

$$r = \frac{2S}{a+b+c},\ R = \frac{abc}{4S}$$

Pick's Theorem 格点多边形面积

$S = I + \frac{B}{2} - 1$. I 内部点, B 边界点。

Euler's Formula 多面体与平面图的点、边、面

Convex polyhedron: $V - E + F = 2$.

Planar graph: $|F| = |E| - |V| + n + 1$,

n : #(connected components).

8.14.5 扇形与圆弧重心

扇形重心与圆心距离为 $\frac{4r\sin(\theta/2)}{3\theta}$,

圆弧重心与圆心距离为 $\frac{4r\sin^3(\theta/2)}{3(\theta-\sin(\theta))}$.

8.14.6 高维球体积

$$V_2 = \pi R^2,\ S_2 = 2\pi R$$
$$V_3 = \frac{4}{3}\pi R^3,\ S_3 = 4\pi R^2$$
$$V_4 = \frac{1}{2}\pi^2 R^4,\ S_4 = 2\pi^2 R^3$$
$$\text{Generally, } V_n = \frac{2\pi}{n} V_{n-2},\ S_{n-1} = \frac{2\pi}{n-2} S_{n-3}$$
$$\text{Where, } S_0 = 2,\ V_1 = 2,\ S_1 = 2\pi,\ V_2 = \pi$$

8.15 三维几何基础操作

```cpp
/* 右手系逆时针绕轴旋转，(x,y,z)A = (x_new, y_new, z_new)
new[i] += old[j] * A[j][i] */
void calc(p3 n, LD cosw) {
 | LD sinw = sqrt(1 - cosw * cosw);
 | n = n.unit();
 | for (int i = 0; i < 3; i++) {
 | | int j = (i + 1) % 3, k = (j + 1) % 3;
 | | LD x = n[i], y = n[j], z = n[k];
 | | A[i][i] = (y * y + z * z) * cosw + x * x;
 | | A[i][j] = x * y * (1 - cosw) + z * sinw;
 | | A[i][k] = x * z * (1 - cosw) - y * sinw; } }
p3 cross (p3 a, p3 b) { return p3(
 a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y - a.y*b.x);}
LD mix(p3 a, p3 b, p3 c) { return dot(cross(a, b), c); }
struct l3 { p3 s, t; };
struct plane { // nor 为单位法向量, 离原点距离 m
 | p3 nor; LD m;
 | plane(p3 r, p3 a) : nor(r.unit()), m(dot(nor, a)) {} };
// 除法注意除零; 点到直线投影: 与二维一致
p3 project_to_plane(p3 a, plane b) { // 点到平面投影
 | return a + b.nor * (b.m - dot(a, b.nor)); }
pair<p3, p3> l3_closest(l3 x, l3 y) { // 两直线最近点
 | LD a = dot(x.t - x.s, x.t - x.s);
 | LD b = dot(x.t - x.s, y.t - y.s);
 | LD e = dot(y.t - y.s, y.t - y.s);
 | LD d = a*e - b*b; p3 r = x.s - y.s;
 | LD c = dot(x.t - x.s, r), f = dot(y.t - y.s, r);
 | LD s = (b*f - c*e) / d, t = (a*f - c*b) / d;
 | return {x.s + (x.t - x.s)*s, y.s + (y.t - y.s)*t}; }
p3 intersect(plane a, l3 b) { // 直线与平面交点
 | LD t = dot(a.nor, a.nor*a.m - b.s)/dot(a.nor, b.t - b.s);
 | return b.s + (b.t - b.s) * t; }
// 平面与平面求交线
l3 intersect(plane a, plane b) {
 | p3 d = cross(a.nor, b.nor), d2 = cross(b.nor, d);
 | LD t = dot(d2, a.nor);
 | p3 s = d2 * (a.m - dot(b.nor*b.m, a.nor))/t + b.nor*b.m;
 | return {s, s + d}; }
// 三个平面求交点
p3 intersect(plane a, plane b, plane c) {
return intersect(a, intersect(b, c));
p3 c1 (a.nor.x, b.nor.x, c.nor.x);
p3 c2 (a.nor.y, b.nor.y, c.nor.y);
p3 c3 (a.nor.z, b.nor.z, c.nor.z);
p3 c4 (a.m, b.m, c.m);
return 1 / mix(c1, c2, c3) * p3(mix(c4, c2, c3), mix(c1, c4, c3), mix(c1, c2, c4)); }
```

8.16 三维凸包

```cpp
vector <p3> p;
int mark[N][N], stp;
typedef array <int, 3> Face;
vector <Face> face;
LD volume (int a, int b, int c, int d) {
 | return mix (p[b] - p[a], p[c] - p[a], p[d] - p[a]); }
void ins(int a, int b, int c) {face.push_back({a, b, c});}
void add(int v) {
 | vector <Face> tmp; int a, b, c; stp++;
 | for (auto f : face) {
 | | if (sgn(volume(v, f[0], f[1], f[2])) < 0) {
 | | | for (auto i : f) for (auto j : f)
 | | | | mark[i][j] = stp; }
 | | else {
 | | | tmp.push_back(f);}
 | } face = tmp;
 | for (int i = 0; i < (int) tmp.size(); i++) {
 | | a = tmp[i][0], b = tmp[i][1], c = tmp[i][2];
 | | if (mark[a][b] == stp) ins(b, a, v);
 | | if (mark[b][c] == stp) ins(c, b, v);
 | | if (mark[c][a] == stp) ins(a, c, v); } }
bool Find(int n) {
 | for (int i = 2; i < n; i++) {
 | | p3 ndir = cross (p[0] - p[i], p[1] - p[i]);
 | | if (ndir == p3(0,0,0)) continue;
 | | swap(p[i], p[2]);
 | | for (int j = i + 1; j < n; j++) {
 | | | if (sgn(volume(0, 1, 2, j)) != 0) {
 | | | | swap(p[j], p[3]);
 | | | | ins(0, 1, 2);
 | | | | ins(0, 2, 1);
 | | | | return 1;
 | } } } return 0; }
mt19937 rng;
bool solve() {
 | face.clear();
 | int n = (int) p.size();
 | shuffle(p.begin(), p.end(), rng);
 | if (!Find(n)) return 0;
 | for (int i = 3; i < n; i++) add(i);
 | return 1; }
```

```cpp
// 卷包裹法: 精度越高, 扰动次数越多
ld rd() {return ((ld)rand() / RAND_MAX - 0.5) * EPS;}
ld vol(cp o, cp a, cp b, cp c) {return mix(a-o, b-o, c-o); }
ld det2d(cp a, cp b) {return a.x * b.y - a.y * b.x;}
namespace Convex {
int n; bool used[MAXN]; bool e[MAXN][MAXN]; // 清空 used e?
void wrap(vector<point>& v, int a, int b) {
 | if(!e[a][b]) {
 | | int c = -1;
 | | for(int i=0;i<n;i++) if(i != a && i != b)
 | | | if(c == -1 || vol(v[c], v[a], v[b], v[i])>0) c = i;
 | | if(c == -1) return;
 | | // 求体积, 求面积, 新建面, ...
 | | used[a] = 1, used[b] = 1, used[c] = 1;
 | | e[a][b] = 1, e[b][c] = 1, e[c][a] = 1;
 | | wrap(v, c, b), wrap(v, a, c);
} }
void make_convex(vector<point>& v) {
 | n = v.size();
 | for(int i=1;i<n;i++) if(v[i].x < v[0].x)
 | | swap(v[0], v[i]);
 | for(int i=2;i<n;i++) if(det2d(v[i]-v[0], v[1]-v[0]) > 0)
 | | swap(v[1], v[i]);
 | wrap(v, 0, 1);
} } // namespace Convex
```

8.17 最小覆盖球

```cpp
vector<p3> vec;
Circle calc() {
 if(vec.empty()) { return Circle(p3(0, 0, 0), 0);
 }else if(1 == (int)vec.size()) {return Circle(vec[0], 0);
 }else if(2 == (int)vec.size()) {
  return Circle(0.5 * (vec[0] + vec[1]), 0.5 * (vec[0] - vec[1]).len());
 }else if(3 == (int)vec.size()) {
  double r = (vec[0] - vec[1]).len() * (vec[1] - vec[2]).len() * (vec[2] - vec[0]).len() / 2 / fabs(cross(vec[0] - vec[2], vec[1] - vec[2]).len());
  Plane ppp1 = Plane(vec[1] - vec[0], 0.5 * (vec[1] + vec[0]));
  return Circle(intersect(Plane(vec[1] - vec[0], 0.5 * (vec[1] + vec[0])), Plane(vec[2] - vec[1], 0.5 * (vec[2] + vec[1])), Plane(cross(vec[1] - vec[0], vec[2] - vec[0]), vec[0])), r);
 }else {
  p3 o(intersect(Plane(vec[1] - vec[0], 0.5 * (vec[1] + vec[0])), Plane(vec[2] - vec[0], 0.5 * (vec[2] + vec[0])), Plane(vec[3] - vec[0], 0.5 * (vec[3] + vec[0]))));
  return Circle(o, (o - vec[0]).len()); } }
Circle miniBall(int n) {
 Circle res(calc());
 for(int i(0); i < n; i++) {
  if(!in_circle(a[i], res)) { vec.push_back(a[i]);
   res = miniBall(i); vec.pop_back();
   if(i) { p3 tmp(a[i]);
  | memmove(a + 1, a, sizeof(p3) * i);
  | a[0] = tmp; } } }
 return res; }
int main() {
 int n; scanf("%d", &n);
 for(int i(0); i < n; i++) a[i].scan();
```

```cpp
  sort(a, a + n); n = unique(a, a + n) - a;
  vec.clear(); random_shuffle(a, a + n);
  printf("%.10f\n", miniBall(n).r); }
```

8.18 黄金三分

```cpp
constexpr LD R = (sqrt(5) - 1) / 2;
auto split = [](LD l, LD r) { return l + (r - l) * R; };
LD solve(LD a, LD c, auto f) {
  LD b = split(a, c), bv = f(b);
  for (int _ = T; _; _--) {
    LD x = split(a, b), xv = f(x);
    if (xv < bv) c = b, b = x, bv = xv; // 最小化，注意符号
    else a = c, c = x;
  } return bv; }
```

8.19 自适应 Simpson

```cpp
// Adaptive Simpson's method : LD simpson::solve (LD (*f) (LD), LD l, LD r, LD eps) : integrates f over (l, r) with error eps.
struct simpson {
LD area (LD (*f) (LD), LD l, LD r) {
  LD m = l + (r - l) / 2;
  return (f (l) + 4 * f (m) + f (r)) * (r - l) / 6;
}
LD solve (LD (*f) (LD), LD l, LD r, LD eps, LD a) {
  LD m = l + (r - l) / 2;
  LD left = area (f, l, m), right = area (f, m, r);
  if (abs (left + right - a) <= 15 * eps) // TLE: || eps < EPS ** 2
    return left + right + (left + right - a) / 15.0;
  return solve (f, l, m, eps / 2, left) + solve (f, m, r, eps / 2, right);
}
LD solve (LD (*f) (LD), LD l, LD r, LD eps) {
  return solve (f, l, r, eps, area (f, l, r));
}};
```

9. 图论

9.1 Hopcroft-Karp $O(\sqrt{V}E)$

```cpp
vector <int> E[N];
vector <int> ml, mr, a, p;
void match (int nl, int nr) { // 1-based
  ml.assign(nl + 1, 0);
  mr.assign(nr + 1, 0); // nr
  while (true) {
    bool ok = 0;
    a.assign(nl + 1, 0);
    p.assign(nl + 1, 0); // nl
    static queue <int> q;
    for (int i = 1; i <= nl; i++)
      if (!ml[i]) a[i] = p[i] = i, q.push(i);
    while (!q.empty()) {
      int x = q.front(); q.pop();
      if (ml[a[x]]) continue;
      for (auto y : E[x]) {
        if (!mr[y]) {
          for (ok = 1; y; x = p[x])
            mr[y] = x, swap(ml[x], y);
          break;
        } else if (!p[mr[y]])
          q.push(y = mr[y]), p[y] = x, a[y] = a[x];
      } } // while (!q.empty())
    if (!ok) break; } }
array<vector<int>, 2> min_edge_cover(int nl, int nr) {
 match(nl, nr); vector <int> l, r;
 for (int i = 1; i <= nl; i++) if (!a[i]) l.push_back(i);
 for (int i = 1; i <= nr; i++) if (a[mr[i]]) r.push_back(i);
 return {l, r}; }
```

9.2 Hungarian $O(VE/w)$

```cpp
using B = bitset<N>; B edge[N];
bool dfs(int x, B& unvis, vector<int>& match) {
  for(B z = edge[x];;) {
    z &= unvis;
    int y = z._Find_first();
    if(y == N) return 0;
    unvis.reset(y);
    if(!match[y] || dfs(match[y], unvis, match))
      return match[y] = x, 1; } }
vector<int> match(int nl, int nr) {
  B unvis; unvis.set();
  vector<int> match(nr + 1), ret(nl + 1);
  for(int i = 1;i <= nl;++i)
    if(dfs(i, unvis, match)) unvis.set();
  for(int i = 1;i <= nr;++i) ret[match[i]] = i;
  return ret[0] = 0, ret; }
```

9.3 Shuffle 一般图最大匹配 $O(VE)$

```cpp
// 该算法的正确性无法保证
int n, m, mat[N], vis[N]; vector<int> E[N];
bool dfs(int tim, int x) {
  shuffle(E[x].begin(), E[x].end(), rng);
  vis[x] = tim;
  for (auto y : E[x]) {
    int z = mat[y]; if (vis[z] == tim) continue;
    mat[x] = y, mat[y] = x, mat[z] = 0;
    if (!z || dfs(tim, z)) return true;
    mat[x] = 0, mat[y] = z, mat[z] = y; }
  return false; }
int main() { // 暗含二分图性质跑一次即可
  for (int _ = 0; _ < 10; _++) {
    fill(vis + 1, vis + n + 1, 0);
    for (int i = 1; i <= n; ++i) if (!mat[i]) dfs(i, i);}}
```

9.4 极大团计数

```cpp
// 0下标，需删除自环（即确保 E_ii = 0，补图要特别注意）
// 极大团计数，最坏情况O(3^(n/3))
ll ans; ull E[64]; #define bit(i) (1ULL << (i))
void dfs(ull P, ull X, ull R) { // 不要方案时可去掉R
  if (!P && !X) { ++ans; sol.pb(R); return; }
  ull Q = P & ~E[__builtin_ctzll(P | X)];
  for (int i; i = __builtin_ctzll(Q), Q; Q &= ~bit(i)) {
    dfs(P & E[i], X & E[i], R | bit(i));
    P &= ~bit(i), X |= bit(i); }}
ans = 0; dfs(n == 64 ? ~0ULL : bit(n) - 1, 0, 0);
```

9.5 KM 最大权匹配 $O(V^3)$

```cpp
ll e[N][M]; //O(l^2*r)
vector<int> KM(int n, int m) {
  vector<ll> l(n + 1), r(m + 1);
  vector<int> p(m + 1), ans(n + 1);
  for (int i = 1; i <= n; ++i) {
    vector<ll> d(m + 1, 1e18);
    vector<int> pre(m + 1), done(m + 1);
    int x, v, u = 0;
    for (p[0] = i; x = p[u]; u = v) {
      done[u] = 1;
      ll min = 1e18;
      for (int j = 1; j <= m; ++j) if (!done[j]) {
        auto w = e[x][j] - l[x] - r[j];
        if (w < d[j]) d[j] = w, pre[j] = u;
        if (d[j] < min) min = d[j], v = j;
      }
      for (int j = 0; j <= m; ++j) {
        if (done[j]) l[p[j]] += min, r[j] -= min;
        else d[j] -= min;
      }
    }
    for (int v; u; u = v) v = pre[u], p[u] = p[v];
  }
  for (int j = 1; j <= m; ++j) ans[p[j]] = j;
  return ans; }
```

9.6 欧拉回路

```cpp
/* comment : directed */
int e, cur[N]/*, deg[N]*/;
vector<int>E[N];
int id[M]; bool vis[M];
stack<int>stk;
void dfs(int u) {
  for (cur[u]; cur[u] < E[u].size(); cur[u]++) {
    int i = cur[u];
    if (vis[abs(E[u][i])]) continue;
    int v = id[abs(E[u][i])] ^ u;
    vis[abs(E[u][i])] = 1; dfs(v);
    stk.push(E[u][i]); }
}// dfs for all when disconnect
void add(int u, int v) {
```

```cpp
  id[++e] = u ^ v; // s = u
  E[u].push_back(e); E[v].push_back(-e);
/*  E[u].push_back(e); deg[v]++; */
} bool valid() {
  for (int i = 1; i <= n; i++)
    if (E[i].size() & 1) return 0;
/*    if (E[i].size() != deg[i]) return 0;*/
  return 1;}
```

9.7 Blossom 一般图最大匹配 $O(|V||E|)$

```cpp
int mch[MAXN], pre[MAXN], fa[MAXN], col[MAXN];
int hscnt = 0, hs[MAXN]; queue<int> q;
int find(int x) {return fa[x]==x ? x : fa[x]=find(fa[x]);}
int lca(int x, int y) {
  for(++hscnt; x; y ? swap(x, y) : (void)0) {
    x = find(x);
    if(hs[x] == hscnt) return x;
    hs[x] = hscnt, x = pre[mch[x]]; } }
void blossom(int x, int y, int rt) {
  for(int z; find(x) != rt; y = z, x = pre[y]) {
    pre[x] = y, z = mch[x];
    if(col[z] == 1) q.push(z), col[z] = 0;
    if(find(x) == x) fa[x] = rt;
    if(find(z) == z) fa[z] = rt; } }
bool bfs(int st) {
  while(!q.empty()) q.pop();
  q.push(st), col[st] = 0;
  while(!q.empty()) {
    int x = q.front(); q.pop();
    for(auto v : e[x]) {
      if(col[v] == -1) {
        pre[v] = x, col[v] = 1;
        if(!mch[v]) {
          for(int y; v; swap(mch[y], v))
            mch[v] = y = pre[v];
          return true; }
        q.push(mch[v]); col[mch[v]] = 0;
      }else if(col[v] == 0 && find(x) != find(v)) {
        int f = lca(x, v); blossom(x, v, f);
        blossom(v, x, f);} } }
  return false;}
int solve() {
  int ans = 0;
  for(int i=1;i<=n;i++) {
    for(int j=1;j<=n;j++) col[j] = -1, fa[j] = j;
    if(!mch[i] && bfs(i)) ans++;}
  return ans;}
```

9.8 2-SAT, 强连通分量 / Bitset Kosaraju

```cpp
int stp, sccs, top; // N 开 **两倍**
int dfn[N], low[N], scc[N], stk[N], ans[N];
void add(int x, int a, int y, int b) { // 注意连边对称
  E[x << 1 | a].push_back(y << 1 | b); }
void tarjan(int x) {
  dfn[x] = low[x] = ++stp;
  stk[top++] = x;
  for (auto y : E[x]) {
    if (!dfn[y])
      tarjan(y), low[x] = min(low[x], low[y]);
    else if (!scc[y])
      low[x] = min(low[x], dfn[y]); }
  if (low[x] == dfn[x]) {
    sccs++;
    do scc[stk[--top]] = sccs;
    while (stk[top] != x); } }
bool solve() {
  int cnt = n + n; stp = top = sccs = 0;
  fill(dfn, dfn + cnt + 1, 0); fill(scc, scc + cnt + 1, 0);
  for (int i = 0; i < cnt; ++i) if (!dfn[i]) tarjan(i);
  for (int i = 0; i < n; ++i) {
    if (scc[i << 1] == scc[i << 1 | 1]) return false;
    ans[i] = (scc[i << 1 | 1] < scc[i << 1]); }
  return true; }
```

```cpp
bitset<N> e[N], re[N], vis; vector<int> sta;
void dfs0(int x, bitset<N> e[]) {
  vis.reset(x);
  while (true) {
    int go = (e[x] & vis)._Find_first();
    if(go == N) break;
    dfs0(go, e); }
  sta.push_back(x); }
vector<vector<int>> solve() { // re 需要连好反向边
  vis.set();
  for(int i = 1;i <= n;++i) if(vis.test(i)) dfs0(i, e);
  vis.set();
  auto s = sta;
  vector<vector<int>> ret;
  for(int i = n - 1;i >= 0;--i) if(vis.test(s[i])) {
    sta.clear(), dfs0(s[i], re), ret.push_back(sta); }
  return ret; }
```

9.9 Tarjan 点双，边双

```cpp
// 强联通分量
void tar(int x) {
  dfn[x] = low[x] = ++tot;
  S[++top] = x, vis[x] = 1;
  for(auto v : e[x]) {
    if(!dfn[v]) tar(v), low[x] = min(low[x], low[v]);
    else if(vis[v]) low[x] = min(low[x], dfn[v]);}
  if(dfn[x] == low[x]) {
    ++bcnt;
    while(S[top] != x) {
      int v = S[top--];
      vis[v] = 0;
      bel[v] = bcnt;}
    top--, vis[x] = 0, bel[x] = bcnt;}}
/*边双*/
// 注意！建立边双树或者圆方树后【边表大小】是否开够
// 求边双：无视掉 bri 后搜出每个连通块，记得多测
int dfn[N], low[N], dfscnt; // clear dfn low bri dfscnt
bool bri[M << 1]; // 注意此处是边数
void tarjan(int x, int last) { // last 是边
  dfn[x] = low[x] = ++dfscnt;
  for (int i = head[x]; i; i = e[i].next) {
    int v = e[i].to;
    if (!dfn[v]) {
      tarjan(v, i);
      low[x] = min(low[x], low[v]);
      if (low[v] > dfn[x]) bri[i] = bri[i ^ 1] = 1;
    } else if (dfn[v] < dfn[x] && ((i ^ 1) != last))
      low[x] = min(low[x], dfn[v]); } }
/** 建立圆方树+求割点 **/
int is_cut[N], dfn[N], low[N], dfscnt, pcnt;
int stk[N], dep; // clear dfn low is_cut dfscnt, let pcnt=n
void tarjan(int x, int fa) {
  int child = 0;
  dfn[x] = low[x] = ++dfscnt; stk[++dep] = x;
  for (auto v : e[x]) {
    if (!dfn[v]) {
      ++child; tarjan(v, x);
      low[x] = min(low[x], low[v]);
      if (low[v] >= dfn[x]) {
        is_cut[x] = true;
        ++pcnt; // square node index
        int j = 0, sz = 1;
        do {
          j = stk[dep--];
          addedge(pcnt, j);
          ++sz;
        } while (j != v);
        addedge(pcnt, x); }
    } else if low[x] = min(low[x], dfn[v]); }
  if (!fa && child == 1) is_cut[x] = false; }
```

9.10 Dominator Tree 支配树

```cpp
vector<int> G[MAXN], R[MAXN], son[MAXN];
int ufs[MAXN]; // R 是反图, son 存的是 sdom 树上的儿子
int idom[MAXN], sdom[MAXN], anc[MAXN];
// anc: sdom的dfn最小的祖先
int pr[MAXN], dfn[MAXN], id[MAXN], stamp;
int findufs(int x) { if (ufs[x] == x) return x;
  int t = ufs[x]; ufs[x] = findufs(ufs[x]);
  if (dfn[sdom[anc[x]]] > dfn[sdom[anc[t]]])
    anc[x] = anc[t];
  return ufs[x]; }
void dfs(int x) {
  dfn[x] = ++stamp; id[stamp] = x; sdom[x] = x;
  for (int y : G[x]) if (!dfn[y]) { pr[y] = x; dfs(y); } }
void get_dominator(int n) {
  for (int i = 1; i <= n; i++) ufs[i] = anc[i] = i;
  dfs(1);
  for (int i = n; i > 1; i--) { int x = id[i];
    for (int y : R[x]) if (dfn[y]) { findufs(y);
```

```
|  |  | if (dfn[sdom[x]] > dfn[sdom[anc[y]]])
|  |  |  | sdom[x] = sdom[anc[y]]; }
|  | son[sdom[x]].push_back(x); ufs[x] = pr[x];
|  | for (int u : son[pr[x]]) { findufs(u);
|  |  | idom[u] = (sdom[u] == sdom[anc[u]] ?
|  |  |  | pr[x] : anc[u]); }
|  | son[pr[x]].clear(); }
| for (int i = 2; i <= n; i++) { int x = id[i];
|  | if (idom[x] != sdom[x]) idom[x] = idom[idom[x]];
|  | son[idom[x]].push_back(x); } }
```

9.11 环计数

```
// Time complexity: O(m√m)
for(int i=1;i<=m;i++) {
 | if(deg[x[i]] > deg[y[i]] || (deg[x[i]] == deg[y[i]] && x[i] > y[i])) swap(x[i], y[i]);
 | e[x[i]].push_back(y[i]);}
// 四元环计数
for(int i=1;i<=n;i++) {
 | for(auto x : e[i]) for(auto y : e[x])
 |  | if(deg[i] < deg[y] || (deg[i] == deg[y] && i <= y))
 |  |  | ans += cnt[y]++; // 注意判等
 | for(auto x : e[i]) for(auto y : e[x]) cnt[y] = 0;}
```

9.12 Dinic 最大流

单位流量网络 在 0-1 流量图上 Dinic 有更好的性质.

- 复杂度为 $O(\min\{V^{2/3}, E^{1/2}\}E)$.
- $\mathrm{dist}(s,t) = d$, 残量网络上至多还存在 E/d 的流.
- 每个点只有一个入/出度时复杂度 $O(V^{1/2}E)$, 例如 Hopcroft-Karp.

```
bool bfs() { // 约定s=n+1, t=s+1
 | for(int i=1;i<=t;i++) dep[i] = 0, cur[i] = head[i];
 | queue<int> q; q.push(s); dep[s] = 1;
 | while(!q.empty()) {
 |  | int x = q.front(); q.pop();
 |  | for(int i=head[x];i;i=e[i].next) {
 |  |  | int v = e[i].to;
 |  |  | if(dep[v] || !e[i].w) continue;
 |  |  | dep[v] = dep[x] + 1; q.push(v);}}
 | return dep[t];}
int dfs(int x, int in) {
 | if(x == t) return in;
 | int out = 0;
 | for(int &i=cur[x];i;i=e[i].next) {
 |  | int v = e[i].to;
 |  | if(dep[v] != dep[x] + 1 || !e[i].w) continue;
 |  | int res = dfs(v, min(in, e[i].w));
 |  | in -= res, out += res;
 |  | e[i].w -= res, e[i^1].w += res;
 |  | if(!in) break;}
 | if(!out) dep[x] = 0;
 | return out;}
```

9.13 原始对偶费用流

```
void dij() { // reversed: from t to s
 | for (int i = 1; i <= t; i++) dis[i] = inf;
 | for (int i = 1; i <= t; i++) vis[i] = 0;
 | dis[t] = 0;
 | priority_queue<Node> q; q.push({t, 0});
 | while(!q.empty()){
 |  | auto [x, t] = q.top(); q.pop();
 |  | if(vis[x]) continue;
 |  | vis[x] = 1;
 |  | for(int i=head[x];i;i=e[i].next){
 |  |  | int v = e[i].to;
 |  |  | if(vis[v] || !e[i^1].w) continue;
 |  |  | ll cost = -e[i].cost + h[x] - h[v];
 |  |  | if(dis[v] > dis[x] + cost) {
 |  |  |  | dis[v] = dis[x] + cost;
 |  |  |  | q.push({v, dis[v]});
} } } }
bool bfs(){
 | for(int i=1;i<=t;i++) dep[i] = 0, cur[i] = head[i];
 | queue<int> q; q.push(s); dep[s] = 1;
 | while(!q.empty()){
 |  | int x = q.front(); q.pop();
 |  | for(int i=head[x];i;i=e[i].next){
 |  |  | int v = e[i].to;
 |  |  | ll cost = e[i].cost + h[v] - h[x];
 |  |  | if(dep[v] || !e[i].w || dis[v] != dis[x] - cost) continue;
 |  |  | dep[v] = dep[x] + 1;
 |  |  | q.push(v);
 | } } return dep[t]; }
ll dfs(int x, ll in){
 | if(x == t) return in;
 | ll out = 0;
 | for(int &i=cur[x];i;i=e[i].next){
 |  | int v = e[i].to;
 |  | ll cost = e[i].cost + h[v] - h[x];
 |  | if(dep[v] != dep[x] + 1 ||
 |  |    dis[v] != dis[x] - cost || !e[i].w) continue;
 |  | ll res = dfs(v, min(in, e[i].w));
 |  | in -= res, out += res;
 |  | e[i].w -= res, e[i^1].w += res;
 |  | if(!in) break; }
 | if(!out) dep[x] = 0;
 | return out; }
pair<ll, ll> solve(){ // first_sssp: DAG最短路或SPFA
 | ll flow = 0, cost = 0;
 | for (first_sssp(); dis[s] != inf; dij()) {
 |  | while (bfs()) {
 |  |  | ll tmp = dfs(s, inf);
 |  |  | flow += tmp, cost += tmp * (dis[s] + h[s]); }
 |  | for(int i=1;i<=t;i++) h[i] += min(dis[i], dis[s]);
 | } return {flow, cost}; }
```

9.14 网络流总结

9.15 网络流总结

最小割集, 最小割必须边以及可行边

最小割集 从 S 出发, 在残余网络中BFS所有权值非 0 的边 (包括反向边), 得到点集 $\{S\}$, 另一集为 $\{V\} - \{S\}$.

最小割集必须点 残余网络中S直接连向的点必在S的割集中, 直接连向T的点必在T的割集中; 若这些点的并集为全集, 则最小割方案唯一.

最小割可行边 在残余网络中求强联通分量, 将强联通分量缩点后, 剩余的边即为最小割可行边, 同时这些边也必然满流.

最小割必须边 在残余网络中求强联通分量, 若S出发可到u, T出发可到v, 等价于 $\mathrm{scc}_S = \mathrm{scc}_u$ 且 $\mathrm{scc}_T = \mathrm{scc}_v$, 则该边为必须边.

常见问题

最大权闭合子图 点权, 限制条件形如: 选择A则必须选择B, 选择B则必须选择C, D. 建图方式: B向A连边, CD向B连边. 求解: S向正权点连边, 负权点向T连边, 其余边容量 ∞, 求最小割, 答案为S所在最小割集.

二次布尔型 (文理分科) n 个点分为两类, i 号点有 l_i 或 r_i 的代价, i, j 同属一侧分别获得 l_{ij} 或 r_{ij} 的代价, 问最小代价. $L \to i : (l_i + 1/2\sum_j l_{ij})$, $i \to R : (r_i + 1/2\sum_j r_{ij})$, $i \leftrightarrow j : 1/2(l_{ij} + r_{ij})$. 实现时边权乘 2 为整数, 求解后答案除 2 为整数. 图拆点可以看作二分图.

如果是二元限制是分类不同时有一个代价 d_{ij}, 建图可以简化为 $L \to i : l_i$, $i \to R : r_i$, $i \leftrightarrow j : d_{ij}$. 经典例子: xor 最小值, 按位拆开建图.

混合图欧拉回路 把无向边随便定向, 计算每个点的入度和出度, 如果有某个点出入度之差 $\deg_i = \mathrm{in}_i - \mathrm{out}_i$ 为奇数, 肯定不存在欧拉回路. 对于 $\deg_i > 0$ 的点, 连接边 $(i, T, \deg_i/2)$; 对于 $\deg_i < 0$ 的点, 连接边 $(S, i, -\deg_i/2)$. 最后检查是否满流即可.

二物流 水源 S_1, 水汇 T_1, 油源 S_2, 油汇 T_2, 每根管道流量共用. 求流量和最大. 建超级源 SS_1 汇 TT_1, 连边 $SS_1 \to S_1$,$SS_1 \to S_2$,$T_1 \to TT_1$,$T_2 \to TT_1$, 设最大流为 x_1. 建超级源 SS_2 汇 TT_2, 连边 $SS_2 \to S_1$, $SS_2 \to T_2$,$T_1 \to TT_2$, $S_2 \to TT_2$, 设最大流为 x_2. 则最大流中水流量 $\frac{x_1+x_2}{2}$, 油流量 $\frac{x_1-x_2}{2}$.

无源汇有上下界可行流 每条边 (u, v) 有一个上界容量 $C_{u,v}$ 和下界容量 $B_{u,v}$, 我们让下界变为 0, 上界变为 $C_{u,v} - B_{u,v}$, 但这样做流量不守恒. 建立超级源点 SS 和超级汇点 TT, 用 du_i 来记录每个节点的流量情况, $du_i = \sum B_{j,i} - \sum B_{i,j}$, 添加一些附加弧. 当 $du_i > 0$ 时, 连边 (SS, i, du_i); 当 $du_i < 0$ 时, 连边 $(i, TT, -du_i)$. 最后对 (SS, TT) 求一次最大流即可, 当所有附加边全部满流时 (即 maxflow == 所有 $du_i > 0$ 之和) 时有可行解.

有源汇有上下界最大可行流 建立超级源点 SS 和超级汇点 TT, 首先判断是否存在可行流, 用无源汇有上下界可行流的方法判断. 增设一条从 T 到 S 没有下界容量为无穷的边, 那么原图就变成了一个无源汇有上下界可行流问题. 同样地建图后, 对 (SS, TT) 进行一次最大流, 判断是否有可行解. 如果有可行解, 删除超级源点 SS 和超级汇点 TT, 并删去 T 到 S 的这条边, 再对 (S, T) 进行一次最大流, 此时得到的 maxflow 即为有源汇有上下界最大可行流.

有源汇有上下界最小可行流 建立超级源点 SS 和超级汇点 TT, 和无源汇有上下界可行流一样新增一些边, 然后从SS到TT跑最大流. 接着加上边 (T, S, ∞), 再从 SS 到 TT 跑一遍最大流. 如果所有新增边都是满的, 则存在可行流, 此时 T 到 S 这条边的流量即为最小可行流.

有上下界费用流 如果求无源汇有上下界最小费用可行流或有源汇有上下界最小费用最大可行流, 用1.6.3.1/1.6.3.2 的构图方法, 给边加上费用即可. 求有源汇有上下界最小费用最小可行流, 要先用1.6.3.3的方法建图, 先求出一个保证必要边满流情况下的最小费用. 如果费用全部非负, 那么这时的费用就是答案. 如果费用有负数, 那么流多了可能更好, 继续做从 S 到 T 的流量任意的最小费用流, 加上原来的费用就是答案.

费用流消负环　新建超级源SS汇TT, 对于所有流量非空的负权边e, 先流满(ans+=e.f*e.c, e.rev.f+=e.f, e.f=0), 再连边SS→e.to, e.from→TT, 流量均为e.f(>0), 费用均为0. 再连边T→S流量 ∞ 费用0. 此时没有负环了. 做一遍SS到TT的最小费用最大流, 将费用累加ans, 拆掉T→S的那条边 (此边的流量为残量网络中S→T的流量). 此时负环已消, 再继续跑最小费用最大流.

整数线性规划转费用流

首先将约束关系转化为所有变量下界为 0, 上界没有要求, 并满足一些等式, 每个变量在均在等式左边且出现恰好两次, 系数为 $+1$ 和 -1, 优化目标为 $\max \sum v_i x_i$ 的形式. 将等式看做点, 等式i右边的值 b_i 若为正, 则 S 向 i 连边 $(b_i, 0)$, 否则i向T连边 $(-b_i, 0)$. 将变量看做边, 记变量 x_i 的上界为 m_i(无上界则 $m_i = inf$), 将 x_i 系数为 $+1$ 的那个等式 u 向系数为 -1 的等式 v 连边 (m_i, v_i).

9.16 Gomory-Hu 无向图最小割树 $O(V^3E)$

每次随便找两个点 s, t 求在原图的最小割, 在最小割树上连 (s, t, w_{cut}), 递归对由割集隔开的部分继续做. 在得到的树上, 两点最小割即为树上瓶颈路. 实现时, 由于是随意找点, 可以写为分治的形式.

9.17 Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$

```cpp
const int N = 601;
int f[N], siz[N], G[N][N];
int getf(int x) {return f[x] == x ? x : f[x] = getf(f[x]);}
int dis[N], vis[N], bin[N];
int n, m;
int contract(int &s, int &t) {  // Find s,t
  memset(vis, 0, sizeof(vis));
  memset(dis, 0, sizeof(dis));
  int i, j, k, mincut, maxc;
  for (i = 1; i <= n; i++) {
    k = -1; maxc = -1;
    for (j = 1; j <= n; j++)
      if (!bin[j] && !vis[j] && dis[j] > maxc) {
        k = j;
        maxc = dis[j]; }
    if (k == -1) return mincut;
    s = t; t = k; mincut = maxc; vis[k] = true;
    for (j = 1; j <= n; j++)
      if (!bin[j] && !vis[j]) dis[j] += G[k][j];
  } return mincut; }
const int inf = 0x3f3f3f3f;
int solve() {
  int mincut, i, j, s, t, ans;
  for (mincut = inf, i = 1; i < n; i++) {
    ans = contract(s, t);
    bin[t] = true;
    if (mincut > ans) mincut = ans;
    if (mincut == 0) return 0;
    for (j = 1; j <= n; j++)
      if (!bin[j]) G[s][j] = (G[j][s] += G[j][t]);
  } return mincut; }
int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; ++i) f[i] = i, siz[i] = 1;
  for (int i = 1, u, v, w; i <= m; ++i) {
    cin >> u >> v >> w;
    int fu = getf(u), fv = getf(v);
    if (fu != fv) {
      if (siz[fu] > siz[fv]) swap(fu, fv);
      f[fu] = fv, siz[fv] += siz[fu]; }
    G[u][v] += w, G[v][u] += w; }
  cout << (siz[getf(1)] != n ? 0 : solve()); }
```

9.18 弦图

弦图的定义　连接环中不相邻的两个点的边称为弦. 一个无向图称为弦图, 当图中任意长度都大于 3 的环都至少有一个弦.

单纯点　一个点称为单纯点当 $\{v\} \cup A(v)$ 的导出子图为一个团. 任何一个弦图都至少有一个单纯点, 不是完全图的弦图至少有两个不相邻的单纯点.

完美消除序列　一个序列 $v_1, v_2, ..., v_n$ 满足 v_i 在 $v_i, \cdots, v_n$ 的诱导子图中为一个单纯点. 一个无向图是弦图当且仅当它有一个完美消除序列.

最大势算法　从 n 到 1 的顺序依次给点标号. 设 label_i 表示第 i 个点与多少个已标号的点相邻, 每次选择 label 最大的未标号的点进行标号. 用桶维护优先队列可以做到 $O(n + m)$.

弦图的判定　判定最大势算法输出是否合法即可. 如果依次判断是否构成团, 时间复杂度为 $O(nm)$. 考虑优化, 设 $v_{i+1}, \cdots, v_n$ 中所有与 v_i 相邻的点依次为 $N(v_i) = \{v_{j1}, \cdots, v_{jk}\}$. 只需判断 v_{j1} 是否与 $v_{j2}, \cdots, v_{jk}$ 相邻即可. 时间复杂度 $O(n + m)$.

弦图的染色　完美消除序列从后往前染色, 染上出度的 mex.

最大独立集　完美消除序列从前往后能选就选.

团数　最大团的点数. 一般图团数 $\le$ 色数, 弦图团数 = 色数.

极大团　弦图的极大团一定为 $\{x\} \cup N(x)$.

最小团覆盖　用最少的团覆盖所有的点. 设最大独立集为 $\{p_1, \ldots, p_t\}$, 则 $\{p_1 \cup N(p_1), \ldots, p_t \cup N(p_t)\}$ 为最小团覆盖.

弦图 k 染色计数　$\prod_{v \in V} k - N(v) + 1$.

区间图　每个顶点代表一个区间, 有边当且仅当区间有交. 区间图是弦图, 一个完美消除序列是右端点排序.

```cpp
vector <int> L[N];
int seq[N], lab[N], col[N], id[N], vis[N];
void mcs() {
  for (int i = 0; i < n; i++) L[i].clear();
  fill(lab + 1, lab + n + 1, 0);
  fill(id + 1, id + n + 1, 0);
  for (int i = 1; i <= n; i++) L[0].push_back(i);
  int top = 0;
  for (int k = n; k; k--) {
    int x = -1;
    for ( ; ; ) {
      if (L[top].empty()) top --;
      else {
        x = L[top].back(), L[top].pop_back();
        if (lab[x] == top) break;
      }
    }
    seq[k] = x; id[x] = k;
    for (auto v : E[x]) {
      if (!id[v]) {
        L[++lab[v]].push_back(v);
        top = max(top, lab[v]);
      } } } }
bool check() {
  fill(vis + 1, vis + n + 1, 0);
  for (int i = n; i; i--) {
    int x = seq[i];
    vector <int> to;
    for (auto v : E[x])
      if (id[v] > i) to.push_back(v);
    if (to.empty()) continue;
    int w = to.front();
    for (auto v : to) if (id[v] < id[w]) w = v;
    for (auto v : E[w]) vis[v] = i;
    for (auto v : to)
      if (v != w && vis[v] != i) return false;
  } return true; }
void color() {
  fill(vis + 1, vis + n + 1, 0);
  for (int i = n; i; i--) {
    int x = seq[i];
    for (auto v : E[x]) vis[col[v]] = x;
    for (int c = 1; !col[x]; c++)
      if (vis[c] != x) col[x] = c;
  } }
```

9.19 Minimum Mean Cycle 最小平均值环 $O(n^2)$

```cpp
// 点标号为 1, 2, ···, n, 0为虚拟源点向其他点连权值为0的单向边.
// f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
ll f[N][N] = {Inf}; int u[M], v[M], w[M]; f[0][0] = 0;
for(int i = 1; i <= n + 1; i ++)
  for(int j = 0; j < m; j ++)
    f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);
double ans = Inf;
for(int i = 1; i <= n; i ++) {
  double t = -Inf;
  for(int j = 1; j <= n; j ++)
    t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
  ans = min(t, ans); }
```

9.20 斯坦纳树

```cpp
LL d[1 << 10][N]; int c[15];
priority_queue < pair <LL, int> > q;
void dij(int S) {
  for (int i = 1; i <= n; i++) q.push(mp(-d[S][i], i));
  while (!q.empty()) {
    pair <LL, int> o = q.top(); q.pop();
    if (-o.x != d[S][o.y]) continue;
    int x = o.y;
    for (auto v : E[x]) if (d[S][v.v] > d[S][x] + v.w) {
      d[S][v.v] = d[S][x] + v.w;
      q.push(mp(-d[S][v.v], v.v));}}}
void solve() {
  for (int i = 1; i < (1 << K); i++)
    for (int j = 1; j <= n; j++) d[i][j] = INF;
  for (int i = 0; i < K; i++) read(c[i]), d[1 << i][c[i]] =
    ↪ 0;
  for (int S = 1; S < (1 << K); S++) {
```

```
|   | for (int k = S; k > (S >> 1); k = (k - 1) & S) {
|   |   | for (int i = 1; i <= n; i++) {
|   |   |   | d[S][i] = min(d[S][i], d[k][i] + d[S ^ k][i]);
|   | } } | dij(S);}}
```

9.21 图论结论

9.21.1 最小乘积问题原理

每个元素有两个权值 $\{x_i\}$ 和 $\{y_i\}$, 要求在某个限制下 (例如生成树, 二分图匹配) 使得 $\Sigma x \Sigma y$ 最小. 对于任意一种符合限制的选取方法, 记 $X = \Sigma x_i, Y = \Sigma y_i$, 可看做平面内一点 (X, Y). 答案必在下凸壳上, 找出该下凸壳所有点, 即可枚举获得最优答案. 可以递归求出此下凸壳所有点, 分别找出距 x, y 轴最近的两点 A, B, 分别对应于 $\Sigma y_i, \Sigma x_i$ 最小. 找出距离线段最远的点 C, 则 C 也在下凸壳上, C 点满足 $AB \times AC$ 最小, 也即

$$(X_B - X_A)Y_C + (Y_A - Y_B)X_C - (X_B - X_A)Y_A - (Y_B - Y_A)X_A$$

最小, 后两项均为常数, 因此将所以权值改成 $(X_B - X_A)y_i + (Y_B - Y_A)x_i$, 求同样问题 (例如最小生成树, 最小权匹配) 即可. 求出 C 点以后, 递归 AC, BC.

9.21.2 最小环

无向图最小环: 每次floyd到 k 时, 判断 1 到 $k-1$ 的每一个 i, j:

$$\text{ans} = \min\{\text{ans}, d(i,j) + G(i,k) + G(k,j)\}.$$

有向图最小环: 做完floyd后, $d(i,i)$ 即为经过 i 的最小环.

9.21.3 度序列的可图性

判断一个度序列是否可转化为简单图, 除了一种贪心构造的方法外, 下列方法更快速. EG定理: 将度序列从大到小排序得到 $\{d_i\}$, 此序列可转化为简单图当且仅当 Σd_i 为偶数, 且对于任意的 $1 \le k \le n-1$ 满足 $\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(k, d_i)$.

9.21.4 切比雪夫距离与曼哈顿距离转化

曼哈顿转切比雪夫: $(x+y, x-y)$, 适用于一些每次只能向四联通的格子走一格的问题. 切比雪夫转曼哈顿: $(\frac{x+y}{2}, \frac{x-y}{2})$, 适用于统计距离.

9.21.5 树链的交

```
bool cmp(int a,int b){return dep[a]<dep[b];}
path merge(path u, path v){
| int d[4], c[2];
| if (!u.x||!v.x) return path(0, 0);
| d[0]=lca(u.x,v.x); d[1]=lca(u.x,v.y);
| d[2]=lca(u.y,v.x); d[3]=lca(u.y,v.y);
| c[0]=lca(u.x,u.y); c[1]=lca(v.x,v.y);
| sort(d,d+4,cmp); sort(c,c+2,cmp);
| if (dep[c[0]] <= dep[d[0]] && dep[c[1]] <= dep[d[2]])
|   | return path(d[2],d[3]);
| else return path(0, 0); }
```

9.21.6 带修改MST

维护少量修改的最小生成树, 可以缩点缩边使暴力复杂度变低. (银川 21: 求有 16 个 '某两条边中至少选一条' 的限制条件的最小生成树)

找出必须边 将修改边标 $-\infty$, 在MST上的其余边为必须边, 以此缩点.

找出无用边 将修改边标 ∞, 不在MST上的其余边为无用边, 删除之.
假设修改边数为 k, 操作后图中最多剩下 $k+1$ 个点和 $2k$ 条边.

9.21.7 差分约束

$x_r - x_l \le c$:`add(l, r, c)` $x_r - x_l \ge c$:`add(r, l, -c)`

9.21.8 李超线段树

添加若干条线段或直线 $(a_i, b_i) \to (a_j, b_j)$, 每次求 $[l, r]$ 上最上面的那条线段的值. 思想是让线段树中一个节点只对应一条直线, 如果在这个区间加入一条直线, 如果一段比原来的优, 一段比原来的劣, 那么判断一下两条线的交点, 判断哪条直线可以完全覆盖一段一半的区间, 把它保留, 另一条直线下传到另一半区间. 时间复杂度 $O(n \log n)$.

9.21.9 Segment Tree Beats

区间 min, max, 区间求和. 以区间取 min 为例, 额外维护最大值 m, 严格次大值 s 以及最大值个数 t. 现在假设我们要让区间 $[L, R]$ 对 x 取 min, 先在线段树中定位若干个节点, 对于每个节点分三种情况讨论: 1, 当 $m \le x$ 时, 显然这一次修改不会对这个节点产生影响, 直接退出; 2, 当 $se < x < ma$ 时, 显然这一次修改只会影响到所有最大值, 所以把 num 加上 $t*(x-ma)$, 把 ma 更新为 x, 打上标记退出; 3, 当 $se \ge x$ 时, 无法直接更新着一个节点的信息, 对当前节点的左儿子和右儿子递归处理. 单次操作均摊复杂度 $O(\log^2 n)$.

9.21.10 二分图

最小点覆盖=最大匹配数. 独立集与覆盖集互补. 最小点覆盖构造方法: 对二分图流图求割集, 跨过的边指示最小点覆盖. **Hall定理** $G = (X, Y, E), |M| = |X| \Leftrightarrow \forall S \subseteq X, |S| \le |A(S)|$.

9.21.11 稳定婚姻问题

男士按自己喜欢程度从高到底依次向每位女士求婚, 女士遇到更喜欢的男士时就接受他, 并抛弃以前的配偶. 被抛弃的男士继续按照列表向剩下的女士依次求婚, 直到所有人都有配偶. 算法一定能得到一个匹配, 而且这个匹配一定是稳定的. 时间复杂度 $O(n^2)$.

9.21.12 三元环

对于无向边 (u, v), 如果 $\deg_u < \deg_v$, 那么连有向边 (u, v)(以点标号为第二关键字). 枚举 x 暴力即可. 时间复杂度 $O(m\sqrt{m})$.

9.21.13 图同构

令 $F_t(i) = (F_{t-1}(i) * A + \sum_{i \to j} F_{t-1}(j) * B + \sum_{j \to i} F_{t-1}(j) * C + D * (i - a)) \bmod P$, 枚举点 a, 迭代 K 次后求得的就是 a 点所对应的 $hash$ 值, 其中 K, A, B, C, D, P 为 $hash$ 参数, 可自选.

9.21.14 竞赛图 Landau's Theorem

n 个点竞赛图点按出度按升序排序, 前 i 个点的出度之和不小于 $\frac{i(i-1)}{2}$, 度数总和等于 $\frac{n(n-1)}{2}$. 否则可以用优先队列构造出方案.

9.21.15 Ramsey Theorem R(3,3)=6, R(4,4)=18

6 个人中存在 3 人相互认识或者相互不认识.

9.21.16 树的计数 Prufer序列

树和其prufer编码一一对应, 一颗 n 个点的树, 其prufer编码长度为 $n-2$, 且度数为 d_i 的点在prufer 编码中出现 $d_i - 1$ 次.
由树得到序列: 总共需要 $n-2$ 步, 第 i 步在当前的树中寻找具有最小标号的叶子节点, 将与其相连的点的标号设为Prufer序列的第 i 个元素 p_i, 并将此叶子节点从树中删除, 直到最后得到一个长度为 $n-2$ 的Prufer 序列和一个只有两个节点的树.
由序列得到树: 先将所有点的度赋初值为 1, 然后加上它的编号在Prufer序列中出现的次数, 得到每个点的度; 执行 $n-2$ 步, 第 i 步选取具有最小标号的度为 1 的点 u 与 $v = p_i$ 相连, 得到树中的一条边, 并将 u 和 v 的度减一. 最后再把剩下的两个度为 1 的点连边, 加入到树中.
相关结论: n 个点完全图, 每个点度数依次为 d_1,d_2,...,d_n, 这样生成树的棵树为: $\frac{(n-2)!}{(d_1-1)!(d_2-1)!...(d_n-1)!}$.
左边有 n_1 个点, 右边有 n_2 个点的完全二分图的生成树棵树为 $n_1^{n_2-1} \times n_2^{n_1-1}$.
m 个连通块, 每个连通块有 c_i 个点, 把他们全部连通的生成树方案数: $(\sum c_i)^{m-2} \prod c_i$

9.21.17 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766

无标号 $a_{n+1} = 1/n \sum_{k=1}^{n} (\sum_{d|k} d \cdot a(d)) \cdot a(n-k+1)$

9.21.18 无根树计数

n 是奇数时, 有 $a_n - \sum_{i}^{n/2} a_i a_{n-i}$ 种不同的无根树.
n 时偶数时, 有 $a_n - \sum_{i}^{n/2} a_i a_{n-i} + \frac{1}{2} a_{n/2}(a_{n/2} + 1)$ 种不同的无根树.

9.21.19 生成树计数 Kirchhoff's Matrix-Tree Thoerem

Kirchhoff Matrix $T = Deg - A$, Deg 是度数对角阵, A 是邻接矩阵. 无向图度数矩阵是每个点度数; 有向图度数矩阵是每个点入度.
邻接矩阵 $A[u][v]$ 表示 $u \to v$ 边个数, 重边按照边数计算, 自环不计入度数.
无向图生成树计数: $c = |K$ 的任意1个 $n-1$ 阶主子式 $|$
有向图外向树计数: $c = |$ 去掉根所在的那阶得到的主子式 $|$

9.21.20 有向图欧拉回路计数 BEST Thoerem

$$\text{ec}(G) = t_w(G) \prod_{v \in V} (\deg(v) - 1)!$$

其中 deg 为入度 (欧拉图中等于出度), $t_w(G)$ 为以 w 为根的外向树的个数. 相关计算参考生成树计数.
欧拉连通图中任意两点外向树个数相同: $\text{t}_\text{v}(G) = \text{t}_\text{w}(G)$.

9.21.21 Tutte Matrix

Tutte matrix A of a graph $G = (V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & \text{if } (i,j) \in E \text{ and } i < j \\ -x_{ij} & \text{if } (i,j) \in E \text{ and } i > j \\ 0 & \text{otherwise} \end{cases}$$

where x_{ij} are indeterminates. The determinant of this skew-symmetric matrix is then a polynomial (in the variables x_{ij}, $i < j$): this coincides with the square of the pfaffian of the matrix A and is non-zero (as a polynomial) if and only if a perfect matching exists.

9.21.22 Edmonds Matrix

Edmonds matrix A of a balanced ($|U| = |V|$) bipartite graph $G = (U, V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & (u_i, v_j) \in E \\ 0 & (u_i, v_j) \notin E \end{cases}$$

where the x_{ij} are indeterminates. G 有完美匹配当且仅当关于 x_{ij} 的多项式 $det(A_{ij})$ 不恒为 0. 完美匹配的个数等于多项式中单项式的个数.

9.21.23 有向图无环定向, 色多项式

图的色多项式 $P_G(q)$ 对图 G 的 q-染色计数.
Triangle $K_3 : x(x-1)(x-2)$
Complete graph $K_n : x(x-1)(x-2)\cdots(x-(n-1))$
Tree with n vertices : $x(x-1)^{n-1}$
Cycle $C_n : (x-1)^n + (-1)^n(x-1)$
acyclic orientations of an n-vertex graph G is $(-1)^n P_G(-1)$.

9.21.24 拟阵交问题

拟阵定义: , S, T 是独立集, 则 S 子集是, 若 $|S| > |T|$, 则 S 能扩充 T. 最大带权拟阵交问题: 全集 U 中每个元素都有权值 w_i. 设同一个全集 U 上有两个满足拟阵性质的集族 $\mathcal{F}_1, \mathcal{F}_2$. 对于 $k = 1..|U|$, 分别求出一个集合 S, 满足 $S \in \mathcal{F}_1 \cap \mathcal{F}_2$ 且 $|S|$ 恰好为 k 的前提下, S 中元素权值和最小.

设集合大小为 k 时已经求出了答案 S. 现在希望求出集合大小为 $k+1$ 的答案. U 中所有元素分为两个集合: 当前答案集合 S, 和剩余集合 $T = U \backslash S$. 考虑 T 中的某个元素 x_i. 记 $A = \{x_i | S \cup \{x_i\} \in \mathcal{F}_1\}$, $B = \{x_i | S \cup \{x_i\} \in \mathcal{F}_2\}$. 如果 T 中某个元素 $x_i \notin A$, 说明 x_i 加进 S 中形成了某个" 环", 从而不满足 $\mathcal{F}_1$ 的限制. 考虑这个" 环" 上每个元素 y_j, 满足 $S \backslash \{y_j\} \cup \{x_i\} \in \mathcal{F}_1$, 将 y_j 向每个 x_i 连边. 如果 T 中某个元素 $x_i \notin B$, 同理找出 S 中每一个元素 y_j 使得 $S \backslash \{y_j\} \cup \{x_i\} \in \mathcal{F}_2$, 将 x_i 向 y_j 连边. 现在求出从 A 到 B 的多源多汇最短路, 权值在点上, 若点属于 T 则权值为正, 否则属于 S, 权值为负. 最短路上每个 T 中的点放进 S, S 中的点放进 T, 则完成了一次增广. 由于每次增广路的起点和终点都在 T 中, 所以每次增广都会使得 $|S|$ 增加1.

最大拟阵交问题可以去掉权值直接求增广路.

9.21.25 双极定向

```cpp
//无向图每条边定向后成为DAG, 使得s可达所有点可达t
//topo为定向后DAG的拓扑序,u->v: 拓扑序中u在v的前面.
int n, dfn[N], low[N], stp, p[N], ord[N], topo[N];
bool fail = 0, sign[N]; vector<int> e[N];
void dfs(int x, int fa, int s, int t){
  dfn[x] = low[x] = ++stp;
  ord[stp] = x, p[x] = fa;
  if (x == s) dfs(t, x, s, t);
  for (int y : e[x]){
    if (x == s && y == t) continue;
    if (!dfn[y]){
      if (x == s) fail = true;
      dfs(y, x, s, t);
      low[x] = min(low[x], low[y]); }
    else if (dfn[y] < dfn[x] && y != fa)
      low[x] = min(low[x], dfn[y]); } }
bool bipolar_orientation(int s, int t){
  e[s].push_back(t), e[t].push_back(s);
  stp = fail = 0, dfs(s, s, s, t);
  for (int i = 1; i <= n; i++)
    if (i != s && (!dfn[i] || low[i] >= dfn[i]))
      fail = true;
  if (fail) return false;
  sign[s] = 0;//memset sign[] is not necessary
  int pre[n + 5], suf[n + 5]; // list
  suf[0] = s; pre[s] = 0, suf[s] = t;
  pre[t] = s, suf[t] = n + 1; pre[n + 1] = t;
  for (int i = 3; i <= n; i++){
    int v = ord[i];
    if (!sign[ord[low[v]]]){ // insert before p[v]
      int P = pre[p[v]];
      pre[v] = P, suf[v] = p[v];
      suf[P] = pre[p[v]] = v; }
    else{ // insert after p[v]
      int S = suf[p[v]];
      pre[v] = p[v], suf[v] = S;
      suf[p[v]] = pre[S] = v; }
    sign[p[v]] = !sign[ord[low[v]]]; }
  for (int x = s, cnt = 0; x != n + 1; x = suf[x])
    topo[++cnt] = x;
  return true; }
```

9.21.26 图中的环

没有奇环的图是二分图, 没有偶环的图是仙人掌. 判定没有奇环仅用深度奇偶性判即可; 判定没有偶环的图需要记录覆盖次数判定是否存在奇环有交.

10. 离线算法（如 CDQ、莫队）

10.1 莫队二次离线

```cpp
for(int i=1;i<=n;i++) {sum[i] = query(arr[i]) + sum[i-1]; add(i);}
sort(Q+1, Q+1+m, cmp);
int l = 1, r = 0;
for(int i=1;i<=m;i++){
  int ql = Q[i].l, qr = Q[i].r;
  if(l > ql) v[r].push_back({ql, l-1, i}), Q[i].ans -= sum[l-1] - sum[ql-1], l = ql;
  if(r < qr) v[l-1].push_back({r+1, qr, -i}), Q[i].ans += sum[qr] - sum[r], r = qr;
  if(l < ql) v[r].push_back({l, ql-1, -i}), Q[i].ans += sum[ql-1] - sum[l-1], l = ql;
  if(r > qr) v[l-1].push_back({qr+1, r, i}), Q[i].ans -= sum[r] - sum[qr], r = qr;
}
// 清空贡献
for(int i=1;i<=n;i++){
  add(i);
  for(auto& [l, r, id] : v[i]){
    ll tmp = 0;
    for(int j=l;j<=r;j++){
      tmp += query(arr[j]);
      // 注意如果j <= i且会产生重复贡献的时候要扣去
    }
    if(id > 0) Q[id].ans += tmp;
    else Q[-id].ans -= tmp;
  }
}
for(int i=1;i<=m;i++) Q[i].ans += Q[i-1].ans;
for(int i=1;i<=m;i++) ans[Q[i].id] = Q[i].ans;
```

11. 随机化与博弈论

11.1 Python Hint

```python
def RandomAndList():
  import random
  random.normalvariate(0.5, 0.1)
  l = [str(i) for i in range(9)]
  sorted(l), min(l), max(l), len(l)
  random.shuffle(l)
  l.sort(key=lambda x:x ^ 1,reverse=True)
  from functools import cmp_to_key
  l.sort(key=cmp_to_key(lambda x, y:(y^1)-(x^1)))
  from itertools import *
  for i in product('ABCD', repeat=2):
    pass # AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD
  for i in permutations('ABCD', repeat=2):
    pass # AB AC AD BA BC BD CA CB CD DA DB DC
  for i in combinations('ABCD', repeat=2):
    pass # AB AC AD BC BD CD
  for i in combinations_with_replacement('ABCD', repeat=2):
    pass # AA AB AC AD BB BC BD CC CD DD
def FractionOperation():
  from fractions import Fraction
  a.numerator, a.denominator, str(a)
  a = Fraction(0.233).limit_denominator(1000)
def DecimalOperation():
  from decimal import Decimal, getcontext, FloatOperation
  getcontext().prec = 100
  getcontext().rounding = getattr(decimal, 'ROUND_HALF_EVEN')
  # default; other: FLOOR, CELILING, DOWN, ...
  getcontext().traps[FloatOperation] = True
  Decimal((0, (1, 4, 1, 4), -3)) # 1.414
  a = Decimal(1<<31) / Decimal(100000)
  print(f"{a:.9f}") # 21474.83648
  print(a.sqrt(), a.ln(), a.log10(), a.exp(), a ** 2)
def Complex():
  a = 1-2j
  print(a.real, a.imag, a.conjugate())
def FastIO():
  import sys, atexit, io
  _INPUT_LINES = sys.stdin.read().splitlines()
  input = iter(_INPUT_LINES).__next__
  _OUTPUT_BUFFER = io.StringIO()
  sys.stdout = _OUTPUT_BUFFER
  @atexit.register
  def write():
    sys.__stdout__.write(_OUTPUT_BUFFER.getvalue())
```

11.2 Hacks: 指令集, 读入优化, Bitset, builtin, 随机种子

```cpp
//fast = O3 + ffast-math + fallow-store-data-races
#pragma GCC optimize("Ofast")
//下: 加include后面|    位运算    |浮点|  SIMD    | 优化寄存器 |
#pragma GCC target("abm,bmi,bmi2,fma,sse2,avx2,tune=native")
const int SZ = 1 << 16; int getc() {
  static char buf[SZ], *ptr = buf, *top = buf;
  if (ptr == top) {
    ptr = buf, top = buf + fread(buf, 1, SZ, stdin);
    if (top == buf) return -1; }
  return *ptr++; }
idx=b._Find_first();idx!=b.size();idx=b._Find_next(idx);
struct HashFunc{size_t operator()(const KEY &key)const{}};
__builtin_uaddll_overflow(a, b, &c) // binary big int
void GospersHack(int k, int n) {
  for (int s = (1 << k) - 1, c, r; s < (1 << n);
  c = s & -s, r = s + c, s = (((r ^ s) >> 2) / c) | r); }
chrono::steady_clock::now().time_since_epoch().count();
```

11.3 试机赛与纪律文件

- —入场检查— 检查身份证件: 胸牌。
- 检查: 手机、智能手表、金属（钥匙）等等。
- 完整测试键盘、鼠标、显示器和座椅。热身赛发现联系工作人员。
- —环境检查— 测试所有可能的提交方式, 找到所有题面、排行榜位置。

- 测试 `-fsanitize address, undefined, define _GLIBCXX_DEBUG`
- 测试 `time` 命令是否能显示内存占用。`/usr/bin/time -v ./a.out`
- **—OJ检查—** 测试编译器版本。C++20 `cin >> (s + 1)`; C++17 `auto [x, y]: a`; C++14 `[](auto x, auto y)`; C++11 `auto`; `bits/stdc++.h`; `pb_ds`。

```cpp
#include <ext/rope>
using namespace __gnu_cxx;
rope <int> R; R.insert(y, x); R[x]; R.erase(x, 1);
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree <int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0); s.order_of_key(5);
```

- `__int128`, `__float128`, `long double`。
- pragma target CE; 代码长度限制 (256K)，尝试 NO-OUTPUT, OLE, MLE。
- `for i in 1..11; do pc2submit a.cpp -y; done` 有没有提交限制。默认一分钟提交 10 次；优先队列，上一发没有测完，下一发优先级 -1。
- clock() 是否能够正常工作; 测试本地性能与提交性能。

```cpp
const int N = 1 << 20;
for (int T = 4; T; T--) {
    int a[N], b[N], c[N];
    for (int i = 0; i < N; i++) a[i] = i, b[i] = N - i;
    ntt_init(N); ntt(a, N, 1); ntt(b, N, 1);
    for (int i = 0; i < N; i++) c[i] = a[i] * b[i];
    ntt(c, N, -1); assert (c[0] == 0xad223d); }
// 机房: 190ms; (1s T = 21)
// QOJ: 340+-40ms; (1s T = 11.9)
// CF: 1050+-100ms; (1s T = 3.9)
```

```python
import sys
sys.set_int_max_str_digits(0) # >= Python 3.11
T = 3
for _ in range(T): # 本地 assert failed 检查下数字抄错没
    assert str(3 ** 100000 * 10 ** 100000)[:9] == "133497141"
# 机房 Pypy: 130ms; (1s T = 39)
# QOJ: 201ms; (1s T = 16)
# CF Python3: 1100ms; (1s T = 2.9) Pypy3: 310ms; (1s T = 12)
```

1. **—提交前检查—** 保存, 编译, 测过样例，测过 n=0/1；
2. 数组大小: N/M；时间空间限制；输入实数？MOD 多少？开 LL
3. 调用 **init**, 清空时数组大小 (0, 1, n 还是 m, n+1)
4. 输出取模取到正值，输出格式
5. 多组没读入完就 break
6. 关流同步, 注意不要关流同步混用
7. 加上需要怀疑的 `assert`
8. 开 `Ofast`，位运算开 `target("abm")`

1. **—提交后检查—** 变量复用后忘记改回去
2. sum 多组输入: 应只用了输入内容 (memset TL, 错下标 WA), 用前清空还是用后
3. int/LL 溢出, INF/-1 大小，浮点数 eps 和 = 0, `__builtin_popcountll`
4. 类似 `pair <LL, int> x = pair <int, int>();` 不会报警告
5. 离散化与二分: `lower_bound upper_bound` +-1, begin(), end()
6. 自定义排序：排序方向，比较函数为小于，考虑坏点: 如叉积 (0, 0)
7. 样例是否为对称/回文的？考虑构造不对称的情况，正序逆序
8. 复制过的代码对应位置正确修改
9. 检查 shadow 变量重名
10. 分类讨论: if 嵌套, break 位置
11. 图: 边标号初始是否为 1, 单双向边, 反向边边权
12. 几何: 共线, sqrt(-0.0), nan / inf, 重点, 除零 det/dot, 旋转方向
13. 浮点数：eps 量纲：应当至少取到（最短线段 × 精度要求）。
14. 二分，三分：次数。

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\mathrm{LCM}(1 \dots n)$	P_n
2	0.30102999	2	2	2	2
3	0.47712125	6	3	6	3
4	0.60205999	24	6	12	5
5	0.69897000	120	10	60	7
6	0.77815125	720	20	60	11
7	0.84509804	5040	35	420	15
8	0.90308998	40320	70	840	22
9	0.95424251	362880	126	2520	30
10	1	3628800	252	2520	42
11	1.04139269	39916800	462	27720	56
12	1.07918125	479001600	924	27720	77
15	1.17609126	1.31e12	6435	360360	176
20	1.30103000	2.43e18	184756	232792560	627
25	1.39794001	1.55e25	5200300	26771144400	1958
30	1.47712125	2.65e32	155117520	1.444e14	5604
P_n	37338_{40}	204226_{50}	966467_{60}	190569292_{100}	$1e9_{114}$

$n \le$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\arg\max d$	6	60	840	7560	83160	720720
$\pi(n)$	4	25	168	1229	9592	78498
$n \le$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\arg\max d$	8648640	73513440	×10	×9.5	×14	...
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \le$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\arg\max d$	1e18: $897612484786617600 = 2^8 3^4 5^2 7^2 11 \dots 37$					
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

```bash
export CXXFLAGS='-g -Wall -Wextra -Wconversion -Wshadow -std=gnu++20'
ulimit -s 1048576
```